République Algérienne Démocratique et Populaire

الجمهوريـــة الجـزائريـة الديمقراطيـة الشعبيـة

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

وزارة التعليـم العـالي والبـحث العلـمي

ECOLE NATIONALE
SUPERIEURE
D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي
(المعهد الوطني للتكوين في الإعلام العالي سابقا)
Ecole Nationale Supérieure d'Informatique

Mémoire de fin d'études

Pour l'obtention du diplôme d'Ingénieur d'Etat en Informatique

Option : Systèmes Informatiques (SIQ)

Thème

Sélection d'attributs pour la classification supervisée en utilisant la métaheuristique BSO hybridée avec l'apprentissage par renforcement

Réalisé par :

- REMACHE Amine Riad
- KARECH Mehdi Nedjemeddine

Encadré par :

- Mme. SADEG Souhila, ESI
- Mme. HAMDAD Leila, ESI

Promotion : 2018/2019

Dédicaces

As little kids, we all had our own naive dreams, but as grow older, we realize that the real world won't fit your ambition if you don't wake up.

Since I was just a small boy, I had that thirst for distinction among my peers, because I always thought that if we were all alike, what could be your motivation to wake up in the morning ?

Throughout my route, I made sure to bond with my people, so that they would remember me, even years after, and not just by name, but with something personal we both share.
This work is dedicated essentially to the little child who's still alive in me, who never stopped dreaming and never imagined that we'd be where we are today. I would also dedicate this work to every teacher that left an impact on me and helped me shape my personality, from my first Quran teacher, "*Chikh Nouri*", who taught me how to read and write at the age of 3, passing by *Mrs. Ouhachi*, who taught us that education isn't about knowledge only and showed us the true meaning of extracurricular activities. My love for mathematics began with *Mrs. Ouabadi*, and I'm still using the basic concepts I learned from her till now, without forgetting Mr. Faizi, the math teacher with his famous punchline : "*Grib el BAC*".

During my five years at ESI, I've had my ups and downs, but I will never forget the day *Mrs. Ansour* made me feel special without realizing it, and made me become one of the top students of her class. Another day I won't forget is the day we heard about the decease of *Mr. Ait Aoudia*, a man that most of my friends and I consider the best teacher we've ever had. My journey at ESI would never be the same without me being a member of the Club Scientifique de ESI, widely known as: *CSE*, it was my second family with whom I spend even more time and never thought that a day to give farewell like this would come.

We often say that we keep the best for the end, and by that, I mean my parents. I wish my father, may Allah have mercy on him, could have been here to live these moments, and I hope this work would've made him proud. And to my mother, may Allah prolong her life and give her the strength, who have always been there for me, and shared with me my best and worst moments. And without forgetting my younger two brothers and sister, may Allah protect them, they were most of the time my motivation to make them have an easier life.

This work is just a step among many others towards self-accomplishment, and it only made my motivation and desire for achievements even bigger.

Amine Riad Remache

Dédicaces

A mes chers parents, pour tous leurs sacrifices, leur amour, leur soutien et leurs prières tout au long de mes études,
A mes chers frères, pour leur appui,
A ma sœur pour sa tendresse et ses coups de pousse
A mon grand-père disparu durant mes années à l'école puisse Dieu le tout puissant l'avoir en sa sainte miséricorde.
A Younes, Karim, Chemssou et Amine avec lesquels j'ai partagé le meilleur et le pire, pour leur motivation, leur présence et leurs encouragements
A tous les membres du Club Scientifique de l'ESI
Que ce travail soit le fuit de votre soutien infaillible,
Merci d'être toujours là pour moi.

Mehdi Nedjemeddine Karech

Remerciements

يقول تعالى في كتابه الكريم: " لَئِن شَكَرْتُمْ لَأَزِيدَنَّكُمْ "
فالحمد لله الذي وفقنا لهذا ولم نكن لنصل إليه لولا فضله علينا
وعن أبي هريرة رضي الله عنه، عن النبي صلى الله عليه وسلم: " مَنْ لمْ يشْكُر النَّاسَ لَمْ يشْكُر الله "

Nous tenons à remercier nos encadrantes, Mme Sadeg Souhila et Mme Hamdad Leila, de nous avoir encadré pour notre projet de fin d'études et dernière année de notre cursus à l'ESI. Nous les remercions profondément pour leurs encouragements continus, disponibilité et leurs critiques constructives qui nous ont permis d'évoluer et nous initier à la contribution au domaine de la recherche.

Nous tenons à remercier aussi, Mme AIT ALI YAHIA Dahbia qui était toujours disponible pour nous écouter et nous orienter, et pour le rôle crucial qu'elle joue dans la réussite et le bon déroulement de cette année malgré toutes les circonstances qu'a connu notre pays.

Nous souhaitons également exprimer notre sincère gratitude à nos familles qui nous ont constamment soutenus. Un GRAND merci pour leurs encouragements et leur compréhension en particulier nos chers parents. De même nous voudrions remercier nos amis qui ont toujours su apporter de la fraicheur dans notre vie.

Enfin, nous adressons nos plus sincères remerciements à tous ceux qui ont participé de près ou de loin à la réalisation de ce modeste travail.

Résumé

L'optimisation combinatoire est un domaine qui englobe, entre autres, des problèmes dits problèmes d'optimisation combinatoire ainsi que leurs méthodes de résolution. Ces problèmes se caractérisent par le fait qu'ils sont formulés d'une manière parfois très facile, mais l'énorme espace dans lequel se trouve les solutions de ces problèmes rend l'opération d'exploitation de la totalité de l'espace de recherche presque impossible, vu le temps nécessaire pour cela.

L'apprentissage automatique a connu une avancée sans précédent durant ces deux dernières décennies, et parmi ses diverses techniques, l'apprentissage par renforcement, qui a fait ses preuves en 2015, en battant le champion du monde du jeu *GO*, un jeu encore plus complexe que les jeux d'échec.

L'utilisation de l'apprentissage par renforcement permet de ne pas explorer cet espace de recherche totalement, mais explorer seulement une partie de cet espace de manière intelligente, en essayant de trouver des solutions de bonne qualité en un temps raisonnable.

Ce travail s'inscrit dans le cadre des métaheuristiques hybrides, précisément, l'intégration de l'apprentissage automatique aux métaheuristiques, une branche qui est de plus en plus sollicitée pour produire des méthodes hybrides généralement plus performantes.

Notre objectif à travers ce travail, est l'intégration de l'apprentissage par renforcement dans une métaheuristique appelée *BSO* (**B**ee **S**warm **O**ptimization) pour la résolution du problème de sélection d'attributs, une métaheuristique qui fournit des solutions de très bonne qualité, mais qui requiert un temps considérable à cause de la recherche locale qu'effectuent les abeilles. L l'intérêt de concevoir des versions hybrides de cette métaheuristique avec des méthodes d'apprentissage par renforcement, notamment, l'algorithme de *Q-Learning*, est d'essayer de maintenir des qualités de solutions aussi bonnes que celles fournies par BSO tout en accélérant le processus de recherche en le rendant guidé et adaptif.

Mots clés : Métaheuristiques hybrides, Apprentissage Automatique, Apprentissage par Renforcement, Optimisation Combinatoire, Sélection d'Attributs, Q-Learning.

Abstract

Combinatorial optimization is a domain that encompasses, among other things, problems called combinatorial optimization problems as well as their resolution methods. These problems are characterized by the fact that they are formulated in a sometimes very easy way, but the enormous space in which the solutions of these problems are found makes the exploitation operation of the totality of the research space almost impossible, given the time required for this.

Machine learning has seen unprecedented progress in the last two decades, and among its various techniques, reinforcement learning, which proved its worth in 2015, beating the GO game world champion, an even more challenging and complex game than chess.

The use of reinforcement learning makes it possible not to explore this research space totally, but to explore only part of that space in an intelligent way, trying to find good quality solutions in a reasonable time.

This work is part of hybrid metaheuristics, specifically, the integration of machine learning metaheuristics, a branch that is increasingly sought to produce hybrid methods generally more efficient.

Our goal through this work, is the integration of reinforcement learning in a metaheuristic called BSO (Bee Swarm Optimization) for the resolution of the problem of selection of attributes, a metaheuristic that provides very good quality solutions, but which takes a considerable amount of time because of the local research done by bees. The interest of designing hybrid versions of this metaheuristic with reinforcement learning methods, in particular the Q-Learning algorithm, is to try to maintain the quality of solutions as good as those provided by BSO while accelerating the research process by making it guided and adaptive.

Keywords: Hybrid metaheuristics, Machine Learning, Reinforcement Learning, Combinatorial Optimization, Feature Selection, Q-Learning.

ملخص

التحسين التجميعي هو مجال يشمل، من بين أمور أخرى، مشاكل تسمى مشكلات التحسين التوافقية بالإضافة إلى أساليب حلها. تتميز هذه المشكلات بحقيقة أنها رغم صيغتها بطريقة سهلة للغاية في بعض الأحيان، ولكن المساحة الهائلة التي توجد فيها حلول هذه المشاكل تجعل عملية استغلال مجمل مساحة البحث يكاد يكون مستحيلا، بالنظر إلى الوقت اللازم لهذا الغرض.

شهد التعلم الآلي تقدمًا غير مسبوق في العقدين الأخيرين، ومن بين أساليبه المختلفة، التعلم التعزيزي، الذي أثبت أهميته عام 2015، بفوزه على بطل العالم في لعبة **جو**، وهي لعبة أكثر تحديا وتعقيدا من لعبة الشطرنج.

يجعل استخدام التعلم التعزيزي من الممكن عدم استكشاف هذه المساحة البحثية بالكامل، ولكن استكشاف جزء فقط من تلك المساحة بطريقة ذكية، في محاولة لإيجاد حلول جيدة النوعية في وقت معقول.

هذا العمل هو جزء من التجريبيات الشاملة الهجينة، وتحديدا، دمج التجريبيات الشاملة مع التعلم الآلي، وهو الفرع الذي يسعى بشكل متزايد لإنتاج أساليب تهجين بشكل عام أكثر كفاءة.

هدفنا من خلال هذا العمل، هو دمج التعلم التعزيزي في تجريبية شاملة تسمى BSO (Bee Swarm Optimization) لحل مشكلة اختيار السمات، والتي هي تجريبية شاملة توفر حلول ذات نوعية جيدة جدا ولكن فيما يتعلق بوقت التنفيذ، تعتبر بطيئة بسبب البحوث المحلية التي قامت بها النحل، ومن هنا تكمن أهمية تصميم نسخ هجينة من هذه التجريبية الشاملة مع أساليب التعلم التعزيزي، ولا سيما خوارزمية Q-Learning ،بينما تحاول الحفاظ على صفات الحلول الجيدة مثل تلك التي تقدمها BSO .

كلمات مفتاحية: التجريبيات الشاملة الهجينة، التعلم الآلي، التعلم التعزيزي، التحسين التجميعي، اختيار السمات، Q-Learning .

Table des matières

Table des matières ..... **VII**

Liste des équations ..... **IX**

Liste des figures ..... **X**

Liste des tableaux ..... **XII**

Liste des abréviations ..... **XIII**

Introduction générale **1**

I. Chapitre 1 : Le problème de sélection d'attributs **3**

1. Introduction 3
2. La sélection d'attributs 3
3. Le processus de sélection d'attributs 4
4. Les principales approches de sélection d'attributs 10
5. Les métaheuristiques pour la sélection d'attributs 12
6. BSO pour la sélection d'attributs 19
7. Conclusion 22

II. Chapitre 2 : L'apprentissage par renforcement **23**

1. Introduction 23
2. Définitions 24
3. Notions de base de l'A/R 27
4. Les approches 30
5. Les applications 39
6. Conclusion 41

III. Chapitre 3 : L'intégration de l'A/R aux métaheuristiques **42**

1. Introduction 42
2. Motivations 42
3. Intégration de l'apprentissage par renforcement aux métaheuristiques 44
4. Etude comparative des travaux 46

5. Conclusion 48
IV. Chapitre 4 : Conception et réalisation 49
1. Introduction 49
2. Hybridation de BSO avec l'A/R 49
3. Réalisation 62
4. Conclusion 68
V. Chapitre 5 : Tests et résultats 69
1. Introduction 69
2. Présentation de l'environnement de tests 69
3. Présentation du plan de tests 72
4. Présentation et interprétation des résultats des tests 75
5. Conclusions par rapport aux interprétations 107
6. Conclusion 108
Conclusion générale et perspectives 109
Bibliographie 111
Webographie 116
Annexe A : Gestion de projet i
Annexe B : Outils & notions théoriques viii
1. L'environnement Colaboratory de Google viii
2. Analyse de variance (ANOVA) viii
3. Le classifieur KNN xiii
4. Validation croisée xv
5. Classification des métaheuristiques hybrides xvii

Liste des équations

Équation 1 20
Équation 2 30
Équation 3 30
Équation 4 33
Équation 5 33
Équation 6 34
Équation 7 34
Équation 8 37
Équation 9 37
Équation 10 52

Liste des figures

Figure I-1 - Processus de sélection d'attributs (Dash & Liu, 1997) 4
Figure I-2 - L'approche enveloppante 10
Figure I-3 - Contenu de la boite noire 10
Figure I-4 - L'approche par filtre 11
Figure I-5 - Classification des méthodes de résolution 18
Figure II-1 - Comparaison entre l'A/R et l'apprentissage supervisé 27
Figure II-2 - L'interaction entre l'agent et l'environnement 27
Figure II-3 - Taxonomie des approches de l'A/R 38
Figure III-1 - Nombre de publications selon google scholar pour la requête : « reinforcement learning and metaheuristics » 43
Figure III-2 - Nombre de citations pour les 10 dernières années selon google scholar pour la requête : « reinforcement learning and metaheuristics » 43
Figure IV-1 – Le processus de recherche de BSO 50
Figure IV-2 - Q-Table ordinaire 52
Figure IV-3 - Q-Table adaptée 53
Figure IV-4 - Exemple QLS_1 54
Figure IV-5 - Exemple du XOR 56
Figure IV-6 - Représentation d'une entrée de la Q-Table de QsBSO-FS 59
Figure IV-7 – L'ancienne initialisation de refSol 61
Figure IV-8 - La nouvelle approche pour initialiser refSol 61
Figure IV-9 - Architecture générale du système 66
Figure IV-10 - Diagramme de classe 66
Figure V-1 - Le paramètre nombre d'abeilles pour iQBSO-FS 77
Figure V-2 - Le paramètre flip pour iQBSO-FS 78
Figure V-3 - Le paramètre MaxChances de iQBSO-FS 79
Figure V-4 - Le paramètre taux d'apprentissage pour iQBSO-FS 81
Figure V-5 - Le paramètre facteur de dépréciation pour iQBSO-FS 82
Figure V-6 - Le paramètre taux d'exploration pour iQBSO-FS 84
Figure V-7 - Le paramètre nombre d'abeilles pour sQBSO-FS 85
Figure V-8 - Le paramètre flip pour sQBSO-FS 87
Figure V-9 - Le paramètre MaxChances pour sQBSO-FS 88
Figure V-10 - Le paramètre taux d'apprentissage pour sQBSO-FS 90

Figure V-11 - Le paramètre facteur de dépréciation pour sQBSO-FS 91
Figure V-12 - Le paramètre taux d'exploration pour sQBSO-FS 93
Figure V-13 - Le paramètre nombre d'abeilles pour QsBSO-FS 94
Figure V-14 - Le paramètre MaxIterations pour QsBSO-FS 96
Figure V-15 - Le paramètre taux d'apprentissage pour QsBSO-FS 97
Figure V-16 - Le paramètre facteur de dépréciation pour QsBSO-FS 99
Figure V-17 - Le paramètre taux d'exploration pour QsBSO-FS 100
Figure V-18 - Comparaison entre le temps d'exécution de sQBSO-FS et iQBSO-FS 103
Figure V-19 - Comparaison du temps d'exécution pour BSO-FS et sQBSO-FS 105
Figure V-20 - Comparaison du nombre de solutions évalué par BSO-FS et sQBSO-FS 106
Figure Annexe B-1 - Exemple de l'algorithme KNN xiv
Figure Annexe B-2 - Classification hiérarchique xvii
Figure Annexe B-3 - Taxinomie des métaheuristiques hybrides selon les problèmes de conception xix
Figure Annexe B-4 - Taxonomie des problèmes liés à l'implémentation xx
Figure Annexe B-5 - Méthodes d'intégration de la connaissance dans une métaheuristique . xxi

Liste des tableaux

Tableau I-1 - Comparaison entre différents types de fonctions d'évaluations 8
Tableau I-2 - Taxonomie des algorithmes de sélection d'attributs 12
Tableau II-1 - Comparaison apprentissage basé/sans modèle 29
Tableau II-2 - Comparaison exploration et exploitation 29
Tableau II-3 - Tableau comparatif des approches de l'A/R 35
Tableau III-1 - Le nombre de publications pour l'hybridation A/R et métaheuristiques 42
Tableau III-2 - Tableau comparatif des travaux 48
Tableau IV-1 - Table de vérité : OU exclusif 56
Tableau IV-2 - Description des classes 67
Tableau V-1 - Datasets pour l'étude empirique 70
Tableau V-2 - Datasets supplémentaires pour l'étude comparative 70
Tableau V-3 - Plan de tests pour iQBSO-FS 74
Tableau V-4 - Plan de tests pour sQBSO-FS 74
Tableau V-5 - Plan de tests pour QsBSO-FS 75
Tableau V-6 - ANOVA pour l'effet du nombre d'abeilles (iQBSO-FS) 76
Tableau V-7 - ANOVA pour l'effet de flip (iQBSO-FS) 77
Tableau V-8 - ANOVA pour l'effet de MaxChances (iQBSO-FS) 79
Tableau V-9 - ANOVA pour l'effet du taux d'apprentissage (iQBSO-FS) 80
Tableau V-10 - ANOVA pour l'effet du facteur de dépréciation (iQBSO-FS) 81
Tableau V-11 - ANOVA pour l'effet du facteur d'exploration (iQBSO-FS) 83
Tableau V-12 - ANOVA pour l'effet du nombre d'abeilles (sQBSO-FS) 84
Tableau V-13 - ANOVA pour l'effet de flip (sQBSO-FS) 86
Tableau V-14 - ANOVA pour l'effet de MaxChances (sQBSO-FS) 87
Tableau V-15 - ANOVA pour l'effet du taux d'apprentissage (sQBSO-FS) 89
Tableau V-16 - ANOVA pour l'effet du facteur de dépréciation (sQBSO-FS) 90
Tableau V-17 - ANOVA pour l'effet du facteur d'exploration (sQBSO-FS) 92
Tableau V-18 - ANOVA pour l'effet du nombre d'abeilles (QsBSO-FS) 93
Tableau V-19 - ANOVA pour l'effet de MaxIterations (QsBSO-FS) 95
Tableau V-20 - ANOVA pour l'effet du taux d'apprentissage (QsBSO-FS) 96
Tableau V-21 - ANOVA pour l'effet du facteur de dépréciation (QsBSO-FS) 98
Tableau V-22 - ANOVA pour l'effet du facteur d'exploration (QsBSO-FS) 99
Tableau V-23 - Meilleurs paramètres pour iQBSO-FS 101

Tableau V-24 - Meilleurs paramètres pour sQBSO-FS 101
Tableau V-25 - Meilleurs paramètres pour QsBSO-FS 101
Tableau V-26 - Tableau comparatif des résultats des hybridations 102
Tableau V-27 - Comparaison entre BSO-FS et sQBSO-FS 105
Tableau V-28 - Comparaison avec les travaux existants 107

Liste des abréviations

AI	Artificial Intelligence
AG	Algorithme Génétique
A/R	Apprentissage par Renforcement
B&B	Branch and Bound
BSO	Bee Swarm Optimization
OC	Optimisation Combinatoire
DP	Dynamic Programming
DRL	Deep Reinforcement Learning
FS	Feature Selection
IA	Intelligence Artificielle
IP	Itération de Politique
IV	Itération de Valeur
MC	Monte Carlo
ML	Machine Learning
MDP	Markov Decision Process
POC	Problème d'Optimisation Combinatoire
RO	Recherche Opérationnelle
RL	Recherche Locale
TD	Temporal Differences
VGA	**V**éhicules à **G**uidage **A**utomatique

Introduction générale

« *Car sans généralisation, la prévision est impossible. Les circonstances où l'on a opéré ne se reproduiront jamais toutes à la fois. Le fait observé ne recommencera donc jamais ; la seule chose que l'on puisse affirmer, c'est que dans des circonstances analogues, un fait analogue se produira. Pour prévoir il faut donc au moins invoquer l'analogie, c'est-à-dire déjà généraliser* » - Henri Poincaré

Les métaheuristiques sont des méthodes approchées très utilisées dans la résolution de problèmes d'optimisation combinatoire difficiles car elles permettent d'obtenir des résultats de très bonne qualité en un temps raisonnable. Cependant, les très grandes instances issues d'applications réelles restent difficiles à résoudre en un temps acceptable même en utilisant des métaheuristiques. De ce fait, les chercheurs s'orientent vers la conception de métaheuristiques hybrides qui donnent généralement de meilleurs résultats que les métaheuristiques classiques car elles combinent plusieurs méthodes et de ce fait profitent des avantages de chacune d'elles.

Ces dernières années, beaucoup de travaux proposent des métaheuristiques intégrant des techniques d'apprentissage automatique dans le but d'exploiter la connaissance extraite de l'instance à résoudre avant ou pendant sa résolution afin de guider le processus de recherche. Nous nous intéressons dans ce travail de projet de fin d'études à l'utilisation de l'apprentissage par renforcement pour améliorer les performances d'une métaheuristique appelée *BSO* (**B**ee **S**warm **O**ptimization) pour la résolution du problème de sélection d'attributs, une métaheuristique qui fournit des solutions de très bonne qualité, d'où l'intérêt de concevoir des versions hybrides de cette métaheuristique avec des méthodes d'apprentissage par renforcement, notamment, l'algorithme de *Q-Learning*, tout en essayant de maintenir des qualités de solutions aussi bonnes que celles fournies par BSO.

L'apprentissage par renforcement est «*une manière de programmer des agents par récompenses et sanctions sans avoir besoin de spécifier comment une tâche doit être effectuée*» - (Kaelbling, Littman, & Moore, 1996). L'algorithme de Q-Learning se distingue parmi les méthodes d'apprentissage par renforcement pour être le plus adapté à notre cas d'étude, et le fait d'être parmi les algorithmes d'A/R les plus stables.

Dans ce mémoire, nous consacrons le premier chapitre à la définition du problème de sélection d'attributs et nous présentons les principales approches de résolution, notamment, la métaheuristique BSO. Dans le second chapitre, nous introduisons l'apprentissage par renforcement, ses différentes approches et ses applications pour des problèmes réels. Dans le troisième chapitre, Nous nous intéressons à l'intégration de l'apprentissage par renforcement aux métaheuristiques, et nous présenterons une classification des métaheuristiques hybrides en général, ainsi que la relation entre les métaheuristiques et l'apprentissage par renforcement et les opportunités d'apprentissage. Ce chapitre est clôturé par une étude comparative des principaux travaux traitant de cette thématique. Le quatrième chapitre est consacré à la présentation de la conception et la réalisation des métaheuristiques proposées dans le cadre de notre contribution. Un cinquième chapitre est dédié à la partie des tests et résultats des méthodes proposées et les interprétations de ces dernières. Nous clôturerons ce mémoire avec une conclusion générale et des perspectives pour des travaux futures basés sur notre contribution.

I. Chapitre 1 : Le problème de sélection d'attributs

1. Introduction

L'utilisation des attributs est très importante pour caractériser un domaine et mieux cerner le problème, mais certain de ces attributs s'avèrent redondants ou bien non pertinents dans la tâche visée. Un attribut non pertinent est un attribut qui affecte peu ou pas du tout la tâche visée, tandis qu'un attribut redondant est un attribut qui est fortement corrélé avec un ou plusieurs autres attributs, et de ce fait ces deux types d'attributs apportent peu ou pas du tout d'informations supplémentaires.

La sélection d'attributs s'intéresse à fournir des sous-ensembles d'attributs pertinents et non redondants pour une tâche donnée. Dans le cadre de notre projet de fin d'étude, nous nous intéressons à la sélection d'attributs pour faire une classification supervisée.

Le reste du chapitre est organisé comme suit : Nous définirons de la sélection d'attributs en section 2. Nous consacrons la section 3 au processus de sélection d'attributs et étudierons plus en détail ses différentes étapes. Nous discuterons, en section 4, les approches les plus utilisées et ensuite on va définir brièvement la classification supervisée en section 5. Enfin, avant de conclure, on parlera du cœur de notre sujet : la métaheuristique BSO pour la sélection d'attributs en section 6.

2. La sélection d'attributs

La sélection d'attributs consiste à extraire un sous ensemble parmi les attributs d'un ensemble de grande taille. Le but étant de trouver le meilleur sous ensemble d'attributs pour la tâche visée. Les principales motivations derrière la sélection d'attributs (Jain & Zongker, 1997), (Yang & Honavar, 1998) :

1. La réduction du nombre d'attributs permet une meilleure compréhension du phénomène étudié, et cela en éliminant les attributs source de bruits.
2. Les ensembles de donnée réduits permettent d'éviter le sur-apprentissage à l'aide d'une meilleure généralisation des données.
3. L'utilisation de sous-ensembles permet de réduire le coût d'apprentissage à savoir les temps d'apprentissage et d'exécution.

Il existe plusieurs définitions de la sélection d'attributs. (Dash & Liu, 1997) ont formulé une définition très intéressante de la sélection d'attributs :
La sélection d'attributs s'intéresse à construire un sous ensemble d'attributs, de taille minimale, et répondant à deux critères :

- L'exactitude de la classification ne décroit pas de façon significative
- La distribution résultante obtenue à partir du sous-ensemble d'attributs sélectionné est très proche de la distribution obtenue en utilisant tout l'ensemble d'attributs.

Ce deuxième critère signifie qu'en passant de l'ensemble d'attributs d'origine au sous-ensemble d'attributs construit, nous aurons peu ou pas de pertes d'informations. Théoriquement, le sous ensemble d'attributs ainsi constitué, donnera les meilleurs résultats. C'est une technique de prétraitement, utilisée pour les ensembles de données très grands.

3. Le processus de sélection d'attributs

En 1997, (Dash & Liu, 1997) ont proposé un modèle de résolution du problème de sélection d'attributs qui se compose essentiellement en 4 étapes :

- **Procédure de génération** : Pour générer les sous-ensembles d'attributs qui vont être évalués ;
- **Fonction d'évaluation** : Pour mesurer la qualité du sous-ensemble généré ;
- **Critère d'arrêt** : Il peut être utilisé ou pas, selon la méthode adoptée, c'est-à-dire si l'on veut faire une recherche exhaustive, à critères ou aléatoire ;
- **Procédure de validation** : Consiste à utiliser un ensemble de test afin de vérifier si l'ensemble d'attributs est valide.

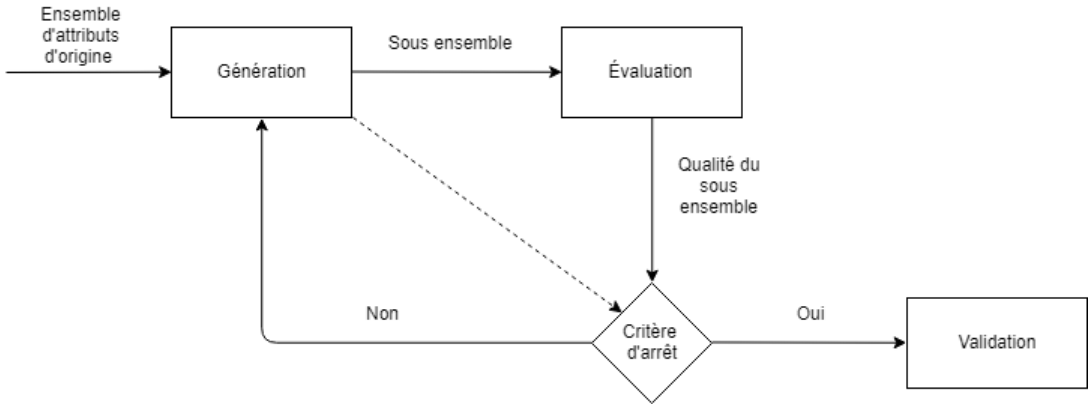


Figure I-1 - Processus de sélection d'attributs (Dash & Liu, 1997)

3.1. La procédure de génération

Un des paramètres principaux à prendre en considération lorsqu'on fait une sélection d'attributs est le nombre d'attributs N, nous avons alors 2^N sous-ensembles d'attributs possibles. Cela rend la tâche de l'exploration de l'ensemble des solutions fastidieuses, même si on prend en compte la puissance des machines actuelles.

La procédure de génération est une procédure de recherche (Siedlecki, Siedlecka, & Sklansky, 1988), (Langley & Sage, 1994). Principalement, elle construit des sous-ensembles d'attributs afin qu'ils soient évalués. La procédure de génération peut se faire avec : (i) un ensemble vide d'attributs, (ii) tout l'ensemble d'attributs, (iii) un sous-ensemble aléatoire d'attributs. Dans les deux premiers cas les attributs sont ajoutés ou retirés répétitivement au fur et à mesure, alors que dans le dernier cas, ils sont ajoutés ou retirés répétitivement ou bien ils sont générés aléatoirement (Langley & Sage, 1994). Il existe différentes approches pour résoudre ce problème, on dénombre trois catégories : méthodes complètes, méthodes heuristiques, méthodes aléatoires.

3.1.1. Méthode complète

Cette procédure de génération effectue une recherche complète dans l'ensemble d'attributs afin de trouver un sous-ensemble d'attributs optimal en utilisant des fonctions d'évaluations. Une recherche exhaustive est une recherche complète. Cela dit, (Schlimmer, 1993) a écrit : « Ce n'est pas parce que la recherche doit être complète qu'elle doit être exhaustive ». En effet, il existe des heuristiques qui permettent de réduire l'espace de recherche sans pour autant diminuer les chances de trouver un ensemble d'attributs optimal, et cela parce que la procédure peut effectuer un retour arrière « Backtracking » (Dash & Liu, 1997). Il existe plusieurs techniques de backtracking, comme : Branch & Bound, best first search et beam search.

3.1.2. Méthode aléatoire

La recherche ne se fait pas sur tout l'ensemble d'attributs, mais uniquement sur un sous-ensemble. Et cela en fixant le nombre maximum d'itérations à effectuer. De ce fait la qualité du sous-ensemble d'attributs sélectionné dépend fortement des ressources disponibles et des contraintes appliquées. La procédure de génération aléatoire a besoin de certains paramètres tel

que le nombre d'itérations et l'exactitude voulue. Ces derniers doivent être bien choisit car c'est de ces paramètres que dépendra la qualité du sous-ensemble obtenu.

3.1.3. Méthode heuristique

La recherche exhaustive dans tout l'ensemble d'attributs peut s'avérer très consommatrice en temps de calcul. De plus, les méthodes aléatoires peuvent facilement rater de bonnes solutions. D'où le fait que certains chercheurs ont développé des algorithmes heuristiques pour la sélection d'attributs. C'est une technique très connue dans les domaines de l'optimisation combinatoire et de l'intelligence artificielle.

Le but de ces méthodes est de diminuer le nombre de sous ensemble à évaluer, on aura N^2 sous ensemble d'attributs à explorer, et de ce fait une complexité quadratique $O(N^2)$. Les algorithmes génétiques (AG) ou la recherche taboue sont très utilisés pour concevoir ce genre de méthodes. Dans le cadre de notre projet de fin d'étude nous aurons à utiliser ce type de méthodes de génération.

3.2. La fonction d'évaluation

Une fonction d'évaluation mesure la qualité (Exactitude) du sous-ensemble d'attributs, produit par la procédure de génération. Les sous-ensembles d'attributs sont comparés entre eux pour faire ressortir le meilleur sous-ensemble en termes de qualité et d'exactitude. Une fonction d'évaluation mesure la capacité d'un attribut ou d'un ensemble d'attributs à se distinguer des autres. En d'autres termes, si l'attribut ou l'ensemble d'attributs est pertinent dans la tâche visée.

La qualité d'un ensemble d'attributs sélectionnés est relative à la fonction d'évaluation utilisée. C'est-à-dire qu'un ensemble choisit en utilisant une certaine fonction d'évaluation peut ne pas l'être en utilisant une autre fonction d'évaluation.

Les fonctions d'évaluations ont été divisées en cinq catégories (Ben-Bassat, 1982) et (Doak, 1992) : mesure de distance, gain d'informations, mesure de dépendance, mesure de consistance, taux d'erreur du classificateur.

3.2.1. Mesure de distance

Soient X et Y deux attributs de classes différentes. Si la distance entre X et la classe à laquelle appartient Y est plus grande que la distance entre Y et la classe à laquelle appartient X, alors X est pris et non Y. Les distances Euclidiennes sont les plus utilisées pour ce type de mesure.

3.2.2. Gain d'information

Le gain d'information d'un attribut X est défini comme étant la différence entre l'incertitude antérieure et l'incertitude postérieure attendue utilisant X. L'attribut X est préférable à l'attribut Y si le gain d'information de l'attribut X est supérieur à celui de Y. L'entropie de Shannon (Shannon, 1948) est une des mesures du gain d'informations les plus utilisées.

3.2.3. Mesure de dépendance

Cette mesure quantifie la corrélation qui existe entre deux attributs. Autrement dit, c'est de savoir à quel degré la valeur d'un attribut peut être prédite par la valeur d'un autre. Plus l'attribut représente au mieux la classe, meilleure est son évaluation. Le coefficient de corrélation est le plus utilisé pour ce type de mesure. Une petite variation permet de mesurer la dépendance entre les attributs, et de ce fait avoir le degré de redondance de chaque attribut.

3.2.4. Mesure de la consistance

C'est une mesure quelque peu nouvelle, elle est différente des autres mesures du fait de son attachement fort et profond avec la sémantique des attributs. Cette mesure trouve le plus petit sous-ensemble qui satisfait le taux d'exactitude fixé par l'utilisateur.

3.2.5. Taux d'erreur du classificateur

Ils constituent la base de l'approche « enveloppante » que nous présenterons plus tard. Sa finalité est de construire un classificateur capable de dénoter correctement chaque instance, tout en minimisant la probabilité d'erreur de celui-ci.
Le tableau suivant dresse une petite comparaison entre ces différents types de fonctions :

Fonctions d'évaluation	Généralisation	Complexité temporelle	Exactitude
Mesure de distance	Oui	Faible	-
Mesure d'information	Oui	Faible	-
Mesure de dépendance	Oui	Faible	-
Mesure de consistance	Oui	Moyen	-
Taux d'erreur du classificateur	Non	Elevé	Très élevée

Tableau I-1 - Comparaison entre différents types de fonctions d'évaluations

Les différents paramètres utilisés pour la comparaison sont :

- Généralité : à quel point le sous-ensemble sélectionné convient-il à différents classifieurs (pas seulement à un seul)
- Complexité temporelle : temps nécessaire à la sélection du sous-ensemble
- Exactitude : quelle est l'exactitude de la prédiction utilisant le sous-ensemble sélectionné

Le « - » dans la dernière colonne signifie que rien ne peut être conclu quant à l'exactitude de la fonction d'évaluation correspondante. Sauf « taux d'erreur du classificateur », l'exactitude de toutes les autres fonctions d'évaluation dépend de l'ensemble de données et du classificateur (pour la classification après la sélection des caractéristiques) utilisés. Les résultats du tableau 1 montrent une tendance non surprenante : plus le temps passé est long, plus l'exactitude est grande. Le tableau nous indique également quelle mesure doit être utilisée dans différentes circonstances, par exemple, avec des contraintes de temps, compte tenu du choix de plusieurs classifieurs, le taux d'erreur du classifieur ne doit pas être sélectionné comme fonction d'évaluation.

3.3. Le critère d'arrêt

Il est très important de fixer des critères d'arrêt. Un critère d'arrêt peut dépendre de la procédure de génération et/ou de la fonction d'évaluation utilisées. Les critères d'arrêt peuvent être :

a. L'atteinte d'un nombre prédéfini d'attributs sélectionnés.
b. L'atteinte d'un nombre prédéfini d'itérations.
c. Si l'ajout ou la suppression d'attributs ne fournit pas un sous-ensemble d'attributs meilleur.
d. Si on atteint un bon sous-ensemble d'attributs, qui répond aux contraintes de temps et de qualité imposées.

3.4. La procédure de validation

Le processus de sélection d'attributs se termine par la construction d'un sous-ensemble d'attributs qui sera soumis à des tests de validation. Cela permettra d'examiner les performances de la classification. La procédure de validation sert à évaluer les règles de classification employées pour sélectionner les attributs.

Il existe deux principales méthodes de validation : la validation croisée et la validation par échantillon de test. La méthode de « validation-croisée » (cross-validation en anglais), est une méthode qui consiste à diviser l'ensemble de données en k blocs. $(k-1)$ blocs sont utilisés pour sélectionner un sous ensemble d'attributs et construire un classificateur. Le sous-ensemble ainsi créé est évalué sur le bloc restant à l'aide du même classificateur (Huerta, 2008).

Cette méthode à un gros désavantage, c'est le fait de réitérer le processus de sélection (k-1) fois, ce qui peut engendrer, surtout pour une approche enveloppante ou intégrée présentées plus tard, des coûts de calculs énormes. La méthode de validation par échantillon mesure la fiabilité du modèle fondé (classificateur) à l'aide d'une technique d'échantillonnage (Refaeilzadeh, Tang, & Liu, 2009).

Cette méthode est très simple, il suffit de diviser l'échantillon de taille n en deux échantillons, l'échantillon d'apprentissage (> 60 % de l'échantillon) et l'échantillon de test. Le classificateur est bâti sur l'échantillon d'apprentissage et validé sur l'échantillon de test. L'erreur est estimée en calculant l'erreur quadratique moyenne.

4. Les principales approches de sélection d'attributs

En se basant sur les techniques utilisées pour générer des sous-ensembles d'attributs, ainsi que sur les fonctions d'évaluations servant à estimer la qualité de ces sous-ensembles, on a pu regrouper les différentes méthodes de sélection d'attributs en trois principales approches : l'approche enveloppante (wrapper), l'approche filtre (filter) et l'approche intégrée (embedded) (Langley & Sage, 1994), (Kohavi & John, 1997), (Guyon & Elisseeff, 2003).

4.1. Les approches enveloppantes

L'approche enveloppante se compose de deux phases (Liu & Motoda, 2012) :

- **Phase 1** : La procédure de génération construit le meilleur sous-ensemble en utilisant comme critère d'arrêt, pour les fonctions d'évaluation, l'exactitude de la classification.
- **Phase 2** : apprentissage et test. On fait une classification à partir du sous ensemble obtenu et on applique des tests de validation.

L'approche enveloppante voit le classificateur, effectuant la sélection d'attributs, comme une boite noire (Yusta, 2009) , en entrée nous avons l'ensemble entier d'attributs et en sortie le sous ensemble d'attributs optimal (relativement à la méthode de génération utilisée).



Figure I-2 - L'approche enveloppante

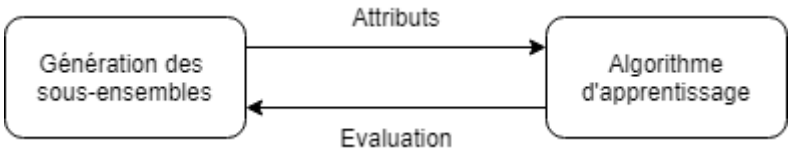


Figure I-3 - Contenu de la boite noire

L'approche enveloppante utilise l'algorithme de prédiction comme fonction d'évaluation, et classe donc les sous-ensembles trouvés selon leur force prédictive (Yusta, 2009). La mesure de performance des techniques se basant sur cette approche est le taux d'erreur du classificateur.

4.2. Les approches par filtre

L'approche par filtre dissocie l'algorithme de prédiction de sa démarche. Elle est concentrée sur les propriétés intrinsèques des données et non pas sur le classificateur.
« Dans les méthodes traditionnelles appelées méthodes par filtres ou de filtrage, le principe consiste à évaluer chaque attribut pour lui assigner un score de pertinence. Ce score permet un classement des attributs et la sélection des attributs les mieux classés c'est-à-dire les plus pertinents. » (Huerta, 2008). Cette approche est basée sur 2 phases (Yusta, 2009) :

- **Phase 1** : sélection des attributs en utilisant des mesures comme : la distance, gain d'informations, la dépendance, la cohérence. Aucune classification n'est utilisée dans cette phase.
- **Phase 2** : apprentissage et test. On fait une classification à partir du sous ensemble obtenu et on applique des tests de validation.

L'approche par filtre est beaucoup utilisée dans la résolution des problèmes de statistiques. Elle s'intéresse à extraire et à quantifier l'information contenu dans chaque attribut. Les différentes fonctions d'évaluations relatives à cette approche sont : mesure de distance, gain d'information, mesure de dépendance, mesure de consistance (Yusta, 2009). Ces mesures sont simples à implémenter et donnent des résultats très rapidement ce qui favorise l'emprunt de cette approche pour les problèmes où l'ensemble de donnée est très grand. Elle est utilisée pour diminuer l'ampleur des données et facilite la tâche pour la classification.

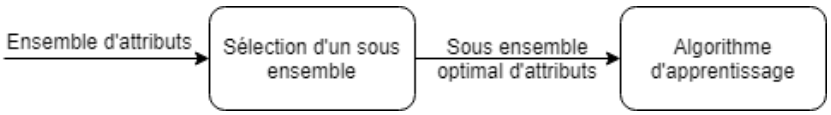


Figure I-4 - L'approche par filtre

4.3. Les approches intégrées

Elle est proche de l'approche enveloppante. Elle combine le processus d'exploration avec un algorithme d'apprentissage (Lal, Chapelle, Weston, & Elisseeff, 2006). Elle utilise un classificateur pour évaluer un sous ensemble de facteurs et gérer le processus de sélection en lui donnant des informations intéressantes (Huerta, 2008).

4.4. Taxonomie des algorithmes de sélection d'attributs

Approches	Avantages	Inconvénients	Exemples
Enveloppante	Simple, interaction avec le classificateur, dépend du type des attributs, prend en considération la dépendance entre les attributs	Recherche gloutonne, classificateur dépend de la sélection d'attribut, calcul intensif, risque de dépassement dans la phase d'apprentissage	SFS, SBS, plus-l-take-away-r, floating search (SFFS et SBFS), algorithmes génétique
Intégrée	Dépend de type des attributs, moins temps de calcul, interaction avec le classificateur, prend en considération la dépendance entre les attributs.	Le classificateur dépend de la sélection d'attribut.	Weighted naive Bayes, ESFS, les arbres de décision (ID3, C 4.5, CART).
Filtre	Rapide, évolutive, indépendante de classificateur.	Ignore la dépendance entre les attributs, pas d'interaction avec le classificateur.	Relief, Relief-F, Focus

Tableau I-2 - Taxonomie des algorithmes de sélection d'attributs

5. Les métaheuristiques pour la sélection d'attributs

Avant de parler des métaheuristiques, on voudrait d'abord faire une abstraction de ce mot qui fait partie d'un domaine plus général : l'optimisation combinatoire (OC). Et donc on essayera de donner des définitions et une taxonomie avant d'entamer l'aspect « métaheuristiques ».

La recherche opérationnelle (RO) est un domaine de recherche très actif qui a pour but principale de trouver la solution optimale (raisonnable) pour un problème d'OC. Il existe deux grandes catégories de méthodes de résolution en RO : les méthodes exactes et les méthodes approchées, les méthodes exactes sont souvent utilisées pour la résolution de petites et moyennes instances pour des problèmes d'OC, tandis que les méthodes approchées sont implémentées pour trouver des solutions raisonnables pour des problèmes de grandes tailles dans un temps réduit.

5.1. Classification des méthodes d'optimisation combinatoire

5.1.1. Problème d'optimisation combinatoire

Un problème d'optimisation combinatoire (POC) est un problème dont le domaine des solutions est fini ou dénombrable mais beaucoup trop grand pour rendre possible une recherche exhaustive. Avant d'aller plus loin, nous donnons une définition plus formelle du principe :

- La fonction objectif, appelée aussi fonction d'évaluation, notée généralement f, sert à évaluer le coût d'une solution d'un problème donné.
- Un problème d'optimisation combinatoire consiste à chercher le minimum (respectivement le maximum) s* d'une fonction f (fonction objectif) à valeur entière ou réelle, sur un ensemble fini S. S étant l'ensemble des solutions réalisables du problème d'optimisation.

$$f(s^*) = Min\ (ou\ Max)\{ f(s)\}\ \ s \in S$$

Une donnée (S, f) est alors appelée un cas ou une instance du POC.

La plupart des problèmes d'OC sont simples et peuvent être résolus, du moins en théorie, de manière triviale. Il suffit pour cela d'énumérer toutes les solutions possibles et en choisir la meilleure. Ceci est le principe même de la plupart des méthodes exactes. Cependant, cette approche n'est applicable, en pratique, qu'à des instances de petite taille.

Les domaines d'application de la RO sont nombreux, on peut prendre comme exemples : la finance, les télécommunications, la planification …etc. Il existe deux grandes catégories de méthodes de résolution en RO **:** les méthodes exactes et les méthodes approchées. Les méthodes exactes sont souvent utilisées pour la résolution de petites et moyennes instances de problèmes d'OC, tandis que les méthodes approchées sont implémentées pour trouver des solutions raisonnables à des instances de grandes tailles en un temps réduit. Les méthodes approchées peuvent être décomposées en trois types : les heuristiques, les métaheuristiques et les hyperheuristiques. Les heuristiques sont spécifiques à un problème particulier alors que les métaheuristiques sont plus générales et peuvent être appliquées pour la résolution de plusieurs problèmes. Quant aux méthodes hybrides, elles sont conçues en combinant deux ou plusieurs méthodes. En ce qui concerne les hyperheuristiques, ce sont des méthodes de plus haut niveau, qui consistent souvent à choisir ou générer des heuristiques ou métaheuristiques parmi un ensemble de méthodes de résolution.

A. Méthodes exactes

Le principe d'une méthode exacte consiste généralement à énumérer, l'ensemble des solutions de l'espace de recherche. Pour éviter d'énumérer toutes les solutions, ces méthodes disposent de techniques pour détecter le plus tôt possible les échecs (calculs de bornes), et d'heuristiques spécifiques pour orienter les différents choix. Ces algorithmes, dits complets, parcourent l'espace de recherche d'une manière exhaustive. Par conséquent, la solution trouvée est optimale. En général, le temps de calcul d'un algorithme complet augmente exponentiellement avec la taille du problème. Ceci rend souvent impossible l'application d'une méthode exacte à de nombreux problèmes pratiques. Parmi les méthodes exactes , on trouve (E. K. Burke & Kendall, 2005) :

- **Programmation linéaire :** Cette approche décrit le problème au moyen d'une équation linéaire appelée Programme Linéaire en Nombre Entier "PLNE". Un problème de programmation linéaire est un problème d'optimisation dans lequel l'objectif (c'est-à-dire l'expression à optimiser) et les contraintes sur la solution peuvent être exprimés sous la forme d'une série d'expressions linéaires sur les variables de décision. Si le problème a n variables alors les contraintes définissent un ensemble d'hyper-plans dans un espace de n dimensions. Ce sont les frontières d'une région n-dimensionnelle qui définissent l'ensemble des solutions réalisables au problème et est connue comme la région possible. (E. K. Burke & Kendall, 2005)

- **Programmation dynamique :** La programmation dynamique "DP" est une méthode ascendante qui suit la logique *diviser pour résoudre*. Elle résout les problèmes d'optimisation en les décomposant en problèmes plus simples. Les étapes de DP sont liées par une relation récursive. Le nom de programmation dynamique découle de sa popularité dans la résolution de problèmes qui nécessitent des décisions à prendre sur une séquence de périodes de temps. Même lorsque ce n'est pas le cas, le nom de programmation dynamique est encore largement utilisé, mais le terme de programmation en plusieurs étapes est parfois utilisé comme alternative. La base de DP est le principe d'optimalité de Bellman 1957 qui déclare que « la sous-politique d'une politique optimale est elle-même optimale en ce qui concerne les états de début et de fin », i.e. dans une séquence optimale, chaque sous-séquence doit aussi être optimale. (E. K. Burke & Kendall, 2005)

- **Recherche arborescente :** la recherche arborescente est l'exploration implicite intelligente de l'espace de solution, en utilisant certaines priorités du problème concerné et une fonction d'évaluation, cette méthode élimine plusieurs solutions, avant l'exploration, qui ne mènent pas à la solution recherchée. Cette méthode est connue sous le nom Branch and Bound. La logique derrière l'algorithme de B&B est de réduire le nombre d'alternatives qui doivent être considérées en partitionnant le problème en plusieurs sous-problèmes plus petits et en utilisant des informations locales sous forme de bornes pour éliminer celles qui ne peuvent pas être quasi-optimales. Les implémentations les plus simples de B&B sont celles basées sur une approche constructive dans laquelle des solutions partielles sont construites un élément à la fois. Le succès d'une implémentation de B&B pour un gros problème dépend du nombre de branches pouvant être élaguées avec succès. Cela dépend en grande partie de la qualité des bounds, mais la stratégie de branchement peut également avoir un effet significatif sur le nombre de nœuds à explorer. (E. K. Burke & Kendall, 2005)

B. Heuristiques

Le recours aux méthodes approchées telles que les heuristiques s'avère de plus en plus intéressant. En effet, dans beaucoup de cas pratiques, disposer d'une solution optimale n'est pas primordial et une solution d'assez bonne qualité trouvée en un temps réduit est jugée plus intéressante qu'une solution optimale trouvée en un temps beaucoup plus long.
Les deux premières définitions des heuristiques dans la littérature sont :

1. Une heuristique est une méthode qui recherche de bonnes solutions à un coût de calcul raisonnable sans pouvoir garantir l'optimalité, et éventuellement pas la faisabilité (Rayward-Smith, Osman, Reeves, & Smith, 1996).

2. Une heuristique est une « règle générale » basée sur la connaissance du domaine d'une application particulière, qui guide la résolution d'un problème. L'heuristique peut donc être très utile la plupart du temps mais ses résultats ou performances ne peuvent être garantis (Illingworth, 1997).

Elles peuvent être classées en deux catégories :

- **Constructives :** Les méthodes constructives partent d'une solution initiale vide S_0, et insèrent à chaque étape k une composante x_k dans la solution partielle courante $S_{k-1} = (x_0, \ldots, x_{k-1})$ tout en respectant les contraintes du problème, pour aboutir enfin à une solution admissible de la forme $S = (x_0, x_1, \ldots, x_n)$

- **De voisinage :** Une méthode typique de voisinage est un processus itératif fondé sur deux éléments essentiels : un voisinage et une procédure exploitant le voisinage. Une méthode de voisinage débute avec une solution initiale (générée aléatoirement ou par l'application d'une heuristique par construction), et réalise ensuite un processus itératif qui consiste à remplacer la configuration courante (solution courante) par l'un de ses voisins en tenant compte de la fonction coût. Ce processus s'arrête et retourne la meilleure configuration trouvée quand la condition d'arrêt est satisfaite. Cette dernière concerne généralement un nombre d'itérations ou un objectif à réaliser.

C. Métaheuristiques

Les métaheuristiques forment une famille de méthodes d'optimisation visant à résoudre des problèmes d'optimisation difficile souvent issus des domaines de la recherche opérationnelle ou de l'intelligence artificielle pour lesquels on ne connaît pas de méthode exacte efficace. Grâce à ces méthodes, on peut proposer aujourd'hui des solutions approchées pour des problèmes d'optimisation classiques de plus grande taille et pour de très nombreuses applications qui étaient impossibles à traiter auparavant. Les métaheuristiques regroupent deux classes de méthodes :

- **Méthodes basées sur une solution unique** manipulant une seule solution telles que le recuit simulé et la recherche tabou.

- **Méthodes basées sur une population** qui manipulent une population de solutions et se basent en général sur l'évolution naturelle ou sur l'observation des comportements sociaux des individus ou des communautés. On retrouve dans cette classe les algorithmes génétiques, les méthodes basées sur les colonies de fourmis, les essaims d'abeilles et les systèmes immunitaires.

D. Métaheuristiques hybrides

On parle d'hybridation quand la métaheuristique considérée est composée de plusieurs méthodes se répartissant les tâches de recherche. Ces méthodes peuvent être des heuristiques, des métaheuristiques, des techniques de programmation mathématique ou des techniques d'apprentissage automatique (Machine learning).

On parlera de la classification des métaheuristiques hybrides selon (Talbi, 2016) en détail dans le chapitre 3 : « L'intégration de l'A/R aux métaheuristiques » .

E. Hyperheuristiques

Les méta-heuristiques et leurs hybridations avec les heuristiques ont été utilisées avec succès pour résoudre beaucoup de problèmes réels, mais la réutilisabilité de ces dernières n'est pas souvent possible, et les performances de ces méthodes dépendent du paramétrage. L'idée est de développer des méthodes applicables sur une plus large classe d'instances de problèmes, tout en gardant une bonne performance en sortie et une facilité d'implémentation. Une des définitions les plus citées présente les hyperheuristiques comme étant « des méthodes qui, en s'aidant d'extraction de connaissance et d'apprentissage, choisit, combine ou génère des heuristiques faciles à implémenter dans le but de résoudre efficacement une instance ou une classe d'instances d'un problème » (E. K. Burke & Kendall, 2005). Les heuristiques manipulées sont appelés "heuristiques de bas niveau". Le processus qui manipule les heuristiques de bas niveau est appelé "Stratégie de haut niveau".

Les hyperheuristiques sont classées en :

1. **Hyperheuristiques de sélection** : Elles sont des techniques qui permettent de choisir une heuristique parmi un ensemble pour produire des combinaisons d'heuristiques.

2. **Hyperheuristiques de génération** : méthodologies pour générer de nouvelles heuristiques depuis les composants d'heuristiques existantes.

Ci-dessous, un schéma présentant une taxonomie des méthodes de résolution de problèmes d'OC :

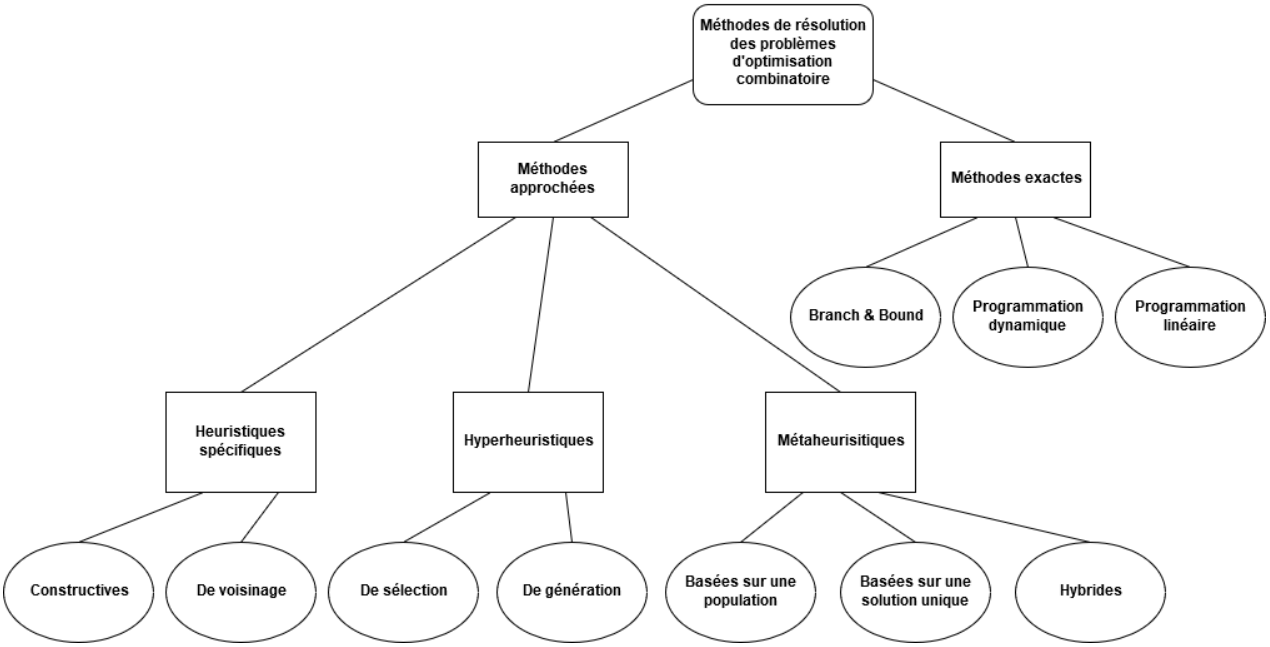


Figure I-5 - Classification des méthodes de résolution

6. BSO pour la sélection d'attributs

6.1. Principe de l'algorithme BSO

Les essaims d'abeilles ont par nature un comportement très intéressant caractérisé par une intelligence collective qui permet une autoadaptation à l'environnement et une affectation dynamique des tâches. Au cours de la dernière décennie, de nombreux nouveaux algorithmes inspirés du comportement naturel des abeilles ont été proposés et appliqués pour différents problèmes. (Sadeg, Hamdad, Benatchba, & Habbas, 2015)

L'optimisation des essaims d'abeilles ou **Bee Swarm Optimization (BSO)** proposée dans (Drias, Sadeg, & Yahi, 2005) est inspirée du comportement de recherche de vraies abeilles. De nombreux chercheurs ont étudié ce comportement, considéré comme la tâche la plus importante de la ruche. Dans son processus de recherche de nourriture, une abeille commence par quitter la ruche afin de trouver une fleur et de récolter le nectar. Ensuite, elle retourne à la ruche et décharge le nectar. Si la source de nourriture est riche, l'abeille communique sa direction et sa distance à ses camarades via une danse. Les autres abeilles suivent naturellement celle qui indique la meilleure source de nourriture.

BSO est un processus de recherche itérative basé sur une population d'abeilles artificielles coopérant pour résoudre un problème d'optimisation en imitant le comportement de recherche de nourriture décrit ci-dessus. Tout d'abord, une solution initiale est générée de manière aléatoire ou via une heuristique. Cette solution sera la solution de référence, $refSol$ à partir de laquelle une région de recherche, un ensemble de solutions candidates, est déterminée. Après cela, chaque solution est affectée à une abeille comme point de départ d'une recherche locale. À la fin de la recherche, chaque abeille communique sa meilleure solution aux autres par le biais d'une table, $Danse$. L'une des solutions de ce tableau sera choisie comme solution de référence lors de la prochaine itération. Afin d'éviter les cycles, les solutions de référence sont stockées dans une $liste\ Tabou$.

6.2. Adaptation de BSO à la sélection d'attributs

Dans cette section, nous présenterons la méthode de sélection d'attributs basée sur l'approche enveloppante qui utilise la métaheuristique BSO pour effectuer le processus de recherche de sous-ensembles. L'application de BSO à la sélection d'attributs nécessite l'adaptation de l'algorithme général aux spécificités du problème. Dans cette section, nous définirons comment une solution est représentée, comment sa qualité est évaluée et comment la région de recherche est déterminée, ainsi que la sélection de la solution de référence et la recherche locale effectuée par les abeilles.

6.2.1. Représentation d'une solution

Une solution est représentée par un vecteur binaire de longueur $\boldsymbol{n}$, où $\boldsymbol{n}$ est le nombre initial d'attributs pour un ensemble donnée. Une position du vecteur est mise à 1 si l'attribut correspondant est sélectionnée, 0 sinon.

6.2.2. Fonction d'évaluation

Représente la qualité de la solution et est noté $f(s)$. Dans nos travaux, elle équivaut à l'exactitude de la classification renvoyée par le classificateur utilisé et est calculée à l'aide de la formule suivante :

$$Exactitude = \frac{nombre\ de\ vrais\ positifs + nombre\ de\ vrais\ négatifs}{taille\ de\ la\ population}$$

Notez que si deux solutions ont la même exactitude, celle qui utilise le moins d'attributs est considérée comme la meilleure.

6.2.3. L'espace de recherche : searchRegion

C'est un ensemble de solutions générées à partir de la solution de référence en retournant dans $refSol$ un nombre de bits égal à (n / $flip$), $flip$ étant un paramètre empirique. La taille de cet ensemble est égale au nombre d'abeilles car chaque solution est affectée à une abeille comme point de départ de sa recherche locale. La valeur de $flip$ a un impact sur les performances du

processus de recherche car elle détermine la distance entre $refSol$ et les solutions qui définissent la région de recherche. En effet, une valeur trop petite favorisera l'exploration plutôt que l'exploitation de l'espace de recherche, tandis qu'une valeur trop élevée pourrait amener l'algorithme à converger vers un optimum local.

6.2.4. Le choix de refSol

La solution initiale est générée d'une façon purement aléatoire (et donc la notion d'aléatoire dépendra lors de l'implémentation, de l'environnement d'exécution), qui est équivalent à dire qu'on aura un vecteur de taille n, qui possède k positions à 1 et $(n - k)$ positions à 0.
Pour le reste des itérations, choisir $refSol$ est basé sur le principe d'intensification/diversification : alors que la meilleure solution globale $bestGlobalSol$ est améliorée après chaque itération, une intensification est réalisée en sélectionnant une solution de référence parmi celles de la table de $Danse$. Sinon, une diversification est effectuée si la région de recherche a atteint le nombre maximal de chances qui lui est accordé ou une intensification est effectuée et le nombre de chances est réduit. Une diversification consiste à choisir une solution de danse qui soit la plus éloignée de toutes les solutions stockées dans la $liste\ Taboue$. Si toutes les solutions de $Dance$ sont dans la $liste\ Taboue$, une solution aléatoire est générée pour être la solution de référence.

6.2.5. La recherche locale

Le processus de recherche locale au niveau de l'abeille est un processus itératif. Le nombre d'itérations s'appelle $locIterations$ et constitue un paramètre empirique. Le processus comprend deux phases :

- Une simple recherche locale, autrement dit, on évalue les solutions du voisinage de la solution de référence qu'on a reçu, et on cherche la meilleure solution en termes d'exactitude.
- Une deuxième phase qui consiste à comparer la meilleure solution avec le voisinage et de retrouver la meilleure solution, et cette fois-ci, en termes de nombre d'attributs choisit.

6.3. L'algorithme général

Entrée : Une instance d'un problème d'optimisation combinatoire
Sortie : La meilleure solution trouvée
$refSol \leftarrow$ une solution initiale trouvée aléatoirement ou par une heuristique
Tant que critère d'arrêt **faire**
    Insérer $refSol$ dans la $liste\ Taboue$
    Déterminer $searchRegion$ à partir de $refSol$
    $nbChances \leftarrow maxChances$
    Attribuer une solution de $searchRegion$ à chaque abeille
    Pour *chaque abeille k* **faire**
        Effectuer une recherche locale
        Stocker le résultat dans la table $Dance$
    Si $f(bestSol) > f(bestGlobalSol)$ **alors**
        $bestGlobalSol \leftarrow bestSol$
        $nbChances \leftarrow maxChances$
        Intensification
    Si non
        Si $nbChances > 0$ **alors**
            $nbChances \leftarrow nbChances - 1$
            Intensification
        Si non
            Diversification
retourner $bestGlobalSol$

7. Conclusion

Dans ce chapitre nous avons abordé le problème de sélection d'attributs qui consiste à essayer de trouver des sous-ensembles d'attributs pertinents qui permettraient de classifier nos données en un minimum de temps. Nous avons étudié en détail le processus suivi pour résoudre ce problème. Nous nous sommes intéressés aux principales approches utilisées. Nous avons remarqué que chacune d'entre elles avait des avantages et bien sûr des inconvénients. Ensuite, nous avons remarqué que l'utilisation de métaheuristiques pourrait permettre d'avoir des résultats relativement bons sans avoir à explorer tout l'espace de solutions possibles. Enfin, nous avons présenté la métaheuristique BSO, qui est le cœur de notre sujet, et comment on l'a utilisé pour le problème de sélection d'attributs.

II. Chapitre 2 : L’apprentissage par renforcement

1. Introduction

L’intelligence artificielle (**IA**) et l’apprentissage automatique (**ML**) ne sont pas la même chose, mais penser qu’ils le sont, mène souvent à des confusions. L'intelligence artificielle existe depuis longtemps, avec l’apparition des premiers ordinateurs qui ont été conçus comme des « machines logiques » et grâce à des capacités de reproduction telles que l'arithmétique de base et la mémoire, les ingénieurs ont compris que leur travail consistait essentiellement à créer des cerveaux mécaniques.

À mesure que la technologie, et surtout notre compréhension du fonctionnement de l’esprit humain progressaient, la conception de ce qui constitue l'IA a changé. Plutôt que des calculs de plus en plus complexes, les travaux dans ce domaine se sont concentrés sur l'imitation des processus de décision humains et la réalisation de tâches de manière automatique.

Le ML dans ce cas, peut être considéré comme le véhicule qui fait avancer le développement de l'IA, et avec l'émergence d'Internet et l'énorme augmentation de la quantité d'informations numériques générées, les ingénieurs ont vite compris qu'il serait plus efficace d’enseigner aux machines comment faire et penser comme des êtres humains à travers ces données.

Le paradigme de l'apprentissage par essai-erreur est l’idée fondamentale derrière l’apprentissage par renforcement (A/R), qui se diffère des autres méthodes d’apprentissage automatique par le fait d’être un apprentissage par interaction dirigé par les objectifs.

L’idée d’apprendre à partir de l’environnement autour n’est pas nouvelle, et vient intuitivement lorsqu’on parle de la nature de l’apprentissage, et l’exemple d’un bébé est le plus expressif, puisqu’au début, il n’interagît qu’avec son environnement par les sens qu’il possède, et personne ne lui dit explicitement ce qu’il doit faire, puisque, déjà il apprend la langue en faisant correspondre les sons avec les faits. Ceci produit une certaine base d’informations sur les causes et les conséquences de ses actions, et sur ce qu’il doit faire pour atteindre un objectif, qu’il soit aussi simple qu’essayer de marcher pour aller d’un point A à un point B, et le fait de tomber lui permet d’apprendre à trouver le bon équilibre pour rester debout.

Dans la suite, nous allons définir l’A/R ainsi que ses notions de bases et essayer de l’expliquer avec des exemples plus concerts.

2. Définitions

2.1. Apprentissage automatique

2.1.1. Définition

« *A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T, as measured by P, improves with experience E* » (Mitchell, Michalski, & Carbonell, 2013). L'apprentissage automatique concerne la conception, l'analyse, le développement et l'implémentation de méthodes permettant à une machine d'apprendre par un processus systématique (apprentissage supervisé, apprentissage non supervisé, apprentissage semi supervisé, apprentissage par renforcement).

2.1.2. Apprentissage supervisé

Il s'agit d'apprendre à partir d'exemples pour pouvoir affecter de nouveaux exemples. Soit un ensemble d'apprentissage constitué de $\boldsymbol{n}$ données d'observations de type entrée-sortie $D = \{(X1, Y1), \ldots, (Xn, Yn)\}$ où les $Xi, i = 1, \ldots, n$ représentent des ayant souvent ($\boldsymbol{p}$ attributs ou caractéristiques) et $Yi\ , i = 1, \ldots, n$, les valeurs d'affectations. Le but est donc de construire à partir de l'ensemble d'apprentissage, un modèle qui permettra de prévoir la sortie d'une nouvelle entrée.

Deux types d'apprentissage supervisé existent :

- **Classification supervisée** : les $Yi\ , i = 1, \ldots, n$ prennent comme valeur $-1, 1$ ou plusieurs valeurs catégoriques.
- **Régression** : les $Yi\ , i = 1, \ldots, n$ sont numériques.

Parmi les méthodes d'apprentissage supervisée, nous retrouvons :

- Les K-plus proches voisins : KNN.
- Les machines à vecteur de support : SVM.
- Les arbres de décisions et forêts aléatoires.
- Les réseaux de neurones
- La régression simple et multiple (linéaire et non linéaire).

2.1.3. Apprentissage non supervisé

Les entrées sont connues et non pas les sorties, ce sont des méthodes dont le rôle est plus descriptif que prédictif. Il s'agit alors de construire un modèle permettant de représenter au mieux les observations $X1, \ldots, Xn$, de manière précise. Deux sous catégories existent :

1. **Le clustering** : Il s'agit de regrouper les données hétérogènes en classes de données homogènes et cela en se basant sur des indices de similarités. Parmi les algorithmes les plus connus :
   - K-means
   - L'algorithme EM (Expectation-Maximization).
   - DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*)
   - SOM (Self Organizing Map)

2. **Les associations** : Trouver des associations significatives, qui décrivent l'ensemble des individus, entres les items d'une base de données.
   Formellement, soient :
   - $I = \{ I1, I2, \ldots , In\}$ l'ensemble de n items dans la base de données
   - $T = \{T1, T2, \ldots , Tm\}$ l'ensemble des transactions de la base de données, m étant leur nombre, une transaction étant définie comme un sous ensemble de I.
   - $X \subseteq I$, $Y \subseteq I$ deux sous-ensembles d'items

   Une règle d'association est définie comme suit :

   $$X \rightarrow Y \text{ ou } X \subseteq T , Y \subseteq Tet \quad X \cap Y = \emptyset$$

   Les algorithmes les plus connus et utilisés dans le les approches à filtrage collaboratif sont (Zheng, Kohavi, & Mason, 2001) : L'algorithme *Apriori* et *FP-Growth*

2.1.4. Apprentissage semi-supervisé

C'est une classe de techniques d'apprentissage automatique qui utilise un ensemble de données avec labels(peu) et sans labels (beaucoup). Il se situe ainsi entre l'apprentissage supervisé qui n'utilise que des données étiquetées et l'apprentissage non-supervisé qui n'utilise que des données non-étiquetées. Il a été démontré que l'utilisation de données non-étiquetées, en

combinaison avec des données étiquetées, permet d'améliorer significativement la qualité de l'apprentissage.

Un autre intérêt, L'étiquetage de données nécessite l'intervention d'un utilisateur humain, et lorsque les jeux de données deviennent très grands, cette opération peut être fastidieuse. Dans ce cas, l'apprentissage semi-supervisé, qui ne nécessite que quelques étiquettes, devient d'un intérêt pratique évident.

Deux types d'apprentissage semi-supervisé :

- Classification semi-supervisée : entraîner sur des données avec labels et exploiter les données (beaucoup) sans labels.
- Clustering semi-supervisé : clustering des données sans labels en s'aidant des données avec labels.

2.1.5. Apprentissage par renforcement

« *L'apprentissage par renforcement consiste à apprendre ce qu'il faut faire (comment mapper des situations sur des actions) afin de maximiser un signal de récompense numérique. On ne dit pas à l'apprenant quelles actions entreprendre, mais il doit plutôt découvrir quelles actions donnent le plus de récompense en les essayant. Dans les cas les plus intéressants et les plus difficiles, les actions peuvent affecter non seulement la récompense immédiate, mais également la situation suivante et, par là même, toutes les récompenses ultérieures.* » - (Sutton & Barto, 1998)

Pour formaliser l'apprentissage par renforcement, nous avons besoin d'un certain nombre de concepts et de notions :

Nous considérons un agent, qui se trouve dans l'un des états $s \in S$, dans un environnement. L'agent peut effectuer certaines actions qui entraîneront généralement des transitions dans les états voisins s'. Les transitions peuvent être soit déterministes, soit plus généralement, probabilistes avec certaines probabilités de transition inconnues.

3. Notions de base de l'A/R

Avant d'aborder les notions de bases de l'A/R, et comme nous avons présenté les notions de base l'apprentissage supervisé et l'A/R, le schéma suivant, tiré de (Reinforcement Learning Summer School) 2017, Montréal, présente une comparaison entre les deux modèles :

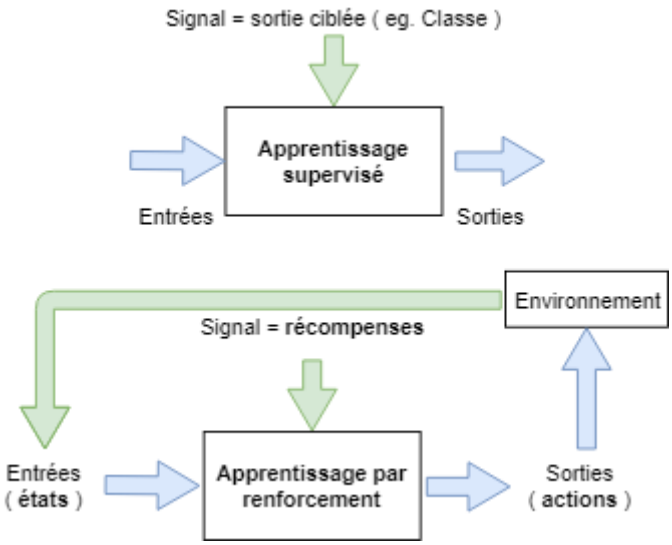


Figure II-1 - Comparaison entre l'A/R et l'apprentissage supervisé

Nous pouvons voir donc, que l'A/R n'est pas différent de ce que nous avons l'habitude de faire, puisque dans le cas de l'A/R, les données sont générées empiriquement, et l'agent apprend des données qu'il génère. Voir donc le modèle de l'apprentissage supervisé sous un autre angle, peut aider à comprendre l'A/R.

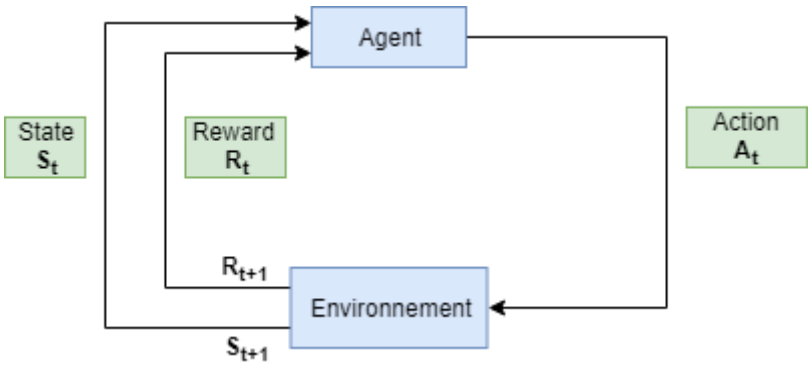


Figure II-2 - L'interaction entre l'agent et l'environnement

A. L'agent

L'apprenant et le décideur s'appelle l'agent, son rôle est d'explorer son environnement.

B. L'environnement

Tout ce qui est externe par rapport à l'agent, est considéré comme environnement, et c'est souvent le contexte ou l'espace des états où l'agent est mis.

C. L'état

Lorsque nous parlons d'état, c'est en premier lieu relatif à l'environnement, et en supposant que l'environnement est totalement observable par l'agent, nous aurons un même état pour l'agent, contrairement dans le cas où l'environnement est partiellement observable, qui demande d'autres hypothèses.

D. L'action

C'est la décision que prend l'agent après avoir évaluer les états qui sont susceptible d'être visité dans le prochain pas, l'ensemble d'actions peut être fini, par exemple aller en haut ou en bas, comme il se pourrait qu'il soit infini.

E. La politique

Définie le comportement de l'agent d'apprentissage en un moment donné, une politique est une correspondance entre les états perçus de l'environnement et les actions à entreprendre dans ces états. Dans certains cas, la stratégie peut être une simple fonction ou une table de consultation, alors que dans d'autres, elle peut impliquer des calculs poussés tels qu'un processus de recherche. La politique est la base d'un agent d'apprentissage par renforcement dans la mesure où elle suffit pour déterminer le comportement, et en général la politique est stochastique.

F. La fonction de récompense

Elle définit le but d'un apprentissage par renforcement elle indique la désirabilité d'un état, Le seul objectif d'un agent d'apprentissage du renforcement est de maximiser la récompense totale qu'il reçoit à long terme. La fonction de récompense définies les bons et les mauvais événements pour l'agent. Les fonctions de récompense sont eux aussi stochastiques.

G. Fonction valeur

Alors qu'une fonction de récompense indique ce qui est bon dans l'immédiat, la fonction de valeur indique ce qui est bon à long terme, la valeur d'un état est le montant total de récompense qu'un agent peut s'attendre à accumuler dans le futur à partir de l'état courant.

H. Modèle

Un modèle n'est pas présent dans tous les cas, et imite le comportement de l'environnement, par exemple étant donné un état et une action, le modèle pourrait prédire le prochain état résultant et la prochaine récompense. En effet tout moyen de décider d'un plan d'action en considérant les situations futures possibles avant qu'elles ne soient réellement vécues. On peut comparer les deux types d'apprentissage dans le tableau suivant.

Sans modèle (direct)	Basé modèle (indirect)
• On a besoin seulement de savoir quelle action prendre et non pas pourquoi	• A besoin de garder toutes les données
• En changeant d'environnement, l'agent doit apprendre de 0.	• Le modèle peut être utilisé dans un environnement similaire
• Moins d'espace : o(Etats x Actions)	• Plus d'espace : o(Etats x Actions x Etats)

Tableau II-1 - Comparaison apprentissage basé/sans modèle

I. Exploration vs Exploitation

L'exploration consiste à trouver plus d'informations sur l'environnement, alors que l'exploitation consiste à exploiter des informations connues pour maximiser la récompense. L'objectif d'un agent est de maximiser la récompense cumulée. Cependant, nous pouvons tomber dans un piège : dans le cas d'un environnement sous forme de grille, où nous avons un nombre infini de récompenses positives (+1), au voisinage de notre agent, il pourrait être coincé dans ce voisinage, alors qu'une meilleure solution pourrait exister ailleurs (+1000), et donc, si nous nous concentrons uniquement sur la récompense, notre agent n'atteindra jamais cette somme gigantesque. C'est ce que nous appelons le compromis exploration/exploitation. Nous devons définir une règle qui aide à gérer ce compromis. Dans le tableau suivant, nous pourrons résumer cela :

Exploration	Exploitation
• Accroitre les connaissances pour un gain à long terme, au détriment d'un gain à court terme	• Tirer parti des connaissances actuelles pour maximiser les gains à court terme
• Peut ralentir l'apprentissage	• Peut mener à des solutions non optimales

Tableau II-2 - Comparaison exploration et exploitation

J. Apprentissage passif vs actif

L'apprentissage passif est lorsque l'environnement est déterministe, autrement dit, c'est lorsque si on se trouve à un état $\boldsymbol{S}$, on sait qu'on va choisir l'action $\boldsymbol{A}$ (action= $\pi_t(s)$). Contrairement à l'apprentissage actif, où l'environnement est souvent stochastique et la politique de choix des actions change au fur et à mesure que la trajectoire (l'ensemble de tous les états menant vers la solution : $s_0, s_1, \dots, s_k$) est formée (action= $\pi_t(a|s)$).

4. Les approches

Pour la classification qu'on va présenter, nous nous sommes basés sur les travaux de (Sutton & Barto, 1998), qui représente selon la littérature, *LA* référence de l'apprentissage par renforcement.

4.1. Tabulaires

Les méthodes tabulaires font face à des problèmes dans lesquels les espaces d'état et d'actions sont suffisamment petits pour que les fonctions de valeur approximative soient représentées sous forme de tableaux.

4.1.1. Les équations de Bellman

L'apprentissage par renforcement est basé sur deux formes de l'équation de Bellman : l'équation d'optimalité de Bellman (EOB) et l'équation de politique de Bellman (EPB), qui sont associés à deux algorithmes : Itération de valeur (IV) et itération de politique (IP).
EOB :

$$J^*(i) = \max_{a \in A(i)} [\, \bar{r}(i,a) + \gamma \sum_{j=1}^{|S|} p(i,a,j) J^*(j) \,] \text{ , pour tout } i \in S$$

EPB :

$$J_{\hat{d}}(i) = \left[\bar{r}\big(i, d(i)\big) + \gamma \sum_{j=1}^{|S|} p(i, d(i), j) J_{\hat{d}}(j) \right] \text{ , pour tout } i \in S$$

Les quantités $J^*(i)$et$J_{\hat{d}}(i)$ sont aussi connus sous le nom de, respectivement, la fonction de valeur optimale et la fonction valeur d'une politique $\hat{d}$, γ est un facteur de dévaluation compris entre 0 et 1 et permettant, selon sa valeur, de prendre en compte les récompenses plus ou moins loin dans le futur pour le choix des actions de l'agent et $\boldsymbol{r}$ est la récompense.

Dans les deux sous sections suivantes nous allons présenter deux algorithmes basés sur les équations de Bellman l'algorithme d'itérations sur les valeurs et l'algorithme de l'itération sur les politiques.

A. L'algorithme des itérations sur les valeurs (IV) :

Etape 1 : $k = 1$ et pour tout $i \in S$ et $a \in A(i)$, $Q^k(i,a) = 0$ et spécifier si $\epsilon > 0$ (ϵ est utilisé pour permettre l'exploration)

Etape 2 : pour tout $i \in S$, calculer

$$Q^{k+1}(i,a) \leftarrow \sum_{j=1}^{|S|} p(i,a,j)\left[r(i,a,j) + \gamma \max_{b \in A(j)} Q^k(j,b)\right]$$

Etape 3 : pour tout $i \in S$, calculer

$$J^{k+1}(i) = \max_{a \in A(i)} Q^{k+1}(i,a) \text{ et } J^k(i) = \max_{a \in A(i)} Q^k(i,a)$$

Si $\left\|\overrightarrow{J^{k+1}} - \overrightarrow{J^k}\right\| < \epsilon(1-\gamma)/2\gamma$, aller à l'étape 4

Si non incrémenter k de 1, et aller à l'étape 2

Etape 4 : pour tout $i \in S$, choisir $d_\epsilon(i) \in \arg\max_{b \in A(i)} Q^k(i,b)$, où $\hat{d}_\epsilon$ représente le politique ϵ-optimale.

B. L'algorithme des itérations sur les politiques (IP) :

Etape 1 : $k = 1$ et choisir arbitrairement une politique d_k

Etape 2 : (Evaluation de la politique) Initialiser les Q-valeurs pour toutes les états et actions arbitrairement (par 0 de préférence)

Puis utiliser la transformation suivante jusqu'à avoir des Q-valeurs qui convergent aux sens de l'ϵ-approximation :

$$Q_{\hat{d}_k}(i,a) \leftarrow \sum_{j=1}^{|S|} p(i,a,j)\left[r(i,a,j) + \gamma Q_{\hat{d}_k}(i, d_k(j))\right]$$

Etape 3 : (Amélioration de la politique) Générer une nouvelle politique $\hat{d}_{k+1}$ en utilisant la relation suivante :

$$d_{k+1}(i) \in \arg\max_{u \in A(i)} Q_{\hat{d}_k}(i,u) \text{ Pour tout } i \in S$$

Etape 4 :

Si la politique $\hat{d}_{k+1}$ est identique à $\hat{d}_k$, arrêter

Sinon incrémenter k de 1 et revenir à l'étape 2

4.1.2. Processus de décision markoviens & programmation dynamique

Un problème de décision de Markov (ou markovien) est un problème de décision séquentielle dans lequel une action doit être sélectionnée dans chaque état de décision visité par le système. La contribution de Bellman dans la théorie du contrôle stochastique montrait que la complexité de calcul pour ces problèmes peut être réduite grâce à la programmation dynamique (DP), cependant, à grand échelle, cette dernière s’est montrée inefficace, à savoir avec ces deux méthodes classiques, qui sont l’itération de politique (IP) ou l’itération de valeur (IV), et la nécessité de calculer, stocker et manipuler les matrices de probabilités de transition était son premier obstacle, on parle souvent dans ce cas, de malédiction de dimension (expression inventée par Richard Bellman dans son livre Dynamic programming en 1957) et la puissance de l’apprentissage par renforcement consiste justement en sa capacité à résoudre des problèmes complexes et de grand échelle en même temps.

Dans un problème de décision de Markov, le système est conduit par des chaînes de Markov sous-jacentes, autrement dit, on construit récursivement la chaine principale qui passe de manière aléatoire d'un état à un autre, par incréments de temps discrets, et la probabilité de passer de l'état actuel à l'état suivant dépend uniquement de l'état actuel et non de l'état antérieur du système (Propriété fondamentale des processus Markoviens). Le système est tenu de choisir une **action** ou une commande parmi un ensemble d'actions. Une politique définit l'action à choisir dans chaque état : c'est un mappage de l'ensemble des états à l'ensemble des actions. Une récompense immédiate, qui peut être positive, négative ou nulle, est gagnée lors du passage d'un état à un autre sous l'influence d'une action. La mesure de performance d'une stratégie est généralement une fonction (la fonction objectif) des récompenses immédiates obtenues lorsque la stratégie associée est suivie sur un horizon temporel prédéterminé. L'horizon temporel peut être fini ou infini, en fonction de l'intention du concepteur du système. Le PDM consiste à trouver la politique optimale qui optimise les mesures de performances et une chaîne de Markov est unique pour chaque politique, il est à noter que ce qui a été mentionné auparavant prend en compte des événements qui se produisent dans un temps discret.

Il existe plusieurs mesures de performances, mais les deux les plus courantes sont la récompense réduite et la récompense moyenne. La première, est la somme des récompenses gagnées à long terme sur l'ensemble de l'horizon temporel lorsque la politique associée est poursuivie, tandis que la récompense moyenne est la récompense espérée (expected en Anglais, de l’espérance en statistiques) en une seule étape.

Nous supposerons que l'espace d'états S et l'espace d'action dans l'état i, $\boldsymbol{A(i)}$, pour chaque $i \in S$ sont des ensembles finis et l'horizon temporel de mesure de la métrique de performance est d'une durée infinie ainsi que des chaînes de Markov *régulières*. Soit $\mathbf{d}(\mathbf{i})$ l'action choisie dans l'état i lorsque la politique est poursuivie : à noter que $\hat{d}$ contient $|S|$ éléments et soit $\boldsymbol{r(i,a,j)}$ la récompense immédiate obtenue en passant de l'état i à l'état j sous l'action a, et $\boldsymbol{p(i,a,j)}$, la probabilité de cette transition.

Définition 1 : La récompense réduite d'une politique $\hat{d}$ commençant à l'état i est définie ainsi :

$$J_{\hat{d}}(i) = \lim_{k\to\infty} E\left[\sum_{s=1}^{k} \gamma^{s-1}\, r(x_s, d(x_s), x_{s+1}) \middle|\, x_1 = i\right]$$

Où $x_s$représente l'état précédent, γ est le facteur de réduction et E est l'opérateur de l'espérance.

Définition 2 : La récompense moyenne d'une politique $\hat{d}$ commençant à l'état i est définie ainsi :

$$\rho_{\hat{d}}(i) = \lim_{k\to\infty} \frac{E\left[\sum_{s=1}^{k} r(x_s, d(x_s), x_{s+1}) \middle|\, x_1 = i\right]}{k}$$

Pour les MDP dans lesquels toutes les chaînes de Markov sont régulières, la récompense moyenne est indépendante de l'état de départ.

Définition 3 : Pour les récompenses réduites, le PDM consiste à déterminer $\hat{d}^*$ tel que :

$$J_{\hat{d}^*}(i) \geq J_{\hat{d}}(i) \; pour\; tout\; \hat{d}\; et\; pour\; tout\; i \in S$$

Pour les récompenses réduites, le PDM consiste à déterminer $\hat{d}^*$ tel que :

$$\rho_{\hat{d}^*}(i) \geq \rho_{\hat{d}}(i) \; pour\; tout\; \hat{d}\; et\; pour\; tout\; i \in S$$

4.1.3. Méthodes Monte Carlo (MC)

Dans les méthodes Monte Carlo (MC), on parle d'épisodes, où on se déplace d'un état initial vers un état terminal, en enregistrant les états, les actions et les récompenses, puis on calcule les $V(s)$ et $Q(s)$ pour chaque état que nous avons traversé, on a donc besoin d'attendre jusqu'à la fin d'un épisode pour pouvoir les calculer.

Nous répétons ce processus en reproduisant plusieurs épisodes, et après chaque épisode, nous obtenons la moyenne des valeurs de $V(s)$ et $Q(s)$. Un algorithme de prédiction MC consisterait donc à simuler le plus d'épisodes possible, pour avoir les valeurs les plus précises possibles.

4.1.4. Différences temporelles (TD)

Les $TD(\lambda)$ peuvent être vu comme la fusion entre les méthodes DP et MC. Il diffuse des épisodes mais n'a pas besoin d'attendre la fin pour connaître le retour. Il calcule la valeur de l'état actuel en fonction d'estimations d'autres états. $TD(0)$ fait référence au fait de ne regarder qu'une seule étape vers l'avant (l'état suivant selon l'action que l'agent va prendre), puis de calculer la valeur de l'état actuel. Il existe deux fameux algorithmes de TD qui sont : Q-Learning et SARSA.

A. L'algorithme de Q-Learning

Nous donnons une définition formelle de l'algorithme de Q-Learning qui représente le cœur de notre travail dans le chapitre conception :
Soit un environnement décrit par les deux ensembles d'états $S = \{s_0, s_1, \dots, s_n\}$ et d'actions $A = \{a_0, a_1, \dots, a_n\}$. Il s'agit d'un ensemble d'actions que l'agent peut sélectionner dans chaque état en S. Chaque fois qu'une action est effectuée pour un état s_t, l'agent reçoit une récompense $R(s_t, a_t)$. Le rôle de l'agent est d'apprendre une politique $\pi: S \to A$, qui maximise le gain à long terme formulé ainsi (basé sur l'équation de Bellman présentée précédemment) :

$$Q(s,a) = Q(s,a) + \alpha[R(s,a) + \gamma * \max_b\{Q(\delta(s,b),b)\} - Q(s,a)]$$

Ou encore sous une autre forme (changement de parenthèses) :

$$Q(s,a) = (1-\alpha) * Q(s,a) + \alpha[R(s,a) + \gamma * \max_b\{Q(\delta(s,b),b)\}]$$

Où $\boldsymbol{\delta(s,a)}$ représente l'état issue d'appliquer l'action $\boldsymbol{a}$ à l'état $\boldsymbol{s}$
Avec cette équation, on met à jour une structure de données appelée : **Q-Table** qui est souvent représentée par un tableau à 2 dimensions qui possède comme lignes les états, et comme colonnes les actions possibles et représente *une mémoire* que les agents de l'A/R utiliseront, et qu'on va définir plus en détail dans le chapitre conception. Et donc $Q(s,a)$ représente la valeur d'une case de ce tableau qui est mise à jour à chaque fois.

Le pseudo algorithme de Q-Learning est le suivant :

Paramètres de l'algorithme : taille du pas $\alpha \in [0,1]$, et $\varepsilon > 0$

Initialiser $Q(s,a)$, pour tout $s \in S^+$, $a \in A(s)$ arbitrairement

Répéter pour chaque $episode$:

Initialiser s

Répéter pour chaque pas d'un $epsiode$:

Définir $A(s)$ à partir de s selon une politique dérivé de la Q-Table

Choisir une action a de $A(s)$, et observer $R(s,a)$ et s'

$$Q(s,a) = Q(s,a) + \alpha[R(s,a) + \gamma * \max_b\{Q(\delta(s,b),b)\} - Q(s,a)]$$

$s \leftarrow s'$

Jusqu'à $condition\ d'arrêt$

4.1.5. Tableau comparatif

Approche	Avantages	Inconvénients
MDP & DP	• Efficace et trouve les politiques optimales en temps polynomial dans la plupart des cas. • Assure une politique optimale	• Ne convient pas aux grandes instances de problèmes. • Requiert la connaissance de la matrice de transition
MC	• Peut être utilisé pour apprendre le comportement optimal directement de l'interaction avec l'environnement. (Sans modèle) • Peut être utilisé avec des modèles de simulation ou de simples échantillons.	• Ne fonctionne que pour les environnements épisodiques : qui ont des états finaux. • Doit attendre la fin d'un épisode avant de connaître la récompense. Pour les problèmes avec de très longs épisodes, cela deviendra trop lent. (Pas de bootstrapping)
TD	• N'a pas besoin de connaître la matrice de probabilité de transition comme dans DP. • TD n'a pas besoin d'attendre la fin de l'épisode pour connaître la récompense, il met à jour la valeur d'état et la valeur d'action de manière empirique.	• Pourrait être coincé dans les minima locaux. • Peut mal se comporter dans certains environnements stochastiques.

Tableau II-3 - Tableau comparatif des approches de l'A/R

4.2. Approximations basées sur les réseaux de neurones

L'approximation par réseau de neurones est utilisée pour faire une amélioration de la politique avec des actions continues. Le problème de l'amélioration d'une politique est défini comme suit : Étant données une politique $\boldsymbol{\pi_k}$ à l'épisode k et une trajectoire Monte-Carlo $(s_0, r_1, s_1, \ldots, r_L, s_L)$ de taille L générée par l'environnement en utilisant $\boldsymbol{\pi_k}$, comment définir $\boldsymbol{\pi_{k+1}} \geq \boldsymbol{\pi_k}$? En l'état, le problème est mal posé, car nous nous intéressons à un espace continu non dénombrable. Il va donc falloir passer par des approximateurs.

Une politique π_ψ paramétré par un ensemble de $\boldsymbol{n}$ paramètres $\theta \in R^n$, associe un état $\boldsymbol{s}$ et les paramètres θ à une action $a \in \mathbf{A}$ comme suit :

$$\pi_\psi : \boldsymbol{S}\ X\ R^n \rightarrow \boldsymbol{A}\ \ tq:\ s, \theta \rightarrow\ \pi_\psi(\boldsymbol{s}, \theta)$$

L'ensemble des paramètres ne suffit pas à décrire complètement une politique paramétrée, il reste à définir le modèle de la politique ψ_θ qui détermine la façon dont s et θ interagissent. Le but est de trouver les meilleurs paramètres pour un modèle ψ donné. Trouver une politique paramétrée optimale c'est trouver le meilleur ensemble de poids θ pour un modèle donné.

Politique paramétrée optimale : Étant donné un modèle de politique paramétrée ψ_θ, les paramètres $\theta \in \Theta = R^n$ sont optimaux, pour le modèle donné, s'ils satisfont la contrainte suivante :

$$\theta^* = \underset{\theta \in \boldsymbol{\Theta}}{\boldsymbol{argmax}}\, \boldsymbol{J}(\boldsymbol{\pi}_\theta)$$

Où J (critère γ-pondéré) permet de quantifier la politique $\boldsymbol{\pi}$ par un scalaire dans un **PDM** à espaces continus : $\boldsymbol{J(\pi)} = \int_{\boldsymbol{S}} \boldsymbol{T_0(s_0)} \boldsymbol{E}[\sum_{\boldsymbol{t=1}}^{\infty} \boldsymbol{\gamma^{t-1} r_t} | \boldsymbol{\pi}, \boldsymbol{s_0}] \boldsymbol{ds_0}$,

Où $0 \leq \gamma \leq 1$ représente l'importance des récompenses futures et $\mathbf{T_0}$ la densité de probabilité définie sur l'état initial.

4.2.1. Policy gradient

Le but de cette méthode est de trouver la manière de modifier les poids $\boldsymbol{\theta}$ pour maximiser le critère J en trouvant le gradient de ce critère par rapport à θ sans qu'il dépende de l'un des gradients de l'environnement :

$\nabla R\ (gradient\ de\ la\ récompense\ )ou\ \nabla T(gradient\ du\ modéle\ de\ transition)\ qui\ sont\ supposé\ inconus$

J Peut-être décomposé par la densité de probabilité d'une trajectoire, et la somme des récompenses décomptées de cette trajectoire comme suit **:**

$$J(\pi_\theta) = \int_\tau d^{\pi_\theta}(\tau) R_\gamma(\tau) d\tau$$

Où $d^{\pi_\theta}(\tau)$ est la densité de probabilité de la trajectoire état-action τ et $R_\gamma(\tau)$, la somme des récompenses décomptées de la trajectoire τ

Cette décomposition permet de supprimer la dépendance des récompenses aux paramètres θ la dérivée est désormais comme suit :

$$\frac{\partial J(\pi_\theta)}{\partial \theta} = \int_\tau R_\gamma(\tau) \frac{\partial d^{\pi_\theta}(\tau)}{\partial \theta} d\tau$$

$$= \int_\tau d^{\pi_\theta} R_\gamma(\tau) \frac{\partial log\, d^{\pi_\theta}(\tau)}{\partial \theta} d\tau$$

L'insertion du log permet de retirer de la multiplication les termes non dépendants de θ.

4.2.2. REINFORCE : Monte Carlo Policy Gradient

Cette méthode est appliquée Quand il s'agit d'une politique paramétré stochastique, elle consiste à trouver une estimation non biaisée du gradient de la politique :

$$\frac{\partial J(\pi_\theta)}{\partial \theta} = E[R_\gamma(\tau) \sum_{t=0}^{L} \frac{\partial \log \pi_\theta(a_t | s_t)}{\partial \theta} | \tau \sim \pi_\theta]$$

Il faut noter que cette approche ne dépend pas de l'environnement. Néanmoins, comme toutes les méthodes de Monte-Carlo, elle souffre d'une grande variance

4.2.3. Actor-critic

Les architectures acteur-critique combinent l'apprentissage de la fonction valeur ainsi que l'amélioration de la politique et fait les deux en même temps

B. Gradient d'une politique stochastique avec fonction de valeur

On définit la densité de probabilité d'un état par la formule suivante :

$$d_\gamma^\pi(s) = \int_S T_0(s_0) \sum_{t=1}^{\infty} \gamma^{t-1} d^\pi(s|s_0, t) ds_0$$

Ou $d^\pi(s|s_0, t)$ est la densité de probabilité d'être dans l'état **s** au temps **t** en partant de l'état s_0 tout en suivant la politique stochastique π.

Le critère $\boldsymbol{J}$ défini comme étant : $\boldsymbol{J(\pi_\theta) = \int_\tau d^{\pi_\theta}(\tau) R_\gamma(\tau) d\tau}$ s'écrit désormais comme suit :

$$\boldsymbol{J(\pi) = \int_S d_\gamma^\pi(s) \int_A \pi_\theta(a|s) R(s,a) da ds}$$

Il faut noter également que le théorème du gradient sur les politiques **(Sutton et al., 1999)** avait montré que la dérivée du critère d'optimisation peut s'écrire à partir de la fonction de valeur **Q**. Ces trois points cités dans cette section ont donné naissance aux méthodes d'apprentissage de la politique $\boldsymbol{\pi}$ et de la fonction de valeur $\boldsymbol{Q^\pi}$ en même temps. On définit ainsi le gradient de la politique stochastique paramétrée $\boldsymbol{\pi_\theta}$ avec fonction de valeur comme suit :

$$\boldsymbol{\frac{\partial J(\pi_\theta)}{\partial \theta} = \int_S d_\gamma^\pi(s) \int_A \frac{\partial \ \pi_\theta(a|s)}{\partial \theta} Q^\pi(s,a) da ds}$$

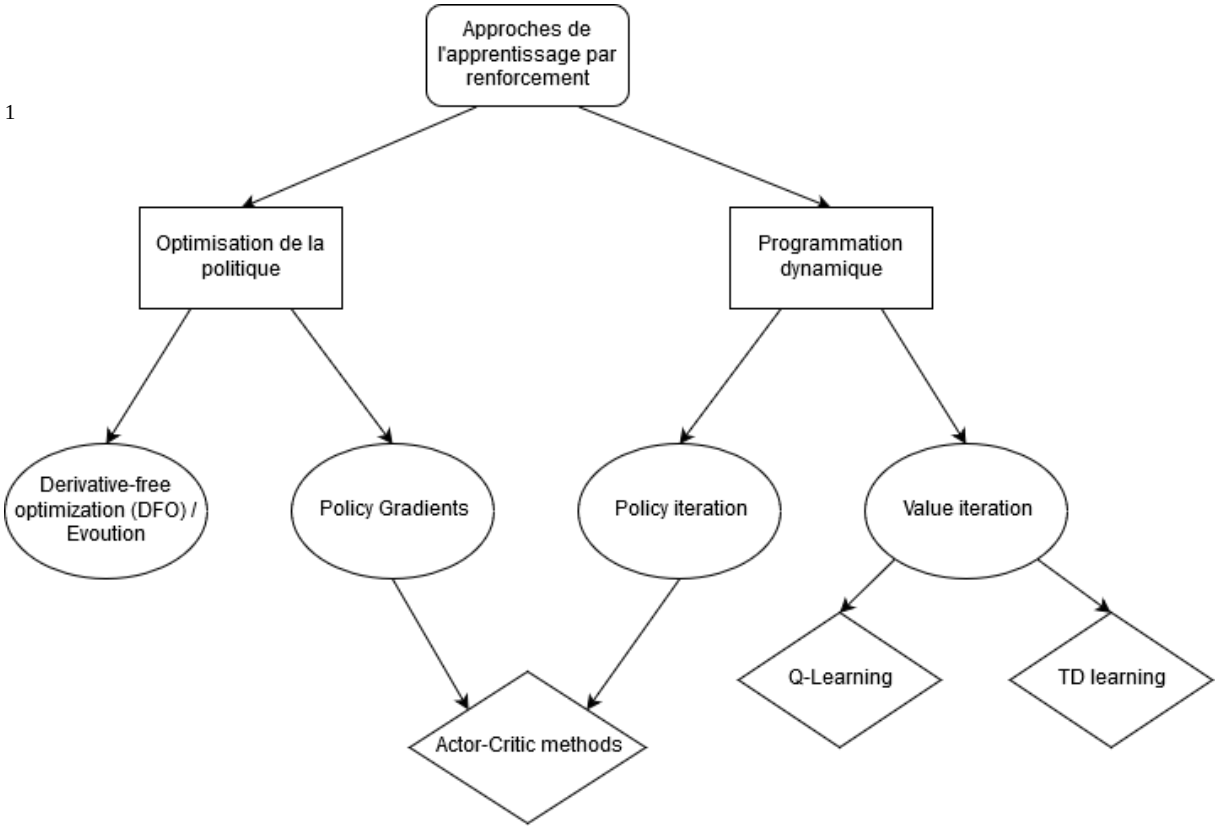
C. Gradient d'une politique déterministe avec fonction de valeur

Pour tout **PDM**, le gradient du coût d'une politique $\boldsymbol{\pi_\theta}$ paramétrée, déterministe s'écrit :

$$\boldsymbol{\frac{\partial J(\pi_\theta)}{\partial \theta} = \int_S d_\gamma^\pi(s) \frac{\partial \ Q^\pi(s,a)}{\partial a}|_{a=\pi_\theta(s)} \frac{\partial \ \pi_\theta(a|s)}{\partial \theta} ds}$$

- **Schéma récapitulatif**

[1]



[1] Joelle Pineau, "Reinforcement Learning: Basic concepts", CIFAR RLSS 2017, Montréal

Figure II-3 - Taxonomie des approches de l'A/R

5. Les applications

L’apprentissage par renforcement classique a été appliqué dans un grand nombre de domaines avec succès. Ici, nous énumérons quelques études de cas liées à la gestion des opérations. Nous décrivons en particulier les spécificités des algorithmes qui les ont adaptés au domaine d'application (Gosavi, 2009).

- Des algorithmes actualisés à temps continu (Bradtke & Duff, 1995) ont été utilisés pour la planification des ascenseurs ((Crites & Barto, 1996), (Pepyne, Looze, Cassandras, & Djaferis, 1996)) car la structure du problème était sous-jacente à une chaîne de Markov à temps continu.
- Le problème de l'emploi du temps dans les ateliers de (Zhang & Dietterich, 1995) avait un caractère épisodique et c'est pourquoi TD (λ) devint préférable ((Sutton, 1985), (Peng & Williams, 1993)).
- Les problèmes de maintenance préventive sont généralement des SMDP avec une fonction objectif non actualisée, ce qui rend SMART (Das, Gosavi, Mahadevan, & Marchalleck, 1999) et R-SMART (Gosavi, 2004b) appropriés.
- Parmi les autres applications de ces algorithmes, citons les réseaux (QoS) avec (Akar & Sahin, 2003) et la sélection du fournisseur pour les chaînes d'approvisionnement dans (Pontrandolfo, Gosavi, Okogbaa, & Das, 2002).
- Le problème de routage des véhicules à guidage automatique (VGA) dans (Tadepalli & Ok, 1998) est l’une des rares études de cas d’A/R sur la construction de modèles pour des problèmes à grande échelle. Le modèle qu’ils apprennent est capable de saisir la dynamique complexe du problème de VGA.
- Un « problème de gestion des revenus » bien connu peut être défini comme une récompense moyenne SMDP ((Gosavi, 2004a), (GOSAVII, Bandla, & Das, 2002)), mais il possède une structure de récompense unique, avec une grande partie de la récompense concentrée dans certains États qui rendent SMART, qui est TD(0), instable.
- Les travaux liés à l'hyperheuristique ((E. K. Burke, Kendall, & Soubeiga, 2003), (E. Burke et al., 2003), (Nareyek, 2003)) peuvent être utilisés lorsque l’A/R doit être utilisé de manière dynamique pour sélectionner une métaheuristique.
- Un SMDP avec un facteur de réduction a été utilisé pour un problème de gestion de réseau de téléphonie cellulaire (Singh & Bertsekas, 1997) permettant une combinaison pratique avec un approximateur de fonctions à base de neurones.

- Le problème de la gestion des stocks des détaillants chez (Van Roy, Bertsekas, Lee, & Tsitsiklis, 1997) ont utilisé le Q-Learning régulier avec un facteur d'escompte.
- (Makar, Mahadevan, & Ghavamzadeh, 2001) ont recours à une liste de tâches hiérarchique, car la tâche de planification AGV dont ils estiment avoir des contrôleurs à plusieurs niveaux.

Les jeux vidéo pouvant varier considérablement, tout en présentant des objectifs intéressants et stimulants pour les humains, ils constituent un excellent banc d'essai pour les agents de l’apprentissage par renforcement profond, qui a connu son apogée en 2015, après la réussite qu’a connu DeepMind. (Arulkumaran, Deisenroth, Brundage, & Bharath, 2017)

- Premier algorithme à avoir du succès avec ces jeux à partir de leurs graphismes, le DQN s'est imposé comme une étape importante dans le développement des algorithmes A/RP.
- ViZDoom fournit une interface au jeu de tir à la première personne de Doom et, faisant écho à la popularité des compétitions d’esports, les compétitions ViZDoom ont maintenant lieu à la conférence annuelle de l’IEEE sur l’intelligence numérique dans les jeux.
- TorchCraft de Facebook et l’environnement d’apprentissage StarCraft II de DeepMind fournissent respectivement des interfaces avec les jeux de stratégie en temps réel StarCraft et StarCraft II, posant des défis en matière de microgestion et de planification à long terme.
- Le projet Malmo de Microsoft a exposé une interface au jeu de sandbox de Minecraft. Les deux environnements fournissent des plates-formes personnalisables pour les agents de l’A/R dans des environnements 3D.
- La plupart des approches DRL se concentrent sur des actions discrètes, mais certaines solutions ont également été développées pour des problèmes de contrôle continus. De nombreux articles de DRL en contrôle continu ont utilisé le moteur physique MuJoCo pour obtenir une dynamique relativement réaliste des problèmes de contrôle en continu multi-articulations, et des efforts ont été déployés pour normaliser ces problèmes.

- Pour aider à la normalisation et à la reproductibilité, la plupart des domaines de l’A/R susmentionnés et d’autres sont disponibles dans OpenAI Gym, une bibliothèque et un service en ligne permettant aux utilisateurs de s’interfacer aisément avec les résultats des algorithmes de l’A/R et de les partager publiquement, ainsi qu’une autre bibliothèque de Google, appelée « Dopamine », qui a été développée dans un cadre de recherche pour le prototypage rapide d'algorithmes d'apprentissage par renforcement.

6. Conclusion

Pour conclure, la liste des applications citées dans la section précédente n’est pas exhaustive, et représente seulement les travaux que nous avons jugé intéressants selon notre point de vue. Il est probable que le domaine de l’A/R appliqué explosera dans les années à venir en raison de sa capacité à résoudre des problèmes précédemment considérés comme insolubles, et surtout avec l’avancée qu’a connu la branche de l’A/R profond ou Deep Reinforcement Learning.

III. Chapitre 3 : L'intégration de l'A/R aux métaheuristiques

1. Introduction

Viser le meilleur est l'un des traits les plus fondamentaux de l'intelligence. Dans toutes les activités, l'être humain a tendance à maximiser les avantages ou, de manière générale, à minimiser les inconvénients dans un sens dépendant du contexte. La recherche du meilleur semble si naturelle avec l'esprit humain que si nous ne le reconnaissons pas dans le comportement de quelqu'un, nous le qualifions facilement d'irrationnel.

La résolution des problèmes d'optimisation combinatoire rencontre plusieurs inconvénients, on peut citer la convergence vers des optimums locaux ainsi que la difficulté de trouver des méthodes qui peuvent résoudre différents problèmes ayant différentes caractéristiques efficacement. Les métaheuristiques ont été introduites pour remplacer une classe de méthodes spécifique par une approches qui répond au même besoin, mais avec l'avancement de l'apprentissage automatique, on a senti le besoin d'hybrider les métaheuristique avec ce dernier (le machine Learning), qui présente des résultats assez satisfaisants face à ces contraintes. Dans ce chapitre nous essayerons de présenter l'intégration de l'A/R aux métaheuristiques, où nous allons parler de la relation qui existe entre ces deux branches, et les différentes opportunités d'hybridation. Nous finirons par présenter des travaux existants sur cette dernière et ainsi conclure avec un tableau qui regroupe ces travaux.

2. Motivations

On peut dire que l'idée d'hybrider l'apprentissage par renforcement aux métaheuristiques n'est pas nouvelle puisque le premier travail scientifique date de 1995 (Gambardella & Dorigo, 1995) cependant, c'est durant ces 10 dernières années que le nombre de travaux a augmenté considérablement. Le tableau ainsi que les graphes suivants nous montrent que ce domaine est en croissance et peut représenter un bon investissement en termes de recherche scientifique.

Année	Nombre de publications	Année	Nombre de publications	Année	Nombre de publications
1995	4	**2003**	69	**2011**	498
1996	5	**2004**	103	**2012**	1175
1997	7	**2005**	118	**2013**	923
1998	14	**2006**	130	**2014**	635
1999	12	**2007**	175	**2015**	597
2000	62	**2008**	248	**2016**	647
2001	66	**2009**	501	**2017**	879
2002	48	**2010**	317	**2018**	1006

Tableau III-1 - Le nombre de publications pour l'hybridation A/R et métaheuristiques

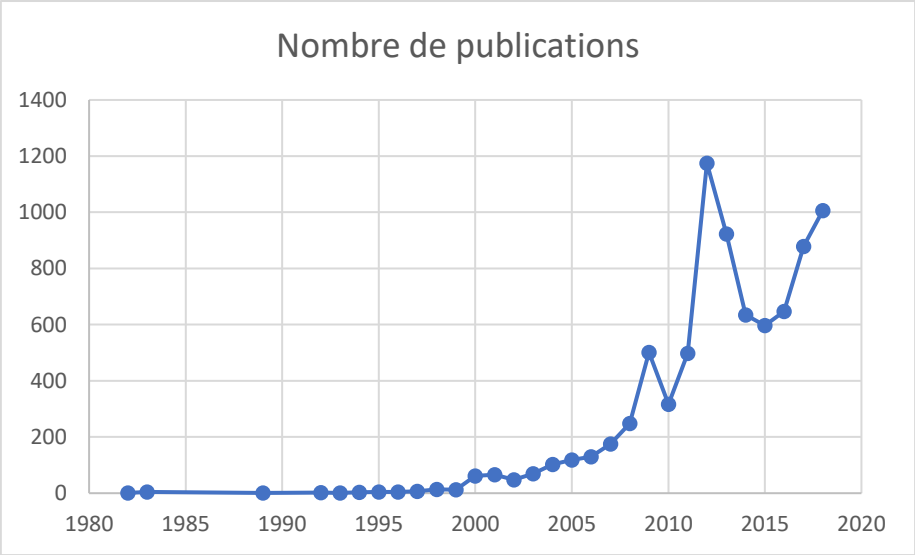


Figure III-1 - Nombre de publications selon google scholar pour la requête : « reinforcement learning and metaheuristics »

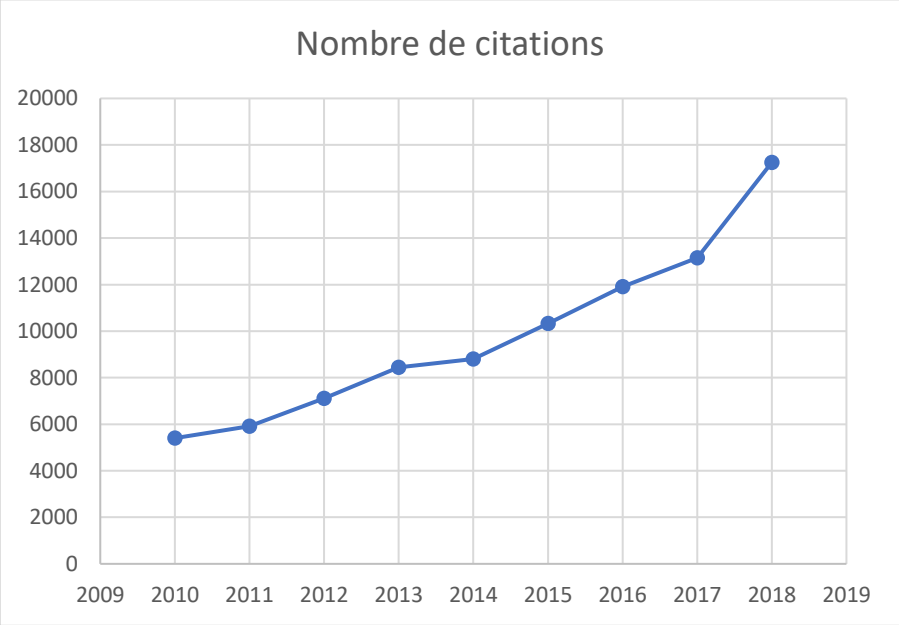


Figure III-2 - Nombre de citations pour les 10 dernières années selon google scholar pour la requête : « reinforcement learning and metaheuristics »

L'augmentation du nombre de publications durant ces 10 dernières années est due en premier lieu à l'amélioration de la puissance de calcul. On peut remarquer qu'il y a eu un pique en nombre d'article en 2012 ceci peut être expliqué par l'avènement du Deep Learning (Apprentissage profond), en effet les exploits réalisés par ce dernier ont poussés les chercheurs à utiliser d'autres techniques du machine Learning comme le A/R dans plusieurs travaux

cependant, les résultats obtenues par l'utilisation de l'Apprentissage profond dans différents domaines a engendré la concentration des chercheurs dans les applications de ce dernier ce qui a provoqué une baisse dans les travaux utilisant les autres types du machine Learning en générale et l'apprentissage par renforcement en particulier dans les deux années suivantes notamment dans l'application du A/R dans l'optimisation combinatoire. Une augmentation considérable a été remarqué depuis 2014 sur le nombre de publication de notre domaine d'étude ceci est dû à la percée faite par le A/R dans les jeux (L'apprentissage par renforcement a permis de battre le meilleure joueur au monde dans le Jeux Go un jeu ou le nombre de positions possibles sur le plateau dépasse le nombre d'atomes dans l'Univers et qui nécessite beaucoup plus de réflexion que les échecs) ce qui a créé une envie pour l'utilisation de cette approche dans tous les domaines et notamment dans l'optimisation combinatoire.

3. Intégration de l'apprentissage par renforcement aux métaheuristiques

Pour plus de détails concernant les métaheuristiques hybrides, veuillez consulter la partie de l'annexe : Classification des métaheuristiques hybrides.

A. Relation métaheuristiques-A/R et opportunités d'apprentissage

Le lien entre l'apprentissage par renforcement et les métaheuristiques n'est pas fin, avec un ensemble d'états, d'actions et une fonction de récompense le problème d'A/R peut être réduit à un problème de recherche de politique. Ainsi que des problèmes similaires, tels que l'exploration et l'exploitation, dans le lexique A/R, appelés diversification et intensification dans la littérature métaheuristique (Wauters, Verbeeck, De Causmaecker, & Vanden Berghe, 2013). On peut citer donc la similarité entre les ACO et l'A/R (Stutzle & Dorigo, 2002) (Dorigo & Blum, 2005), puisque même s'ils aient différentes sources d'inspiration, ils ont plusieurs caractéristiques en commun, telles que l'utilisation du processus de décision de Markov comme moyen de formuler le problème et fournir des preuves de convergence pour les algorithmes, ainsi que la similarité entre la fonction valeur de l'action dans l'A/R et la phéromone dans l'ACO, dans l'A/R, l'action qui maximise la fonction valeur est choisie, le principe du ACO est assez similaire en effet, une fourmi choisie le chemin avec la plus grande quantité de phéromone. (Bianchi, Ribeiro, & Costa, 2009)

Les méthodes d'apprentissage par renforcement peuvent être utilisées de différentes façons pour stimuler les métaheuristiques. Par exemple l'A/R peut être utilisé pour apprendre les propriétés de bonnes solutions initiales pour une fonction objectif. On peut aussi appliquer de

l'A/R pour la sélection du voisinage. En effet, l'apprentissage permet d'apprendre quels sont les meilleurs voisinages pour construire ou modifier une solution pendant la recherche. L'A/R peut être également utilisé pour la sélection d'heuristiques dans les hyperheuristiques. Pour résumer nous pouvons dire que les composantes sur lesquelles le A/R est généralement appliqué sont :

- La solution initiale
- La fonction objectif
- Le voisinage
- La sélection de nouvelles solutions

Il faut noter aussi que la présence d'une composante aléatoire (génération de la solution initiale dans une heuristique ou dans une métaheuristique d'une manière aléatoire par exemple) permet d'inclure le A/R dans la recherche en remplaçant cette composante aléatoire par une approche d'apprentissage par renforcement, l'algorithme développe ainsi un mécanisme de décision plus intelligent.

B. Etat et action

Avant d'appliquer l'A/R pour un problème le nombre d'état et le nombre d'action pour chaque étape doivent être défini au préalable. Plusieurs approches existent pour faire cela elles sont comme suit :

- **Search-dependent state space :** Utilise des observations du processus de recherche lui-même, tel que l'itération en cours, le nombre d'itérations non améliorantes, ou amélioration totale par rapport à la solution initiale.
- **Problem-dependent :** Définie par l'utilisation de caractéristiques génériques du problème.
- **Instance-dependent :** Se base sur les caractéristiques de l'instance du problème.

Les trois approches peuvent être combinées. Quand un apprentissage est effectué sur un espace de recherche search-dependent ou problem-dependent il peut être transféré vers d'autres instances du même problème.

L'ensemble des actions possibles dans chaque état est défini par l'ensemble des paramètres ou des composants de la métaheuristique que l'on veut apprendre.

C. Fonction récompense

Dans les problèmes d'optimisation combinatoire, une seule valeur numérique est disponible, indiquant la qualité d'une solution, ce qui veut dire qu'on n'a pas toutes les composantes d'une

solution, de ce fait avoir de l'expérience sur toutes les composantes d'une solution est très rare (aucune métrique qui caractérise toutes les composantes d'une solution). Dans l'A/R cette expérience est mise en valeur grâce à la fonction de récompense. Il y a trois paramètres à satisfaire pour la fonction de récompense pour l'application du A/R dans les métaheuristiques (Wauters et al., 2013) :

- Donner la meilleure récompense aux meilleures solutions
- Trouver une politique efficace de l'A/R pour la recherche de solution
- Elle doit être normalisée pour qu'elle soit transférable pour de nouvelles instances du problème traité.

4. Etude comparative des travaux

D'après notre étude, on peut distinguer quatre utilisations du A/R dans les métaheuristiques :

- La première hybridation consiste à utiliser le A/R comme une nouvelle composante de la métaheuristique (l'utilisation directe du A/R dans la métaheuristique), dans cette approche on utilise l'apprentissage par renforcement soit pour générer les solutions initiales soit pour apprendre les meilleures valeurs de la fonction d'évaluations pour guider la recherche vers les meilleures solutions soit pour combiner les deux. L'article (Júnior, Neto, & De Melo, 2010) qui explique l'utilisation du A/R dans les AG, est un très bon exemple de cette hybridation.
- La deuxième manière d'utiliser le A/R dans les MH comporte son utilisation pour trouver les meilleures valeurs d'un ou plusieurs paramètres de la métaheuristique, l'article de (Battiti & Campigotto, 2008) explique l'utilisation de l'apprentissage par renforcement pour trouver la valeur optimale de la taille de la liste tabou dans une métaheuristique qui s'appelle recherche tabou réactif pour la résolution du problème MAX-SAT.
- Dans la troisième approche le A/R est employé pour sélectionner un voisinage (sélectionner une recherche locale parmi plusieurs par exemple), dans l'article de (dos Santos, de Melo, Neto, & Aloise, 2014) le A/R est utilisé pour améliorer la métaheuristique appelé recherche à voisinage variable (RVV) pour trouver le meilleure voisinage (la meilleure recherche locale) pour la résolution du problème du voyageur de commerce.
- Dans La dernière méthode d'hybridation le A/R est utilisé pour remplacer une heuristique qui peut être une composante d'une métaheuristique (le A/R peut être utilisé comme une recherche locale par exemple), dans l'article de (Júnior et al., 2010) les auteurs ont remplacé

la recherche locale dans l'heuristique GRASP par l'algorithme du Q-Learning pour résoudre le problème du voyageur de commerce.

Le tableau suivant résume les différents travaux qu'on a étudié :

Article	Année	Métaheuristique	Approche d'hybridation	Algorithme d'A/R utilisé	But de l'hybridation
Gambardella et al	1995	Systèmes de fourmis	Directe en utilisant la table des q-valeurs pour guider la recherche vers les meilleurs solutions	Q-Learning	Résolution du problème du voyageur de commerce
Moll et al	1998	2-opt (recherche locale)	Directe pour apprendre les meilleures valeurs de la fonction objectives	TD (λ)	Résolution du problème Dial-a-ride
A.E. Eiben et al	2006	Algorithme génétique	Ajustement des paramètres de l'AG à savoir croisement, mutation et la sélection	Q-Learning/SARSA	Calibration de paramètres dans un algorithmes génétiques
De Lima et al	2010	GRASP	Génération de la solution initiale, et remplacement de la recherche locale par le A/R	Q-Learning	Résolution du problème de voyageur de commerce
De Lima et al	2010	AG	Application directe du A/R dans l'AG (dans la Génération de la solution initiale ainsi que pour guider la recherche)	Q-Learning	Résolution du problème de voyageur de commerce
João Paulo Queiroz dos Santos et al	2014	Recherche à voisinage variable	Sélection d'un voisinage (sélection d'une recherche locale)	Q-Learning	Résolution du problème de voyageur de commerce
Zhou et al	2016	Descent based local search	Remplacement de la recherche locale par un A/R	/	Résolution du problème de coloration de graph

Fairee et al	2018	Colonie d'abeilles artificiels	Directe en utilisant un vecteur de renforcement pour guider la recherche	/	Résolution d'un problème à D dimensions
EMARY et al	2018	Optimisation par loups gris	Ajustement du paramètre de diversification de cet algorithme	Apprentissage par renforcement profond	Résolution du problème de sélection d'attributs

Tableau III-2 - Tableau comparatif des travaux

5. Conclusion

La résolution efficace de problèmes complexes doit impliquer des idées inspirées de différents paradigmes : métaheuristiques, programmation, apprentissage automatique, théorie des graphes, calcul parallèle et distribué, etc. Les métaheuristiques pures ne sont généralement pas bien adaptées aux problèmes complexes et de grande dimension. Les métaheuristiques hybrides représentent en réalité les algorithmes les plus efficaces pour de nombreux problèmes difficiles réels, et ceci est prouvé par le grand nombre de métaheuristiques hybrides efficaces proposées pour résoudre une grande variété de problème.

IV. Chapitre 4 : Conception et réalisation

1. Introduction

La sélection d'attributs pour la classification peut être modélisée comme étant un problème d'optimisation combinatoire dont l'objectif est de sélectionner un sous ensemble de n attributs parmi un ensemble initial de N attributs tel que $n < N$. L'espace de recherche comporte 2^N solutions et la qualité d'une solution peut correspondre à la qualité du modèle de classification construit en utilisant ce sous ensemble d'attributs. Dans ce chapitre, nous proposons des méthodes pour booster la métaheuristique BSO en utilisant l'apprentissage par renforcement afin de faire de l'optimisation intelligente.

Les parties précédentes de ce mémoire portaient sur l'étude bibliographique des différents concepts liés au sujet, et elles ont été réparties sur trois chapitres. Dans le premier nous avons présenté les notions les plus importantes liées à la sélection d'attributs. Le deuxième chapitre a été consacré à la présentation du concept d'apprentissage par renforcement et ses approches et le troisième chapitre a été dédié aux différentes techniques d'intégration de l'apprentissage par renforcement dans les métaheuristiques. Dans ce présent chapitre, nous allons présenter notre contribution consistant à proposer des variantes hybrides de BSO intégrant une technique d'A/R pour la sélection d'attributs. Notre choix est inspiré de l'algorithme original de Q-Learning (Watkins & Dayan, 1992), puisque comme nous avons déjà présenté dans le chapitre d'apprentissage par renforcement, cette méthode est sans modèle et passive, qui possède un espace de recherche réduit par rapport aux méthodes basées sur un modèle comme SARSA et passive car l'environnement est déterministe (le choix de l'action répond aux objectifs que nous allons nous fixer par la suite).

2. Hybridation de BSO avec l'A/R

Le but de notre travail est de proposer des hybridations de BSO avec l'A/R pour la résolution du problème de sélection d'attributs dans le but d'améliorer la métaheuristique classique. Ces améliorations peuvent concerner :

- Le temps d'exécution : les hybridations proposées peuvent permettre de réduire le temps de BSO toute en gardant ou en améliorant la qualité des solutions trouvées.
- L'exactitude du modèle (Accuracy) : la BSO classique donne de très bon résultats en termes d'exactitude du modèle, mais une l'hybridation avec l'A/R peut permettre d'améliorer cette exactitude.
- Le nombre d'attributs : l'hybridation peut permettre de trouver des solutions de meilleures ou de même qualité que les solutions trouvées par BSO avec moins d'attributs

Avant d'intégrer l'A/R à la métaheuristique BSO, nous devons d'abord examiner cette dernière afin d'identifier les opportunités d'amélioration en utilisant l'A/R.

2.1. Opportunités d'amélioration

Le processus de recherche dans la métaheuristique BSO classique illustré dans Figure IV-1 – Le processus de recherche de BSO se déroule comme suit :

1. Déterminer la prochaine $refSol$ et l'insérer dans la liste tabou.
2. Définir la searchRegion à partir de refSol, et affecter une solution initiale pour la recherche locale pour chaque abeille
3. Chaque abeille effectue une recherche locale et évalue les solutions visitées.
4. Chaque abeille retourne la meilleure solution trouvée durant sa recherche.
5. Les meilleures solutions sont insérées dans la table $Dance$.
6. Une nouvelle solution de référence est sélectionnée parmi celles qui sont dans la table *Dance.*

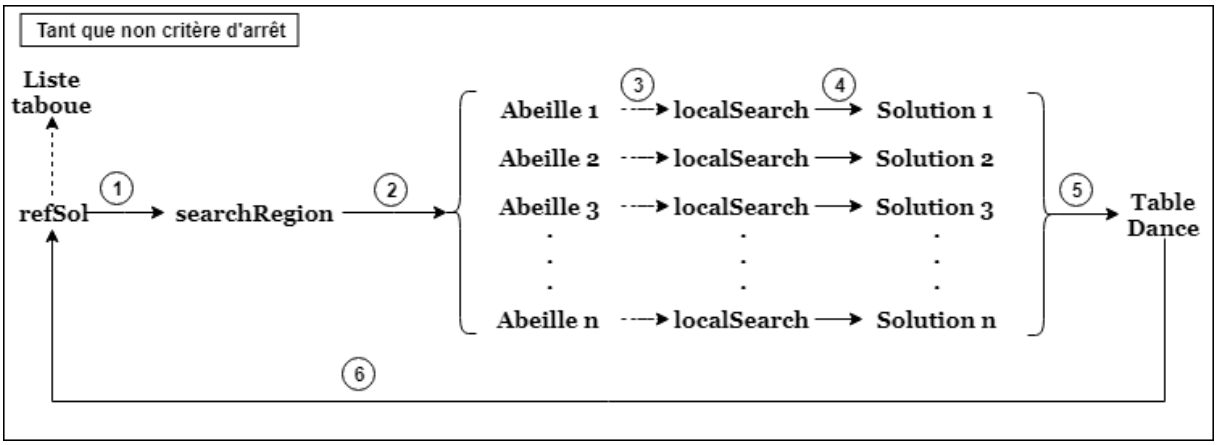


Figure IV-1 – Le processus de recherche de BSO

D'après le processus que nous venons de présenter, nous pouvons considérer l'intégration de l'A/R à BSO à 2 niveaux : la recherche locale effectuée par l'abeille et les décisions prises par l'essaim.

En effet, la métaheuristique BSO est basée sur un essaim d'abeilles qui effectuent individuellement une recherche locale mais certaines décisions sont prises de manière collective, comme le choix de la prochaine solution de référence, la poursuite de l'exploitation de la zone de recherche ou son abandon pour une autre … etc.

De ce fait, nous proposons d'explorer ces deux possibilités en intégrant l'A/R à ces deux niveaux. Concernant l'abeille, celle-ci peut être assimilée à un agent d'A/R. De ce fait, la recherche locale est remplacée par l'algorithme Q-Learning. En ce qui concerne l'essaim, l'apprentissage par renforcement pourrait être utilisé de manière directe dans la sélection de de la solution de référence.

2.2. Intégration de l'A/R au niveau de l'abeille

La première approche d'hybridation de BSO avec l'A/R consiste à intégrer cette technique au niveau de l'abeille en considérant celle-ci comme étant un agent d'A/R. De ce fait, les abeilles sont dotées d'une connaissance supplémentaire qui est la *Q-Table* qu'elles utiliseront dans le choix de leurs actions au lieu d'effectuer une simple recherche locale.

Deux algorithmes hybrides sont proposés :

Pour le premier, chaque abeille possède sa propre *Q-Table*, alors que dans le deuxième, une seule *Q-Table* est partagée par toutes les abeilles de l'essaim.

Ces algorithmes sont respectivement nommés **iQBSO-FS** et **sQBSO-FS** et l'intégration de l'A/R se fait au niveau de la recherche locale effectuée par l'abeille pour les deux cas.

L'intégration de l'algorithme d'A/R, Q-Learning, à BSO au niveau du processus de recherche effectué par l'abeille nécessite la définition des éléments suivants :

- Un état est une solution. Dans le cadre de notre application, c'est un sous ensemble d'attributs. Pour rappel, celle-ci est représentée par un vecteur binaire dans lequel la valeur 1 (respectivement 0) signifie que l'attribut est sélectionné (respectivement non sélectionné).
- Une action est le fait de sélectionner ou supprimer un attribut.
- La récompense associée au choix d'une action *a* à partir d'un état *s* est définie comme suit :

$$r(s,a) = \begin{cases} pne, & pe < pne \\ pne - pe, & pe > pne \\ 1/2\ (pne), & Si\ pe = pne\ ET\ NbAtts(etat) > NbAtts(nouvel_etat) \\ -1/2\ (pne), & Si\ pe = pne\ ET\ NbAtts(etat) < NbAtts(nouvel_etat) \end{cases}$$

Où pne représente l'exactitude dans le nouvel état et pe représente l'exactitude de l'état actuel. Cette formule permet de calculer la récompense attribuée à une action à partir d'un état donné en considérant l'exactitude de la classification comme critère principal et le nombre d'attributs comme critère secondaire :

- Si pne est plus élevée que $pe\ alors$ la récompense est égale à pne, sinon, la récompense est égale à $(pne - pe)$
- En cas d'égalité, le nombre d'attributs est pris en compte avec un poids de ½

En plus de la définition de l'ensemble des états, des actions et de la récompense, il est nécessaire de définir la Q-Table qui dans l'algorithme Q-Learning se présente comme suit :

Q-Table ordinaire

	1	2	...	n-1	n
00000001	q_val	q_val	...	q_val	q_val
00000011	q_val	q_val	...	q_val	q_val
.			.		
.			.		
.			.		
111111110	q_val	q_val	...	q_val	q_val
111111111	q_val	q_val	...	q_val	q_val

Figure IV-2 - Q-Table ordinaire

Dans notre travail, nous avons apporté une modification à cette structure afin d'optimiser le temps d'accès et l'espace occupée en mémoire. En effet, nous avons adopté la structure illustrée dans la Figure IV-3 - Q-Table adaptée :

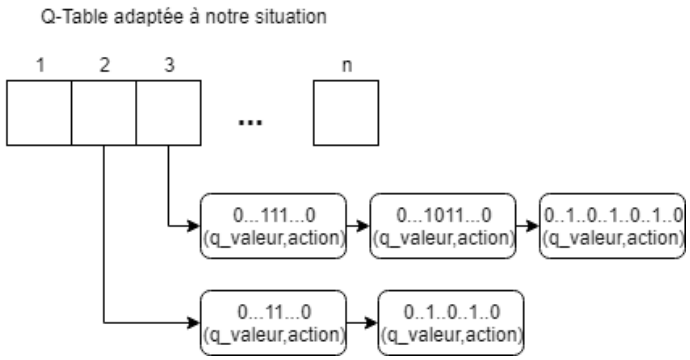


Figure IV-3 - Q-Table adaptée

- Chaque case de la première dimension, qui est un vecteur, représente le nombre d'attributs que possède cette solution, et donc sa taille est le nombre d'attributs maximum de notre instance du problème.
- La deuxième dimension, est une liste qui regroupe les combinaisons de solutions ayant le même nombre d'attributs, et c'est une liste car nous ne connaissons pas le nombre de solutions qu'on aura à visiter préalablement, contrairement à la taille de l'instance, et donc cette liste est construite progressivement, et contient les informations qui caractérisent une solution : la Q-valeur et l'action qui a mené à cet état.

2.2.1. iQBSO-FS

Dans la métaheuristique BSO classique, chaque abeille effectue une recherche locale classique qui consiste à démarrer d'une solution donnée et à évaluer les solutions dans son voisinage immédiat puis retourner la meilleure solution trouvée. Une solution *s* est un sous-ensemble d'attributs et une solution voisine *s'* est un autre sous ensemble contenant un attribut de plus (en inversant une position dans la solution s de la valeur 0 à 1) ou un attribut de moins (en inversant une position dans la solution *s* de la valeur 1 à 0). En sachant que chaque solution visitée est évaluée en mesurant l'exactitude de la classification donnée par le sous ensemble d'attributs qu'elle représente, la recherche locale devient très coûteuse car elle fait appel au classificateur pour l'évaluation de chaque solution. Dans le but d'accélérer le processus de recherche, la recherche locale est remplacée par un algorithme Q-Learning qui permet de guider le processus vers des solutions de bonne qualité en se basant sur la Q-Table individuelle de chaque abeille.

A. QLS_1

Entrée : une solution S de la *searchRegion*
Sortie : La meilleure solution trouvée grâce à la *Q-Table*
solution ← S
Pour *chaque pas j* **faire**
    état ← solution
    nouv_état, *action* ← pas(état)
    acc_état ← évaluer(état)
    acc_nouv_état ← évaluer(nouv_state)
    Si *acc_état < acc_nouv_état* **alors**
        récompense ← acc_nouv_état
    Sinon si *acc_état > acc_nouv_état* **alors**
        récompense ← acc_nouv_état - acc_état
    Sinon
        Si *nbAttribs(état) > nbAttribs (*nouv_état*)* **alors**
            récompense ← 1/2 * acc_nouv_état
        Sinon
            récompense ← -1/2 * acc_nouv_état
    fitness ← q_value(état, action)
    learn(état, action, *récompense*, nouv_état)
    solution ← nouv_état
return *solution*

Explications :

Cette approche étant la première variante dérivée de l'algorithme initiale, le but était initialement d'améliorer le temps d'exécution, tout en gardant une aussi bonne exactitude de classification et nombre d'attributs. Nous appliqué fidèlement l'algorithme du *Q-Learning*.

Exemple :

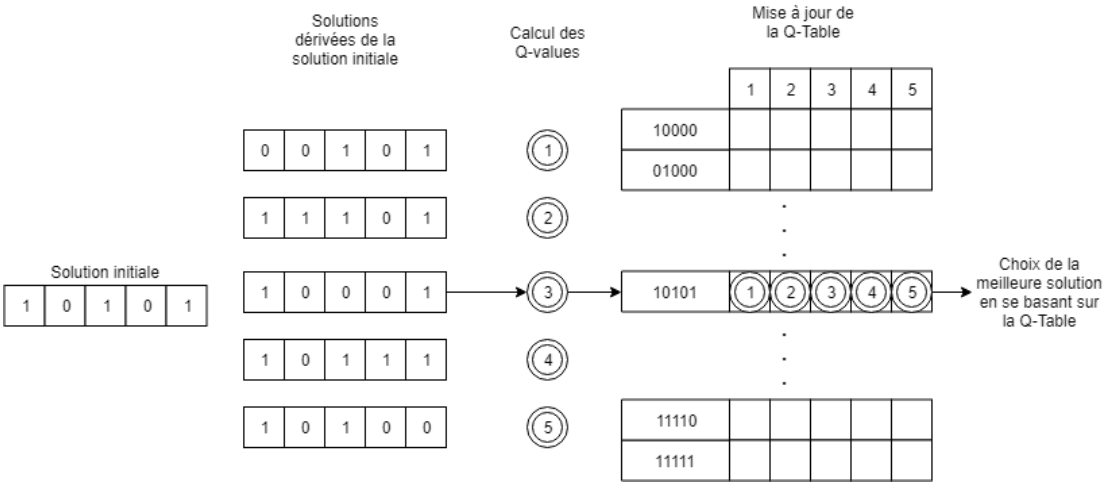


Figure IV-4 - Exemple QLS_1

Pour mieux expliquer le fonctionnement de cette approche nous posons comme hypothèse qu'on a une solution de taille 5 (**10101**)

1. Les prochaines solutions possibles seront donc : **00101**, **11101**, **10001**, **10111**, **10100**
2. Nous calculons les Q-Values de chaque solution qu'on peut visiter
3. Nous mettons à jour la *Q-Table* avec les Q-Values de ces solutions
4. En se basant sur la *Q-table* on décide l'action à choisir, qui maximise le gain à long terme.

Critiques :

Nous avons remarqué que cette approche était aussi bonne que l'algorithme initiale pour les petites et moyennes instances en termes de temps d'exécution et d'exactitude, mais pour les grandes instances, il se trouve que nous visitons plus ($\times$ 1.5 fois) d'états que l'algorithme initiale sans une amélioration considérable de l'exactitude, et avec un coût légèrement plus élevé en termes de temps d'exécution.

B. Le pseudo algorithme de QLS_2

Le but de cet algorithme est de corriger les problèmes rencontrés avec le premier algorithme en réduisant le nombre de solutions à évaluer. Pour ce faire, l'algorithme n'évalue parmi les solutions voisines, que celles qui présentent des similarités avec la meilleure solution trouvée jusqu'à lors.

L'algorithme QLS-2 est présenté ci-dessous :

Entrée : une solution S de la *searchRegion*
Sortie : La meilleure solution trouvée grâce à la *Q-Table*
solution ← S
    Pour *chaque pas j* **faire**
    état ← solution
    espace_actions ← xor_solutions(état,globalBestSol)
    nouve_état, *action* ← pas(état, espace_actions)
    acc_état ← évaluer(état)
    acc_nouv_état ← évaluer (nouv_état)
    Si *acc_état < acc_nouv_état* **alors**
        récompense ← acc_nouv_état
    Sinon si *acc_état > acc_nouv_état* **alors**
        récompense ← acc_nouv_état - acc_état
    Sinon
        Si *nbAttribs(état) > nbAttribs (*nouv_état*)* **alors**
            récompense ← 1/2 * acc_nouv_état
        Sinon *récompense* ← -1/2 * acc_nouv_état
    return ← q_value(état, action)
    fitness ← *acc_nouv_état*
    learn(état, action, *récompense*, nouv_état)

solution ← nouv_état
return *solution*

Explications :

L'un des problèmes majeurs rencontrés avec la première méthode était la taille de l'espace de recherche et donc le but ici est de réduire cet espace d'actions possibles selon une stratégie, où la définition de l'espace d'actions est basée sur le ***XOR*** logique pour trouver les différences entre deux solutions : on aura l'ensemble des attributs présents dans la $globalBestSol$ mais pas dans la solution courante, et depuis cet ensemble, on va choisir notre prochaine action en se basant sur la *Q-Table*.

Pour rappel, la table de vérité d'un XOR (OU exclusif) est la suivante :

Entré		Sortie
A	B	
0	0	0
0	1	1
1	0	1
1	1	0

Tableau IV-1 - Table de vérité : OU exclusif

Exemple :

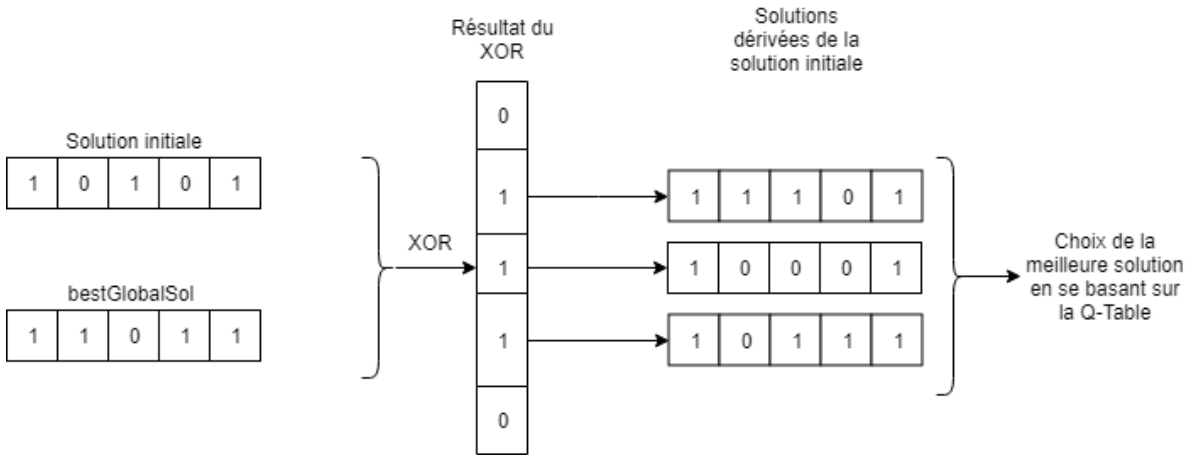


Figure IV-5 - Exemple du XOR

Pour expliquer le fonctionnement de cette approche nous posons comme hypothèse qu'on a une solution de taille 5 (**10101**)

1. Dans le cas normal, les prochaines solutions possibles sont : **00101**, **11101**, **10001**, **10111**, **10100**
2. Mais pour cette approche, supposant que la *bestGlobalSol* est **11011**,
3. En faisant un XOR entre **10101** et **11011**, on aura **01110**, et on fait sortir les index des attributs à changer, qui sont [2,3,4], qui représentent les actions qu'on peut prendre.
4. Donc les états candidats seront : **11101**, **10001** et **10111**, et en se basant sur la *Q-table* on décide l'action à choisir.

Remarques :

Lorsque la taille d'une instance est assez importante, le nombre de différences issue du XOR peut-être relativement grand, et donc nous avons limité la taille de l'échantillon en fonction du paramètre *flip*.

L'algorithme général de **iQBSO-FS** serait :

Entrée : Une instance d'un problème d'optimisation combinatoire
Sortie : La meilleure solution trouvée
$refSol \leftarrow$ une solution initiale trouvée
Tant que critère d'arrêt **faire**
    Insérer $refSol$ dans la $liste\ Taboue$
    Déterminer $searchRegion$ à partir de $refSol$
    $nbChances \leftarrow maxChances$
    Attribuer une solution de $searchRegion$ à chaque abeille
    Pour *chaque abeille k* **faire**
        Effectuer un **QLS_1** ou **QLS_2**
        Stocker le résultat dans la table $Dance$
    Si $f(bestSol) > f(bestGlobalSol)$ **alors**
        $bestGlobalSol \leftarrow bestSol$
        $nbChances \leftarrow maxChances$
        Intensification
    Si non
        Si $nbChances > 0$ **alors**
            $nbChances \leftarrow nbChances - 1$
            Intensification
        Si non
            Diversification
retourner $bestGlobalSol$

2.2.2. sQBSO-FS

Dans la version hybride proposée précédemment, chaque abeille possède une *Q-table* propre à elle, ce qui veut dire que l'apprentissage réalisé par une abeille n'est pas partagé avec ses congénères. Dans cette version, les abeilles partagent leurs expériences à travers une Q-Table commune afin que chacune d'elles bénéficie de l'apprentissage acquis par les autres. Ceci ne modifie pas l'algorithme général de iQBSO-FS et la différence de l'utilisation de la *Q-Table* n'y figure pas.

Pour les algorithmes remplaçant la recherche locale (QLS_1 et QLS_2), ils sont les mêmes que ceux de iQBSO-FS

L'algorithme général d **QBSO-FS** se présente donc comme suit :

Entrée : Une instance d'un problème d'optimisation combinatoire
Sortie : La meilleure solution trouvée
$refSol \leftarrow$ une solution initiale trouvée selon le principe de *Pareto*
Tant que critère d'arrêt **faire**
    Insérer $refSol$ dans la $liste\ Taboue$
    Déterminer $searchRegion$ à partir de $refSol$
    $nbChances \leftarrow maxChances$
    Attribuer une solution de $searchRegion$ à chaque abeille
    Pour *chaque abeille k* **faire**
        Effectuer un **QLS_1** ou **QLS_2**
        Stocker le résultat dans la table $Dance$
    Si $f(bestSol) > f(bestGlobalSol)$ **alors**
        $bestGlobalSol \leftarrow bestSol$
        $nbChances \leftarrow maxChances$
        Intensification
    Si non
        Si $nbChances > 0$ **alors**
            $nbChances \leftarrow nbChances - 1$
            Intensification
        Si non
            Diversification
retourner $bestGlobalSol$

2.3. Intégration de l'A/R au niveau de l'essaim

L'utilisation de l'A/R au niveau de l'essaim a principalement deux objectifs : premièrement, rendre le choix de la solution de référence guidé par un algorithme basé sur le Q-Learning et deuxièmement, supprimer le paramètre max chance de BSO ainsi que la liste taboue ce qui rendra la diversification liée uniquement au paramètre d'exploration de l'A/R.

Dans cette hybridation le même algorithme que BSO classique est exécuté jusqu'à l'étape de la sélection d'une solution de référence, où un algorithme de Q-Learning est utilisé.

La structure de la Q-table de cet algorithme est comme suit :

Chaque solution de référence représente une entrée de la table, une liste des Q-valeurs est associé à chaque solution de référence, la taille de cette liste est égale à la taille de la table *Danse* c'est-à-dire au nombre d'abeilles. Chaque Q-valeur représente la qualité des récompenses futures que l'on peut obtenir si on choisit la solution qui lui est associé comme prochaine solution de référence, la correspondance entre Q-valeurs et solutions stockés dans la table *Danse* se fait selon l'indice de la solution dans cette table (la Q-valeurs qui a comme indice 3 dans la liste des Q-Valeurs est associé à la solution retournée par l'abeille numéro 3)

Le schéma suivant représente une entrée de la Q-Table :

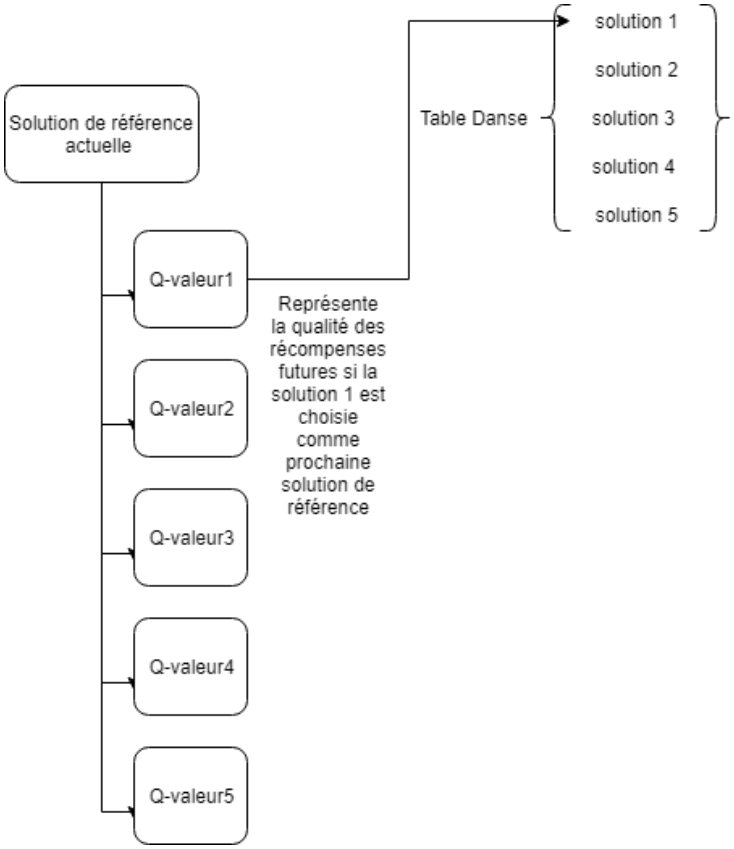


Figure IV-6 - Représentation d'une entrée de la Q-Table de QsBSO-FS

Dans cette approche un état est défini comme étant une solution de référence lors d'une itération donnée. Une action est le choix d'une nouvelle solution de référence parmi les solutions retournée par les abeilles. La récompense est définie de la même manière que dans l'hybridation au niveau de l'abeille. Les paramètres de cette méthode sont le nombre d'itération maximales de BSO, le nombre d'abeilles et les paramètres classique de l'A/R à savoir alpha, gamma et epsilon.

L'algorithme de ***sélection de la nouvelle solution de référence*** est comme suit :

Entrée : la solution de référence actuelle (l'état actuel)
Sortie : L'action à prendre (la sélection de la nouvelle solution de référence)
Générer un nombre aléatoire ***nbr*** entre 0 et 1
Si $nbr > epsilon$ **alors**
    Choisir l'action_suivante avec la plus grande Q-valeur
    Si plusieurs Q-valeurs sont égales **alors**
        Choisir une d'entre elles aléatoirement
    Sinon choisir une action aléatoirement
Choisir état_suivant selon l'action sélectionnée
Si $f(état_suivant) > f(bestGlobalSol)$ **alors**
    $Récompense \leftarrow f(état_suivant)$
Sinon Si $f(état_suivant) < f(bestGlobalSol)$
    $Récompense \leftarrow f(état_suivant) - f(bestGlobalSol)$
    Sinon
    Si $nbr_d'attribut(état_suivant) < nbr_d'attribut(bestGlobalSol)$
    $Récompense \leftarrow \frac{1}{2} * f(état_suivant)$
    Sinon
        $Récompense \leftarrow -\frac{1}{2} * f(état_suivant)$
$Q(état_actuel, action_actuel)$
    $\leftarrow Récompense + gamma * Q(état_suivant, action_suivante)$
    $+ (1 - alpha) * Q(état_actuel, action_actuel)$
retourner $état_suivant$

L'algorithme **QsBSO-FS** devient ainsi :

Entrée : Une instance d'un problème d'optimisation combinatoire
Sortie : La meilleure solution trouvée
$refSol \leftarrow$ une solution initiale trouvée selon le principe de *Pareto*
Tant que max $_iteration$ non atteint
    Déterminer $searchRegion$ à partir de $refSol$
    Attribuer une solution de $searchRegion$ à chaque abeille
    Pour *chaque abeille k* **faire**
        Effectuer une recherche locale ordinaire
        Stocker le résultat dans la table $Dance$
    $refSol \leftarrow selectRefSolRl(refSol)$
    Si $f(refSol) > f(bestGlobalSol)$ **alors**
        $bestGlobalSol \leftarrow refSol$
retourner $bestGlobalSol$

2.4. Améliorations communs à toutes les hybridations

Pour la *refSol*, nous avons changé la fonction de l'initialisation de façon à prendre au début 20% des attributs comme solution, en se basant sur le principe probabiliste de *Pareto*, connu aussi sous le nom de **loi des 80-20** : un phénomène empirique constaté dans certains domaines qui dit qu'environ 80% des effets sont le produit de 20% des causes. Ensuite, la $searchRegion$ va procéder de la même façon que le processus initial.
La figure suivante représente la méthode aléatoire utilisée initialement :

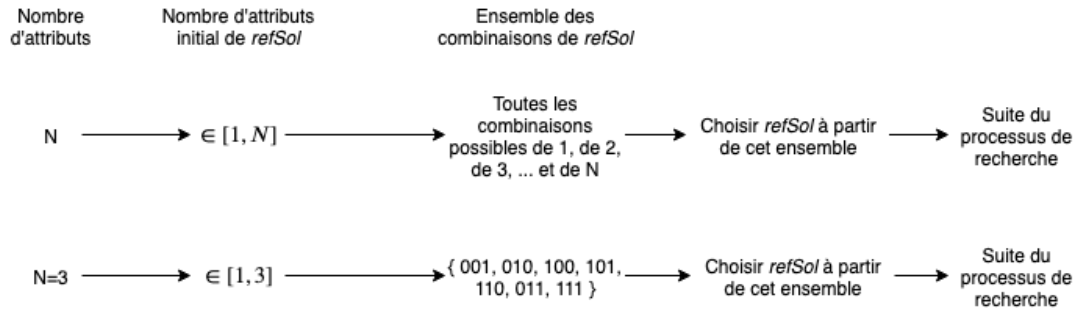


Figure IV-7 – L'ancienne initialisation de refSol

Alors que la nouvelle approche consiste à initialiser *refSol* selon une stratégie, et qui va réduire l'ensemble de combinaisons possibles. Le schéma suivant présente la différence entre l'ancienne et la nouvelle approche d'initialiser *refSol* :

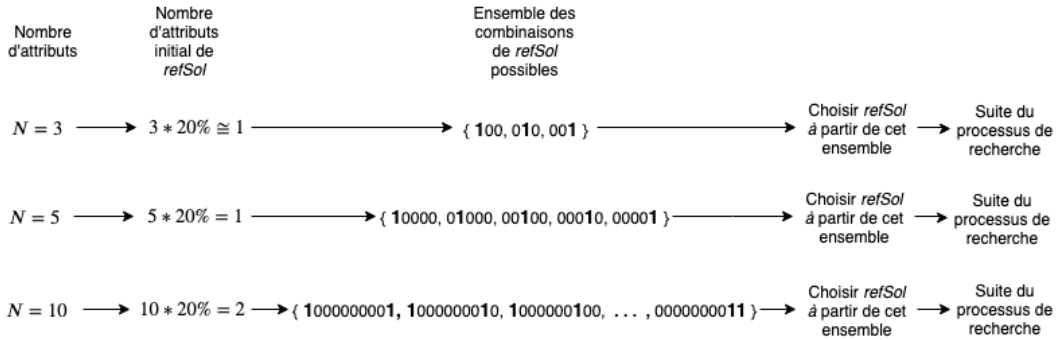


Figure IV-8 - La nouvelle approche pour initialiser refSol

L'utilisation de cette approche pour l'initialisation au niveau des trois approches a permis d'améliorer la qualité des solutions trouvées en termes de nombre d'attributs (Une aussi bonne exactitude de classification avec moins d'attributs).

3. Réalisation

Dans cette partie nous allons présenter les différents outils logiciels avec lesquels nous avons effectué notre travail, ainsi que l'architecture générale de notre système. L'implémentation de la solution a été réalisée dès la finalisation de la partie conception. Au départ, nous avions assuré d'avoir une *base de code* stable pour pouvoir rajouter le reste des classes et fonctions nécessaires sans problème d'extensibilité. Ensuite, nous avons adopté le git pour pouvoir gérer le versionning du projet et générer les notebooks Jupyter, et nous avons hébergé notre code sur la plateforme GitHub.

Vous trouverez notre planning prévisionnel du projet dans la partie A de l'annexe, sous le nom de « Gestion de projet ».

3.1. Outils de développement

3.1.1. Le langage Python



Python est un langage de programmation interprété, de haut niveau et polyvalent. Créé par Guido van Rossum et publié pour la première fois en 1991, Python repose sur une philosophie de conception qui met l'accent sur la lisibilité du code, notamment en utilisant des espaces significatifs. Il propose des concepts permettant une programmation claire, à petite et grande échelle.

Pourquoi Python pour ce type de projets ?

Les projets IA diffèrent des projets logiciels traditionnels, ces différences résident dans la pile de technologies utilisées, les compétences requises pour un tel projet et la nécessité de recherches approfondies. Pour mettre en œuvre ses aspirations, on devrait utiliser un langage de programmation stable, flexible et doté d'outils. Python offre tout cela, c'est pourquoi nous voyons beaucoup de projets Python pour l'IA et le ML aujourd'hui.

Python est bien adapté au ML. Tout d'abord, c'est très simple : si vous n'êtes pas familier avec Python mais avez une autre expérience en programmation (C ou autres langages de programmation), débuter serait vraiment rapide.

Deuxièmement, la communauté de Python est forte : Cela rend la documentation Python non seulement riche, mais également facile à lire. Vous pouvez également trouver des réponses détaillées à de nombreuses questions sur StackOverflow.

3.1.2. Jupyter notebooks

L'une des questions les plus fréquemment posées par les utilisateurs est de savoir quel IDE / environnement / outil utiliser, lorsqu'ils travaillent sur des projets de data science. Et comme les options disponibles ne manquent pas (des IDE spécifiques comme RStudio, PyCharm aux éditeurs comme SublimeText ou Atom) le choix peut-être intimidant pour un débutant.
S'il existe un outil que chaque informaticien devrait utiliser ou avec qui il devrait être à l'aise, il s'agit des notebooks Jupyter (précédemment appelés notebooks IPython).



Jupyter Notebooks sont un format de document open-source basé sur JSON et supportent une variété de langages de programmations populaires parmi les *Data Scientists*, notamment Python et R. Ils contiennent un enregistrement complet des sessions de l'utilisateur et incluent du code, du texte narratif, des équations et une sortie très riche et diverse de graphiques, tout dans un même environnement. Ces notebooks permettent aux scientifiques de rationaliser leur travail et permettent plus de productivité et une collaboration facile, et surtout d'encourager une tâche critique que souvent les programmeurs considèrent comme ennuyeuse : *commenter son code*. Et lorsque vous ouvrez un nouveau notebook, vous devriez voir l'extension .*ipynb* qui signifie : **IPY**thon **N**ote**B**ook
Les notenooks Jupyter sont très efficaces lorsque vous êtes encore en phase de prototypage. En effet, votre code est écrit dans des cellules indépendantes, qui sont exécutées individuellement. Cela permet à l'utilisateur de tester un bloc de code spécifique sans avoir à exécuter le code à partir du début. [68]
Pour notre projet, nous avons utilisé des notebooks Jupyter dérivés de notre *base de code*.

3.1.3. Git, VCS & GitHub



Git est un logiciel de gestion de versions décentralisé. C'est un logiciel libre créé par Linus Torvalds, auteur du noyau Linux, et distribué selon les termes de la licence publique générale GNU version 2. En 2016, il s'agit du logiciel de gestion de versions le plus populaire qui est utilisé par plus de douze millions de personnes.

Un **logiciel de gestion de versions** (ou **VCS** en anglais, pour **V**ersion **C**ontrol **S**ystem) est un logiciel qui permet de stocker un ensemble de fichiers en conservant la chronologie de toutes les modifications qui ont été effectuées dessus. Il permet notamment de retrouver les différentes versions d'un lot de fichiers connexes. Les logiciels de gestion de versions sont utilisés

notamment en développement logiciel pour conserver le code source relatif aux différentes versions d'un logiciel.

GitHub est un service web d'hébergement et de gestion de développement de logiciels, utilisant le logiciel de gestion de versions Git. Le site assure un contrôle d'accès et des fonctionnalités destinées à la collaboration comme le suivi des bugs, les demandes de fonctionnalités, la gestion de tâches et un wiki pour chaque projet. En avril 2016, GitHub a annoncé avoir dépassé les 14 millions d'utilisateurs et plus de 35 millions de dépôts de projets le plaçant comme le plus grand hébergeur de code source au monde.
Le nom **GitHub** est composé du mot «*git*» faisant référence au système de contrôle de version open-source et le mot «hub» faisant référence au réseau social bâti autour du système git, mais aussi à une plate-forme de correspondance qui est appelée en anglais un «hub».
Pour notre projet nous avons utilisé la plateforme GitHub pour héberger notre code source.

3.1.4. Bibliothèques Python utilisées

NumPy : NumPy est une extension du langage de programmation Python, destinée à manipuler des matrices ou tableaux multidimensionnels ainsi que des fonctions mathématiques opérant sur ces tableaux.
Plus précisément, cette bibliothèque logicielle libre et open source fournit de multiples fonctions permettant notamment de créer directement un tableau depuis un fichier ou au contraire de sauvegarder un tableau dans un fichier, et manipuler des vecteurs, matrices et polynômes. NumPy est la base de SciPy, regroupement de bibliothèques Python autour du calcul scientifique.

Pandas : Pandas est une bibliothèque écrite pour le langage de programmation Python permettant la manipulation et l'analyse des données. Elle propose en particulier des structures de données et des opérations de manipulation de tableaux numériques et de séries temporelles. Pandas est un logiciel libre sous licence BSD.
Les principales structures de données sont les *Series* (pour stocker des données selon une dimension - grandeur en fonction d'un index), les *DataFrames* (pour stocker des données selon 2 dimensions - lignes et colonnes) et les *Panels* (pour représenter des données selon 3 dimensions et plus)

Scikit-learn : Scikit-learn est une bibliothèque libre Python destinée à l'apprentissage automatique. Elle est développée par de nombreux contributeurs notamment dans le monde académique, et comprend notamment des fonctions pour estimer des forêts aléatoires, des régressions logistiques, des algorithmes de classification, et les machines à vecteurs de support (SVM). Elle est conçue pour s'harmoniser avec d'autres bibliothèques libres Python, notamment NumPy et SciPy.



Matplotlib : Matplotlib est une bibliothèque du langage de programmation Python destinée à tracer et visualiser des données sous formes de graphiques. Elle peut être combinée avec les bibliothèques python de calcul scientifique NumPy et SciPy. Plusieurs points rendent cette bibliothèque intéressante :

- Export possible en de nombreux formats matriciels (PNG, JPEG...) et vectoriels (PDF, SVG...)
- Documentation en ligne en quantité, nombreux exemples disponibles sur internet
- Forte communauté très active
- Interface pylab : reproduit fidèlement la syntaxe MATLAB
- Bibliothèque haut niveau : idéale pour le calcul interactif

3.2. Architecture et implémentation de la solution

Pour ce projet, nous avons utilisé les bonnes pratiques de la conception et la programmation orientée objet pour fournir un code extensible et indépendant de l'environnement d'exécution sous forme de notebooks Jupyter (qu'on a défini dans la section précédente).

3.2.1. Processus générale du fonctionnement de notre système

Ce processus présente 3 grandes étapes importantes qui sont :

1. Génération des fichiers notebooks Jupyters
2. Exécution des notebooks Jupyter sur la plateforme Colab
3. Manipulation des fichiers de résultats, notamment, extraire des données représentatives pour chaque cas, visualisation de ces données, génération de fichiers de profiling.

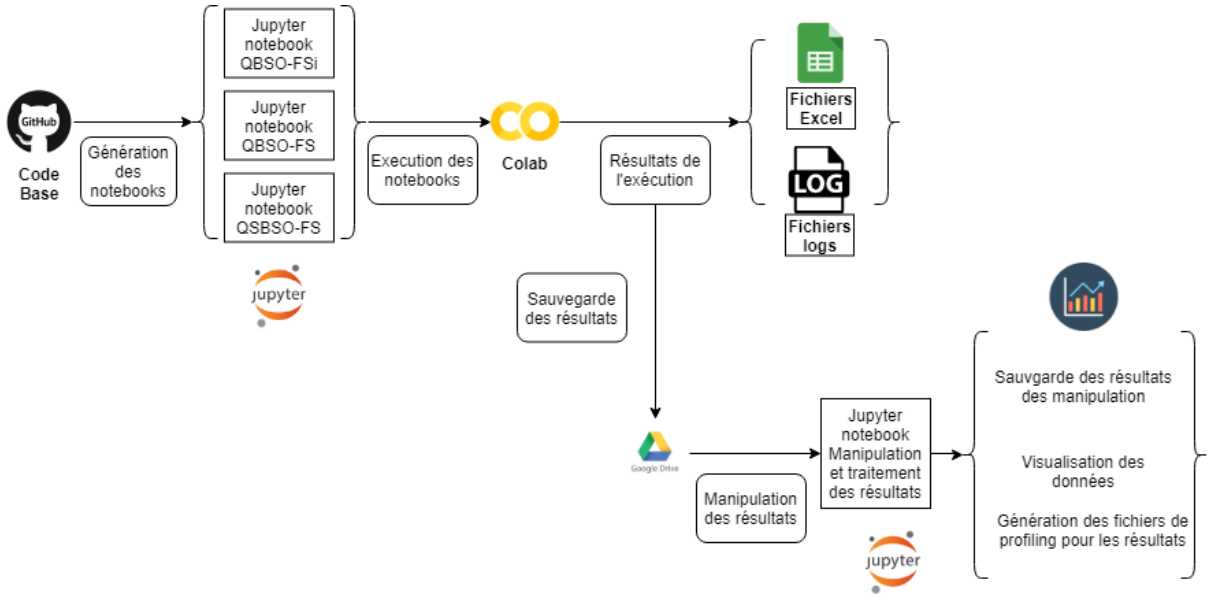


Figure IV-9 - Architecture générale du système

3.2.2. Diagramme de classes

Cette partie est dédiée à l'architecture du code utilisé pour implémenter notre solution.

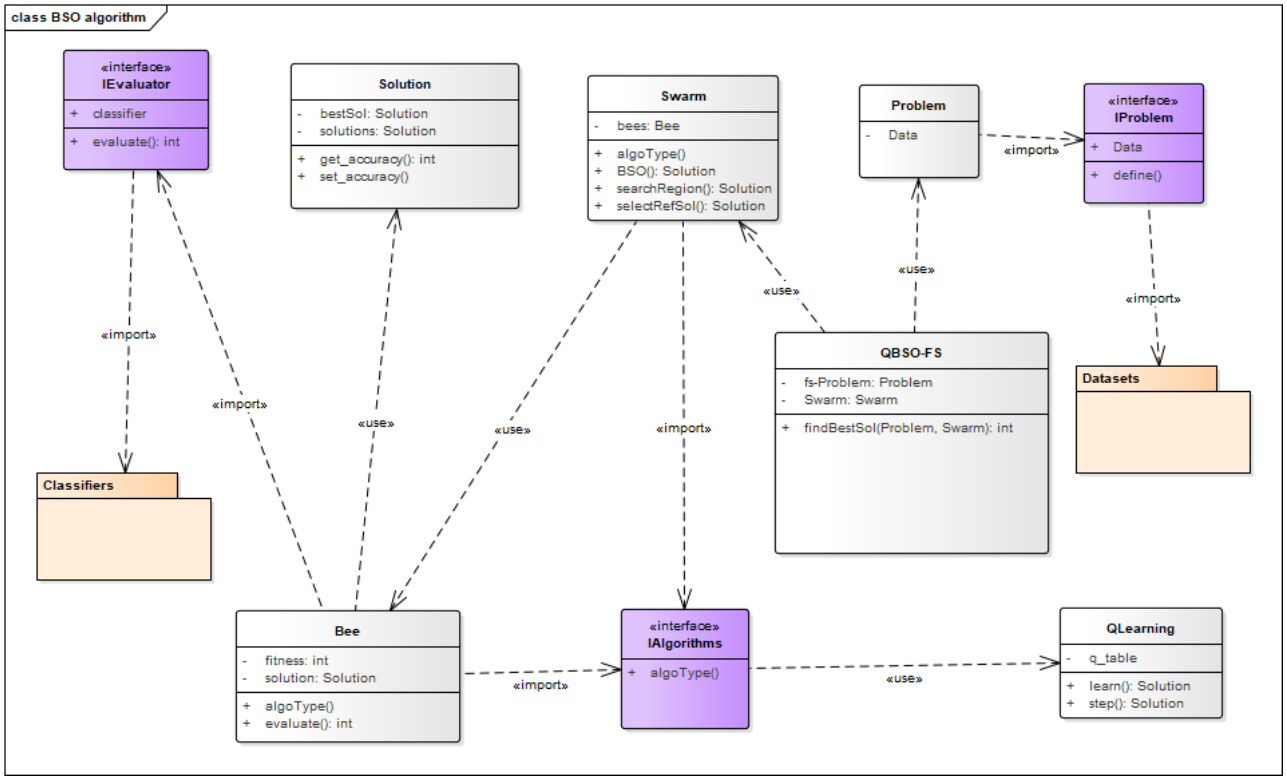


Figure IV-10 - Diagramme de classe

3.2.3. Description des classes

Ce diagramme de classe a été fait, en essayant au maximum de respecter les bonnes pratiques de développement qu'on a vu durant notre cursus, notamment d'avoir des classes les plus indépendantes possibles, autrement dit, flexibles, pour permettre une extension pour d'autres classes de problèmes. Le tableau suivant contient la description des classes présentés précédemment :

Classe	Description
Bee	La classe qui s'occupera d'instancier les abeilles et faire la recherche locale
IEvaluator	L'interface qui contiendra les fonctions liées à l'évaluation d'une solution à un problème définit
Classifiers	Un package contenant des fonctions prédéfinies pour les classfieurs comme KNN ou SVM
Solution	Cette classe servira à définir les fonctions de base que doit avoir une solution
Swarm	Cette classe s'occupera de l'aspect BSO, et contiendra l'algorithme BSO en tant que fonction. Elle utilise les objets de la classe Bee comme et une fonction de recherche de région va affecter à chacune une solution.
IAlgorithms	Cette interface sert à garder le choix d'algorithmes à implémenter extensible
QLearning	Cette classe implémente l'algorithme du Q-Learning
Problem	Cette classe définit le problème traité, dans notre cas, la sélection d'attributs
IProblem	Cette interface sert à définir les fonctions de base relatives aux divers types de problèmes
Datasets	Ce package contient les instances (datasets) liés au problème
sQBSO-FS	Cette classe est la classe principale de l'algorithme qui implémente le reste des classes et résout le problème définit.
Problem	C'est une classe qui implémente l'interface IProblem et contient les éléments de base d'un problème d'optimisation
Solution	Cette classe est responsable d'instancier les éléments de base d'une solution d'un problème résolue par une abeille.

Tableau IV-2 - Description des classes

4. Conclusion

Ce chapitre a été dédié à la présentation de la conception de nouvelles métaheuristiques hybridées avec l'A/R (**iQBSO-FS**, **sQBSO-FS** et **QsBSO-FS**) et basées sur la métaheuristique BSO pour la résolution du problème de sélection d'attributs. Les algorithmes étant définis théoriquement, il était nécessaire de les implémenter dans un environnement de simulation dans le but de les appliquer à des problèmes de la littérature. Nous avons présenté aussi dans la deuxième section du chapitre, la partie implémentée de notre solution pour atteindre les objectifs définis préalablement. Le chapitre suivant, qui sera le dernier de ce mémoire, contiendra les tests et l'interprétation des résultats obtenus pour chacune de ces métaheuristiques proposées.

V. Chapitre 5 : Tests et résultats

1. Introduction

Nous présentons dans ce dernier chapitre les résultats expérimentaux obtenus après une série de tests dont le but est d'évaluer la performance de chaque algorithme que nous avons proposé et implémenté en termes de qualité et de temps d'exécution sur un ensemble de datasets diversifiés en termes de nombre d'attributs. Les performances de nos méthodes sont comparées à d'autres méthodes qui existent dans la littérature, notamment, à BSO-FS sur laquelle elles sont basées et que nous souhaitons améliorer. Les tests sont effectués sur la plateforme Colaboratory de Google qui donne accès à des machines très performantes, gratuitement.

Ce chapitre est divisé en 4 sections : dans la première, une présentation de l'environnement de tests est donnée, la deuxième présentera le plan des tests, la troisième sera réservée à la présentation des résultats de l'exécution du plan de tests, la quatrième et dernière section contiendra les interprétations et conclusions tirés des résultats.

2. Présentation de l'environnement de tests

2.1. Les datasets utilisés

Pour tester les méthodes **iQBSO-FS**, **sQBSO-FS** et **QsBSO-FS**, et particulièrement, pour la partie de l'étude empirique, nous avons utilisé 10 datasets des plus récurrents dans la littérature. Nous avons choisi les datasets selon leurs diversités dans les trois critères : Le nombre d'attributs, le nombre d'instances, le nombre de classes.

Nous avons considéré trois catégories de datasets selon leur nombre d'attributs : Les datasets de petites tailles (de 19 attributs), les datasets de moyennes tailles (entre 20 et 40 attributs), les datasets de grandes tailles (plus de 50 attributs).

Nous avons utilisé le classifieur KNN, avec $k = 1$, et la méthode de validation croisée *k-fold* pour $k = 10$. Plus de détails sont dans l'annexe B.

Les différents datasets sont issu du UCI Machine Learning Repository qui est une plateforme hébergeant les datasets les plus utilisé dans les travaux sur l'apprentissage automatique.

Le tableau suivant décrit les datasets utilisés :

Dataset	Nombre d'attributs	Nombre d'instances	Nombre de classes
Glass	9	214	7
Breastcancer	10	699	2
Wine	13	178	3
Zoo	16	101	7
Hepatitis	19	155	2
Spect	22	267	2
WDBC	30	569	2
Ionosphere	34	351	2
Sonar	60	208	2
Movementlibras	90	360	15

Tableau V-1 - Datasets pour l'étude empirique

Pour l'étude comparative, nous avons rajouté 10 datasets supplémentaires pour tester sQBSO-FS ainsi que la BSO classique (BSO-FS) et les comparer avec d'autres travaux existants dans la littérature.

Le tableau suivant résume les datasets supplémentaires :

Dataset	Nombre d'attributs	Nombre d'instances	Nombre de classes
Iris	4	150	3
Diabets	8	768	2
Vowel	10	901	15
Heart-C	13	303	5
Heart-Statlog	13	270	2
Congress	16	435	2
Lymphography	18	148	8
Vehicule	18	846	4
German	24	1000	2
Lung-Cancer	56	32	3

Tableau V-2 - Datasets supplémentaires pour l'étude comparative

2.2. Les métriques utilisées

2.2.1. L'exactitude de la classification (Accuracy)

L'exactitude est la proportion de vrais résultats (à la fois vrais positifs et vrais négatifs) parmi le nombre total de cas examinés, nous rappelons la formule définie dans le premier chapitre :

$$Exactitude = \frac{(VP + VN)}{(VP + VN + FP + FN)}$$

Où : **VP** = Vrai positif, **FP** = Faux positif, **VN** = Vrai négatif, **FN** = Faux négatif
Dans notre travail, nous avons utilisé cette métrique comme première métrique, et elle est considéré comme la plus importante.

2.2.2. Le nombre d'attributs d'une solution

Le nombre d'attributs d'une solution représente le sous-ensemble d'attributs (features) que nous avons gardé parmi l'ensemble de tous les attributs d'un dataset (le nombre de positions mises à 1, par rapport à notre représentation vectorielle).

2.2.3. La taille de l'espace des solutions visitées

Cette métrique a été définit après des observations empiriques, puisque nous avons remarqué que le fait d'avoir un large espace de solutions visités ne signifie pas forcément avoir une meilleure précision en sortie. Et donc, nous avons opté pour l'optimisation de ce paramètre et nous l'avons considéré comme métrique pour nos tests.

2.2.4. Temps d'exécution

Le temps d'exécution est une métrique à prendre en considération intuitivement. Cette métrique est fortement corrélée avec la taille de l'espace des solutions visitées, puisque plus l'algorithme visite de nouvelles solutions et plus il met du temps.

3. Présentation du plan de tests

3.1. Présentation des paramètres étudiés

3.1.1. BSO

Les paramètres empiriques de BSO, ont été fixés aux valeurs suivantes : Nombre d'abeilles à 10, *flip* à 5, le nombre de chances accordées (*MaxChances*) à 3 et le nombre d'itérations de la métaheuristique et de la recherche locale (*MaxIterations* et *LocIterations*) à 10. Le choix des valeurs par défaut s'est basé sur les résultats des anciens travaux de la métaheuristique BSO et ses variantes.

1) Nombre d'abeilles *N*

Représente le nombre d'abeilles constituant l'essaim.

2) Flip

La valeur de flip a un impact sur les performances du processus de recherche car elle détermine la distance entre refSol et les solutions qui définissent la *SearchArea*. En effet, une valeur trop petite favorisera l'exploration plutôt que l'exploitation de l'espace de recherche, tandis qu'une valeur trop élevée pourrait amener l'algorithme à converger vers un optimum local.

3) MaxChances

MaxChances est un paramètre qui représente le nombre maximum de chances accordé à *SearchArea*.

4) MaxIterations

Représente le nombre d'itérations de l'algorithme général de la métaheuristique.

5) LocIterations

Représente le nombre d'itérations au niveau de la recherche locale initiale.

3.1.2. Q-Learning

Les paramètres empiriques du Q-Learning ont été fixés aux valeurs suivantes : alpha (α) à 0.1, gamma (γ) à 0.9 et epsilon (ε) à 0.1.

Le choix des valeurs par défaut pour les paramètres de Q-Learning sont basées sur les valeurs trouvées dans la littérature des travaux semblables et représentent des valeurs de bonnes pratiques.

1) Le taux d'apprentissage α

Le taux d'apprentissage contrôle la rapidité avec laquelle nous modifions nos estimations. Souvent, on espère commencer avec un taux d'apprentissage élevé, ce qui permet des changements rapides et diminue le taux d'apprentissage au fil du temps.
Les conditions de base sont que la somme des taux d'apprentissage va à l'infini (pour que toute valeur puisse être atteinte) et que la somme des carrés de ces taux d'apprentissage soit finie (ce qui est nécessaire pour montrer que la convergence est avec une probabilité 1). (Even-Dar & Mansour, 2004)

2) Le facteur de dépréciation γ

Ce facteur sert à régler l'importance accordée aux récompenses futures vis-à-vis des récompenses immédiates, autrement dit, plus une récompense est lointaine dans le futur, moins son poids est important. (Even-Dar & Mansour, 2004)
Si $\gamma = 0$, les récompenses futures sont ignorées et seules les récompenses immédiates comptent (agent cigale), alors que si $\gamma = 1$, les récompenses futures comptent autant que les récompenses immédiates (agent fourmi). Le plus souvent, on choisit $\gamma \in]0,1[$.

3) Le facteur d'exploration ε

Ce paramètre est utilisé dans le cas d'une politique ε-*greedy* ou politique gloutonne. Avec une probabilité de ε, on choisit une action au hasard, alors que pour une probabilité de $(1 - \varepsilon)$, on choisit l'action gloutonne a^*.

3.2. Etude paramétrique

Afin d'ajuster les différents paramètres des métaheuristiques implémentées, nous avons effectué nos expérimentations pour les 10 datasets présentés dans la partie précédente (Glass, Breastcancer, Wine, Zoo, Hepatitis, Spect, WDBC, Ionosphere, Sonar, Movementlibras) avec 10 exécutions pour chacun. Le choix de la valeur 10 comme nombre d'exécutions et les 10 datasets utilisés est fait sur la base de la littérature, pour pouvoir comparer les résultats plus tard dans la partie réservée à l'étude comparative.

Nous avons utilisé l'ANOVA (Analyse de variance) à un facteur pour chaque paramètre afin d'étudier l'effet d'un paramètre sur l'exactitude de classification de nos datasets, et nous considérons l'hypothèse : $H_0 : \mu_1 = \ldots = \mu_J$, qui revient à dire que la moyenne est indépendante du niveau ou encore que le paramètre étudié n'ait pas d'effet, contre l'hypothèse $H_1 : \exists(j,k)\ tel\ que\ \mu_j \neq \mu_k$.

Pour plus de détails sur l'ANOVA, veuillez voir l'annexe B : Analyse de variance.

Notre plan de test pour l'étude comparative est présenté par le tableau suivant :

3.2.1. iQBSO-FS

Type de paramètre	Paramètre	Plage de valeurs
BSO	Nombre d'abeilles	5, 10, 15, 20
	Flip	2, 3, 4, 5, 10
	MaxChances	2, 3, 4, 5, 10
Q-Learning	α	0.1, 0.25, 0.5, 0.75, 1
	γ	0.8, 0.85, 0.9, 0.95, 0.99
	ε	0.01, 0.05, 0.1, 0.2, 0.5, 0.8

Tableau V-3 - Plan de tests pour iQBSO-FS

3.2.2. sQBSO-FS

Type de paramètre	Paramètre	Plage de valeurs
BSO	Nombre d'abeilles	5, 10, 15, 20
	Flip	2, 3, 4, 5, 10
	MaxChances	2, 3, 4, 5, 10
Q-Learning	α	0.1, 0.25, 0.5, 0.75, 1
	γ	0.8, 0.85, 0.9, 0.95, 0.99
	ε	0.01, 0.05, 0.1, 0.2, 0.5, 0.8

Tableau V-4 - Plan de tests pour sQBSO-FS

3.2.3. QsBSO-FS

Type de paramètre	Paramètre	Plage de valeurs
BSO	Nombre d'abeilles	$5, 10, 15$
	MaxIterations	$10, 15, 20$
Q-Learning	α	$0.1, 0.2$
	γ	$0.8, 0.9$
	ε	$0.05, 0.1$

Tableau V-5 - Plan de tests pour QsBSO-FS

3.3. Etude comparative

Dans cette partie, nous allons comparer les performances des métaheuristiques hybrides proposées selon les paramètres que nous avons présentés précédemment, ensuite nous allons choisir la meilleure métaheuristique, qu'on comparera à BSO-FS. Pour cela, nous avons opté pour 30 exécutions au lieu de 10 (souvent la valeur présente dans littérature) et nous avons rajouté 10 datasets supplémentaires pour regrouper deux objectifs : premièrement, que les exécutions servent à comparer entre ces deux méthodes, et deuxièment, pour tester la stabilité de résultats obtenus.

4. Présentation et interprétation des résultats des tests

Dans cette section nous allons présenter les résultats de l'étude paramétriques que nous avons faite ainsi qu'une étude comparative entre les méthodes proposée.

4.1. Etude paramétrique

Pour voir l'effet de chaque paramètre sur les performances des algorithmes, une ANOVA est effectuée. Les résultats sont montrés sous formes de graphiques ainsi que de tableaux d'ANOVA. Les meilleures valeurs des paramètres sont déduites et utilisées afin de faire une étude comparative entre les approches proposés par la suite.

4.1.1. iQBSO-FS

A. Paramètres de BSO

1) Nombre d'abeilles

Analyse de variance :

Nombre d'abeilles

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
5	10	895.93	89.593	111.309357		
10	10	896.889	89.6889	110.642583		
15	10	898.38	89.838	112.638951		
20	10	897.21	89.721	112.551632		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	0.30639008	3	0.10213003	**0.00091362**	0.99996108	**2.86626555**
A l'intérieur des groupes	4024.28271	36	111.785631			
Total	4024.5891	39				

Tableau V-6 - ANOVA pour l'effet du nombre d'abeilles (iQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du nombre d'abeilles n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

D'après le graphique nous pouvons constater qu'avec 15 abeilles nous avons les meilleurs résultats pour tous les datasets.

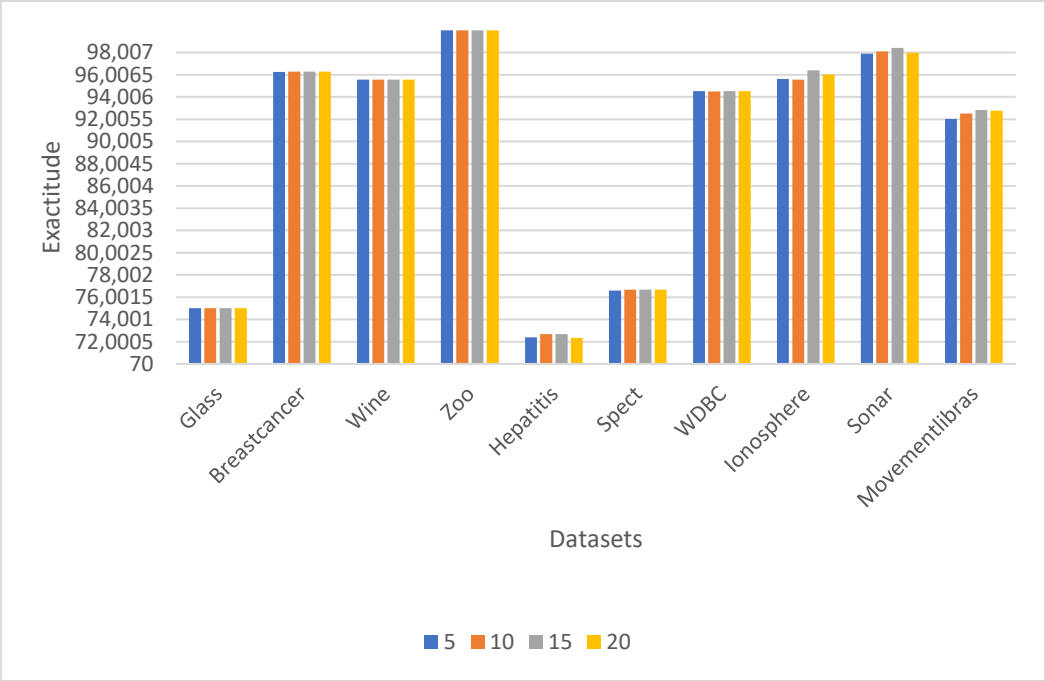


Figure V-1 - Le paramètre nombre d'abeilles pour iQBSO-FS

2) Flip

Analyse de variance :

Flip

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
2	10	895.794	89.5794	109.058234		
3	10	896.944	89.6944	110.448254		
4	10	897.966	89.7966	112.219062		
5	10	896.838	89.6838	110.732577		
10	10	896.818	89.6818	112.46772		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	0.2368176	4	0.0592044	**0.00053344**	0.99999941	**2.57873918**
A l'intérieur des groupes	4994.33263	45	110.98517			
Total	4994.56945	49				

Tableau V-7 - ANOVA pour l'effet de flip (iQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation de flip n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est

considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

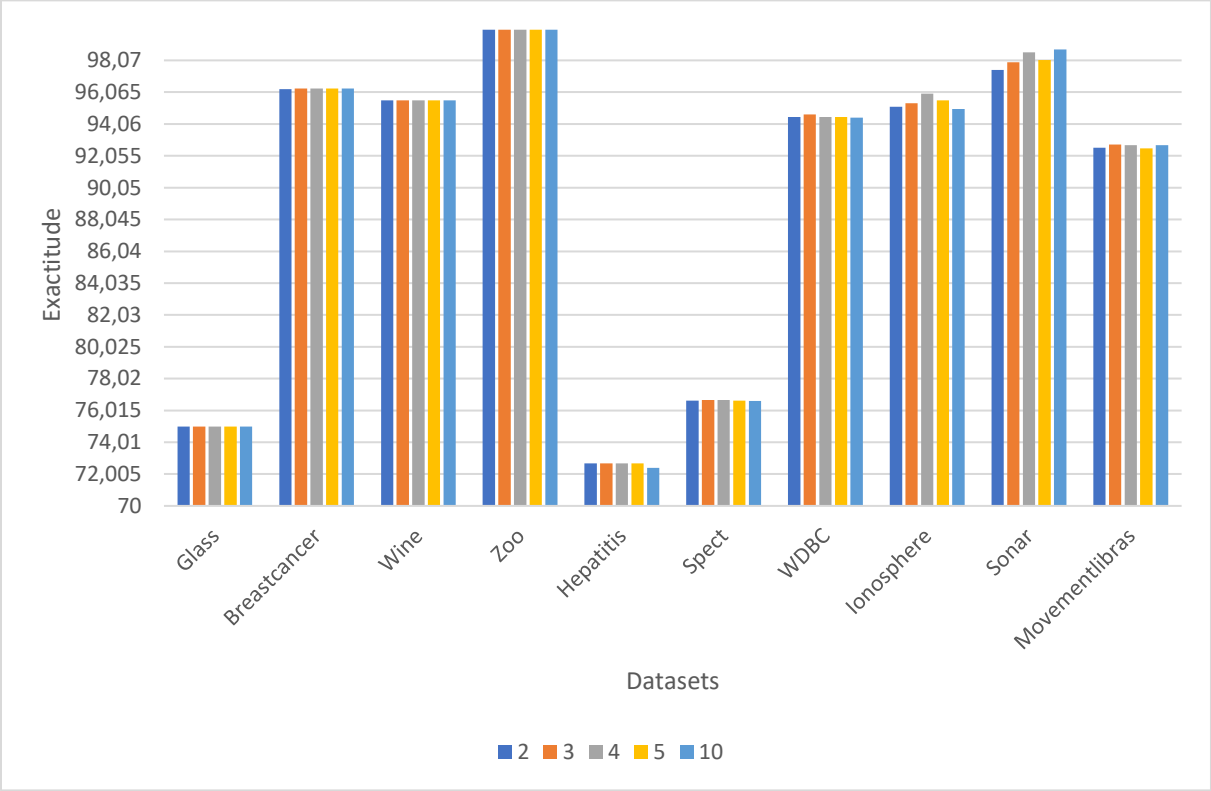


Figure V-2 - Le paramètre flip pour iQBSO-FS

D'après le graphique nous pouvons constater qu'avec une valeur de flip égale à 4 nous avons la meilleure exactitude pour 9 des 10 datasets.

3) MaxChances

Analyse de variance :

MaxChances

RAPPORT DÉTAILLÉ				
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*
2	10	896.905	89.6905	110.866116
3	10	897.041	89.7041	110.805691
4	10	897.181	89.7181	111.148102

5	10	898.065	89.8065	112.615812		
10	10	896.619	89.6619	112.062319		
			ANALYSE DE VARIANCE			
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	0.11913088	4	0.02978272	**0.00026711**	0.99999985	**2.57873918**
A l'intérieur des groupes	5017.48236	45	111.499608			
Total	5017.60149	49				

Tableau V-8 - ANOVA pour l'effet de MaxChances (iQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation de MaxChances n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

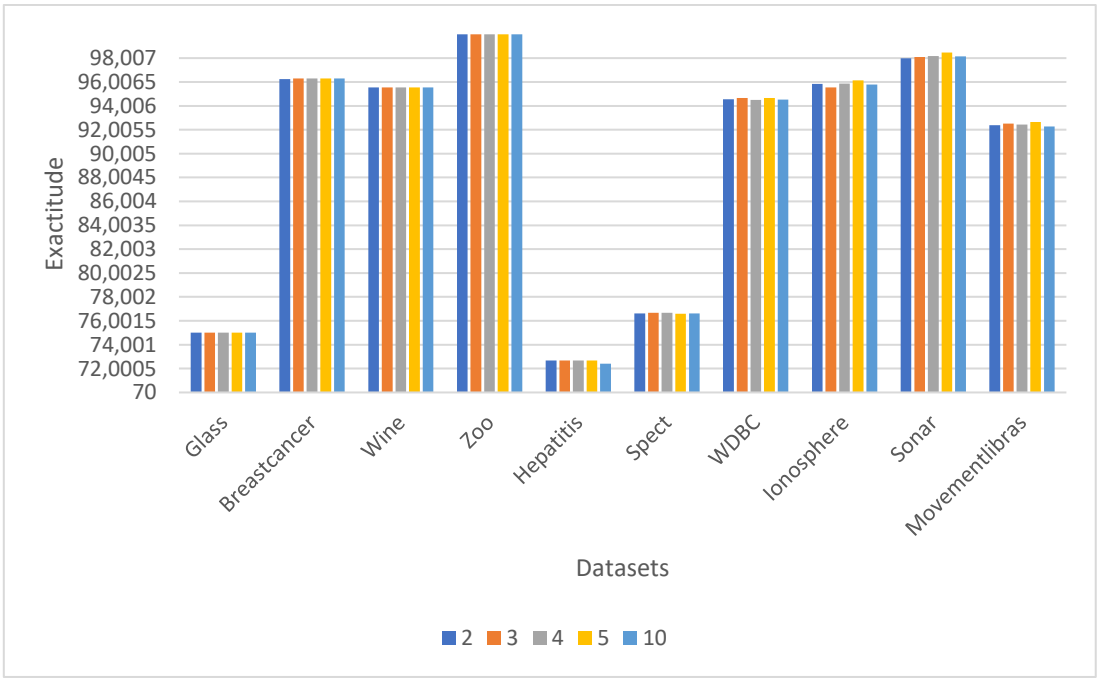


Figure V-3 - Le paramètre MaxChances de iQBSO-FS

Pour le paramètre de diversification max chance nous pouvons voir que la valeur 5 est la plus adéquate vu qu'elle donne les meilleurs résultats sur 9 datasets.

B. A/R

1) Le taux d'apprentissage (α)

Analyse de variance :

Le taux d'apprentissage (α)

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
0.1	10	896.889	89.6889	110.642583		
0.25	10	896.989	89.6989	110.978418		
0.5	10	896.63	89.663	111.394668		
0.75	10	896.992	89.6992	111.136443		
1	10	896.48	89.648	110.311158		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	0.0211726	4	0.00529315	**4.7732 e-05**	1	**2.57873918**
A l'intérieur des groupes	4990.16944	45	110.892654			
Total	4990.19061	49				

Tableau V-9 - ANOVA pour l'effet du taux d'apprentissage (iQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du taux d'apprentissage n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

La valeur 0.1 nous permet d'avoir la meilleure exactitude sur 9 des 10 datasets et donc elle sera choisie comme valeur du paramètre alpha lors de la comparaison des différentes méthodes.

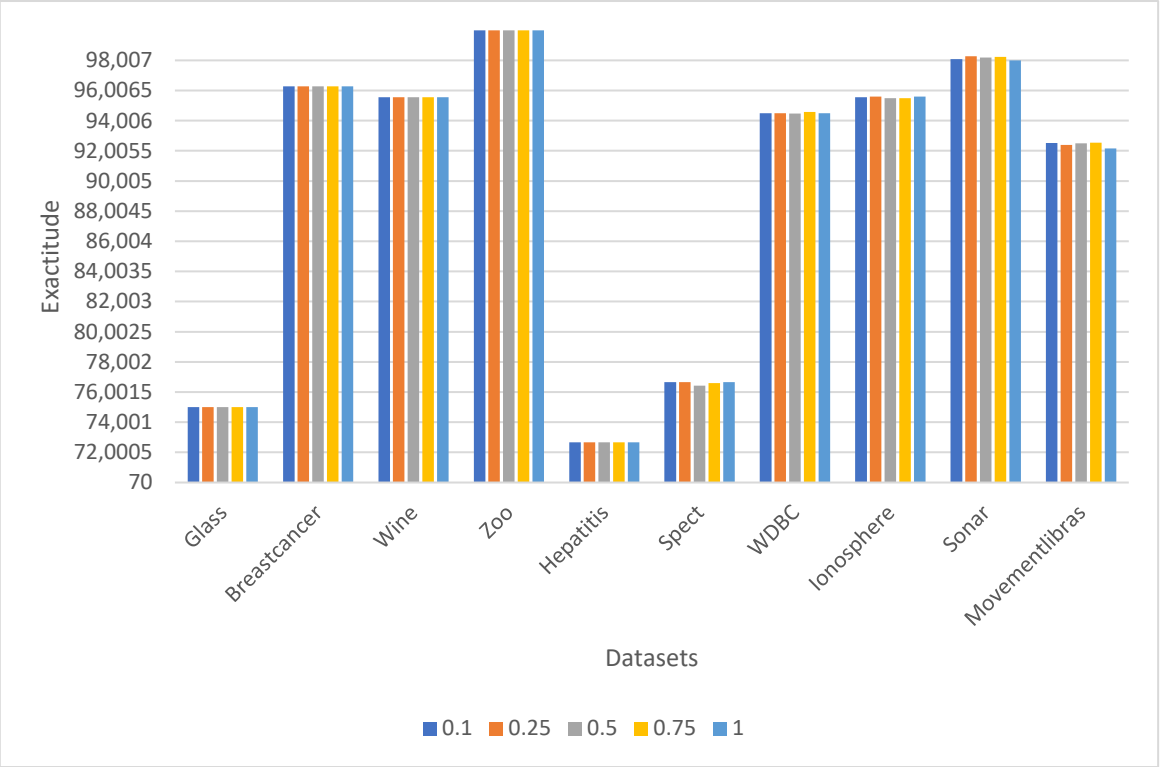


Figure V-4 - Le paramètre taux d'apprentissage pour iQBSO-FS

2) Le facteur de dépréciation (γ)

Analyse de variance :

Le facteur de dépréciation (γ)

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
0.8	10	897.23	89.723	111.314179		
0.85	10	897.534	89.7534	111.78201		
0.9	10	896.889	89.6889	110.642583		
0.95	10	897.012	89.7012	112.889941		
0.99	10	896.115	89.6115	112.646422		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	0.1124066	4	0.02810165	**0.00025123**	0.99999987	**2.57873918**
A l'intérieur des groupes	5033.47622	45	111.855027			
Total	5033.58863	49				

Tableau V-10 - ANOVA pour l'effet du facteur de dépréciation (iQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du facteur de dépréciation n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

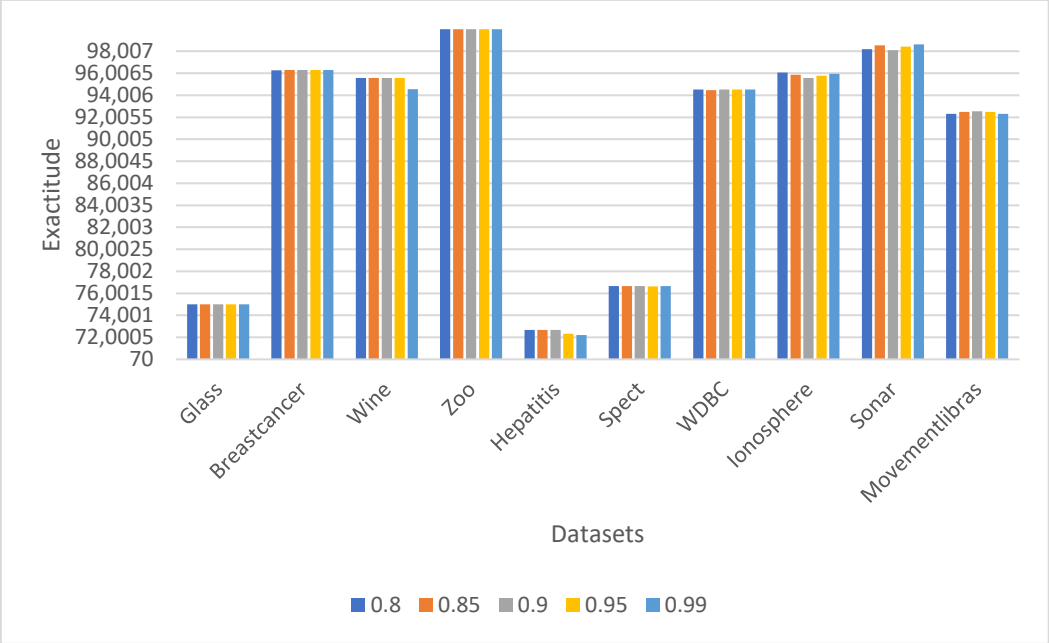


Figure V-5 - Le paramètre facteur de dépréciation pour iQBSO-FS

D'après l'histogramme les valeurs 0.8 et 0.9 permettent d'avoir les meilleurs résultats, chaque valeur a été performante sur 9 datasets (pour 5 datasets les deux valeurs ont donné les mêmes résultats).

3) Le facteur d'exploration (ε)

Analyse de variance :

Le facteur d'exploration (ε)

RAPPORT DÉTAILLÉ				
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*
0.01	10	897.326	89.7326	111.169885
0.05	10	897.228	89.7228	112.094912
0.1	10	897.22	89.722	111.164707

0.2	10	896.07	89.607	111.537646		
0.5	10	893.77	89.377	110.596601		
0.8	10	883.54	88.354	117.819516		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	14.5378107	5	2.90756215	**0.02586863**	0.99967432	**2.38606986**
A l'intérieur des groupes	6069.44939	54	112.397211			
Total	6083.9872	59				

Tableau V-11 - ANOVA pour l'effet du facteur d'exploration (iQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du facteur d'exploration n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

Pour epsilon nous remarquons que la valeur 0.01 permet de donner les meilleures performances sur 9 datasets (même performance pour les datasets Glass, Breastcancer, Wine, Zoo pour toute les valeurs, la valeur 0.01 donne les meilleurs résultats pour 4 des 5 datasets restants).

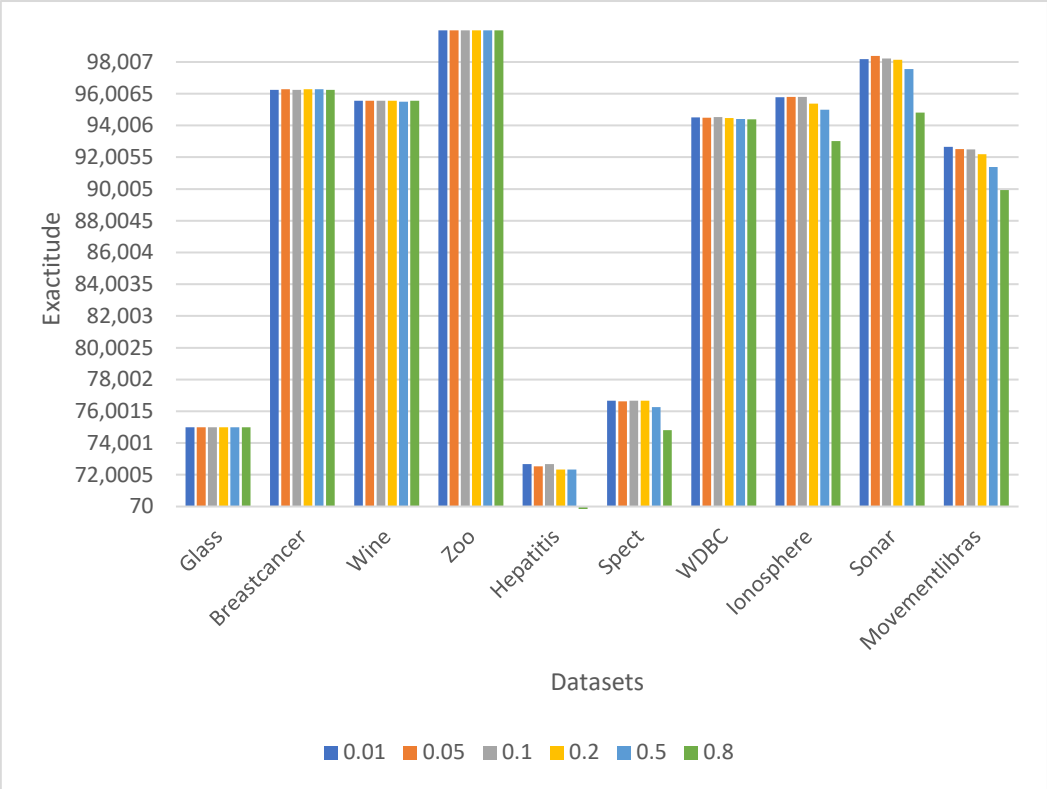


Figure V-6 - Le paramètre taux d'exploration pour iQBSO-FS

4.1.2. sQBSO-FS

A. Paramètres de BSO

1) Nombre d'abeilles

Analyse de variance :

Nombre d'abeilles

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
5	10	895.93	89.593	111.343534		
10	10	897.221	89.7221	111.151138		
15	10	897.807	89.7807	111.755495		
20	10	904.83	90.483	123.817957		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	4.7990654	3	1.59968847	**0.013969**	0.99770265	**2.86626555**
A l'intérieur des groupes	4122.61312	36	114.517031			
Total	4127.41218	39				

Tableau V-12 - ANOVA pour l'effet du nombre d'abeilles (sQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du nombre d'abeilles n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

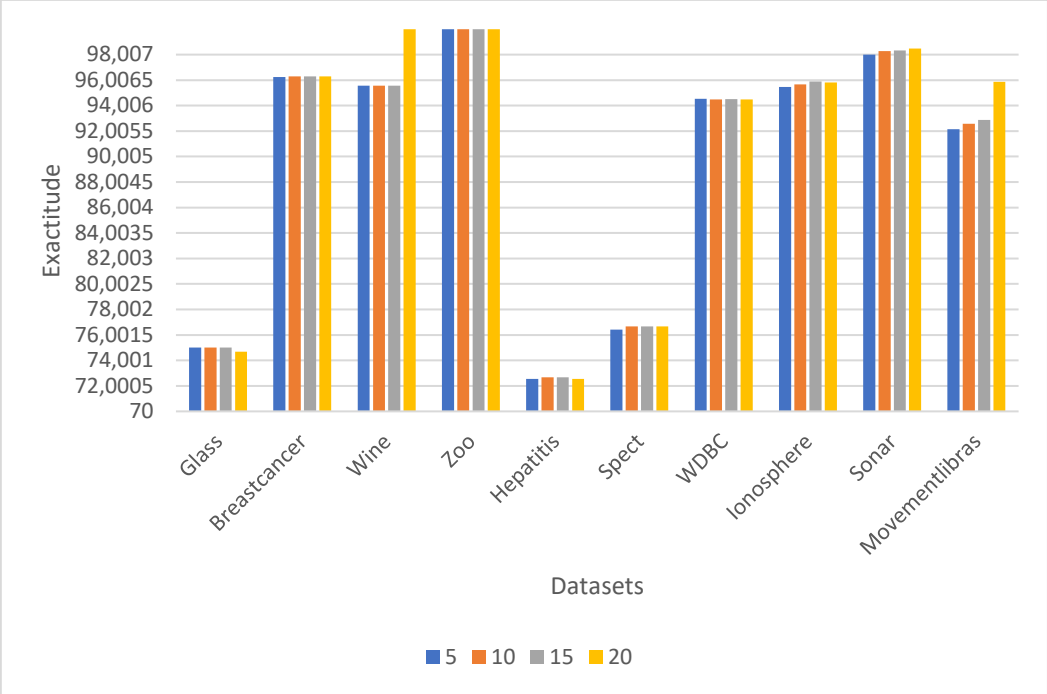


Figure V-7 - Le paramètre nombre d'abeilles pour sQBSO-FS

D'après le graphique un nombre d'abeilles qui est égale à 15 ainsi qu'un nombre d'abeilles qui est égale à 20 donnent les meilleures performances sur 7 des 10 datasets (avec un nombre d'abeilles égale à 15 on a des meilleures performances sur le dataset Ionosphere par rapport aux autres valeurs, pour la valeur 20 elle donne les meilleurs résultats pour les datasets Sonar et Movementlibras), ceci peut être expliqué que plus on a d'abeilles plus on a d'apprentissage et donc plus de meilleures solutions.

2) Flip

Analyse de variance :

Flip

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
2	10	896.185	89.6185	111.103437		
3	10	896.936	89.6936	110.496914		
4	10	897.208	89.7208	111.309722		
5	10	897.452	89.7452	111.873003		
10	10	897.865	89.7865	112.928695		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	0.15806508	4	0.03951627	**0.00035427**	0.99999974	**2.57873918**
A l'intérieur des groupes	5019.40593	45	111.542354			
Total	5019.56399	49				

Tableau V-13 - ANOVA pour l'effet de flip (sQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation de flip n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

D'après le graphique nous pouvons constater qu'une valeur de flip égale à 10 nous avons la meilleure exactitude pour 8 des 10 datasets (une même performance avec les autres valeurs pour 5 datasets).

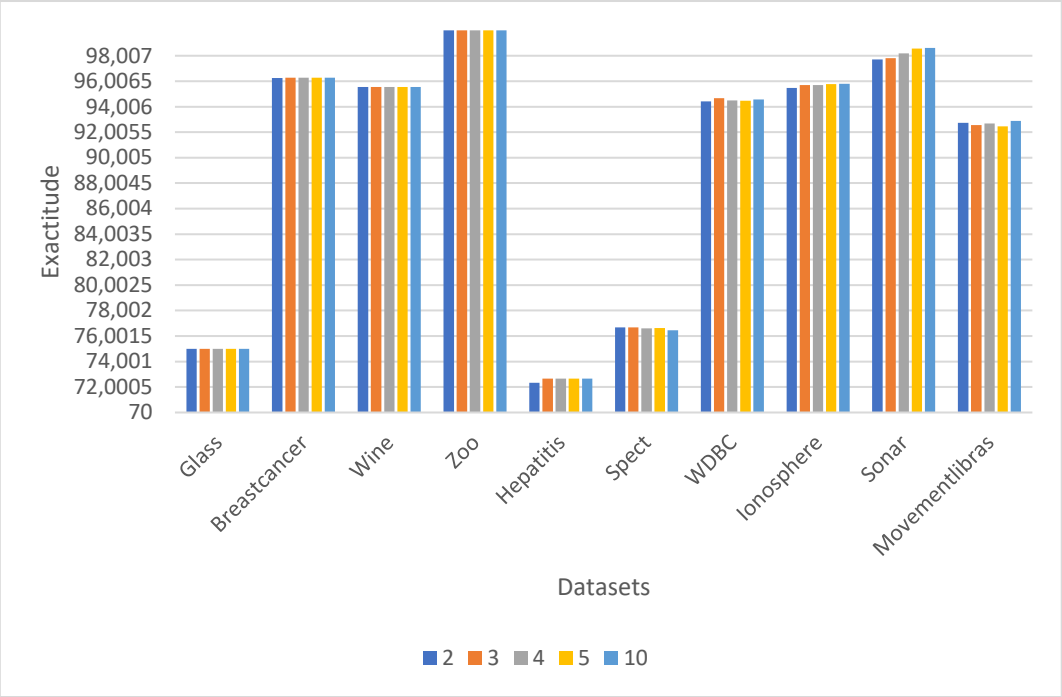


Figure V-8 - Le paramètre flip pour sQBSO-FS

3) MaxChances

Analyse de variance :

MaxChances

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
2	10	896.038	89.6038	111.049366		
3	10	896.958	89.6958	110.580334		
4	10	897.208	89.7208	111.309722		
5	10	895.982	89.5982	109.289476		
10	10	896.718	89.6718	110.34575		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	0.12076768	4	0.03019192	**0.00027319**	0.99999984	**2.57873918**
A l'intérieur des groupes	4973.17182	45	110.514929			
Total	4973.29259	49				

Tableau V-14 - ANOVA pour l'effet de MaxChances (sQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation de MaxChances n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

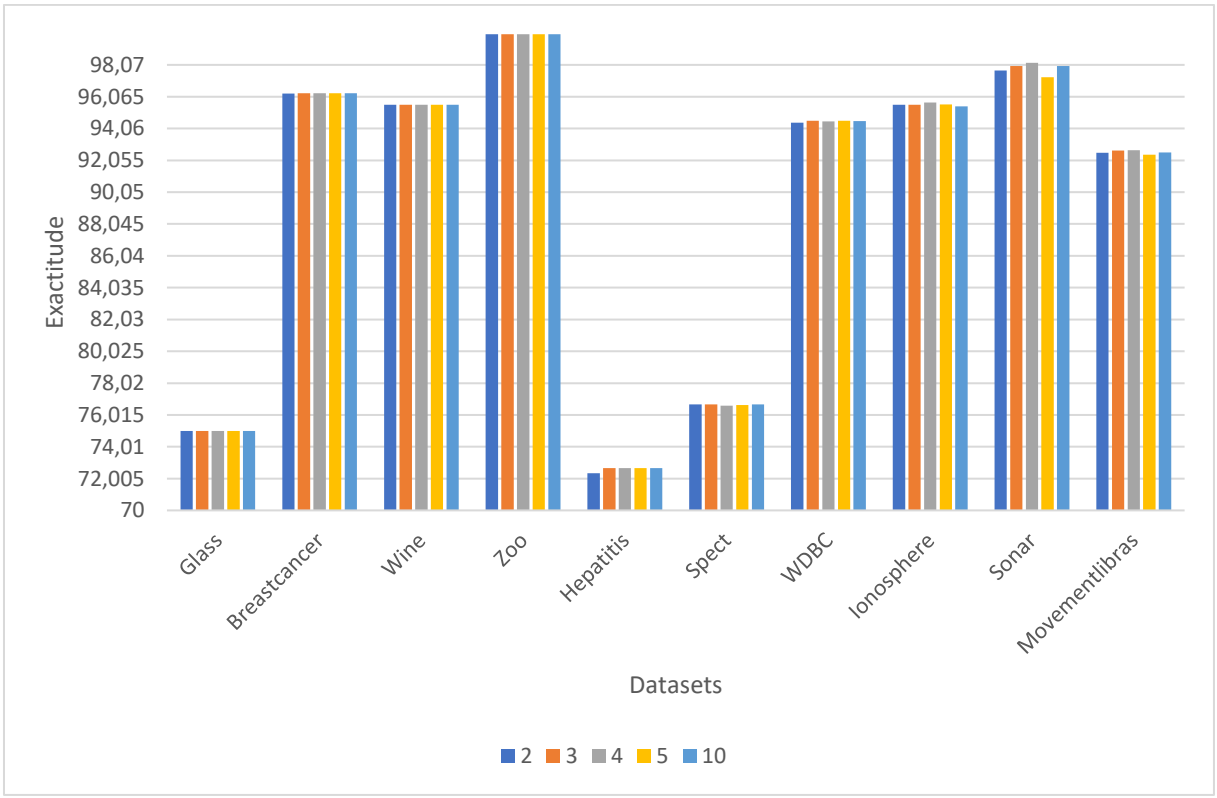


Figure V-9 - Le paramètre MaxChances pour sQBSO-FS

Pour le paramètre de diversification max chance nous pouvons voir que la valeur 4 est la plus adéquate vu qu'elle donne les meilleurs résultats sur 7 datasets (même performance avec d'autres valeur pour 6 datasets).

B. A/R

1) Le taux d'apprentissage (α)

Analyse de variance :

Le taux d'apprentissage (α)

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
0.1	10	897.22	89.722	111.164707		
0.25	10	897.45	89.745	111.593872		
0.5	10	897.59	89.759	111.680588		
0.75	10	897.668	89.7668	112.200894		
1	10	897.509	89.7509	111.650663		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	0.01165112	4	0.00291278	**2.6087 e-05**	1	**2.57873918**
A l'intérieur des groupes	5024.61651	45	111.658145			
Total	5024.62816	49				

Tableau V-15 - ANOVA pour l'effet du taux d'apprentissage (sQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du taux d'apprentissage flip n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre)) Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

Pour le paramètre alpha on peut voir qu'il n'y a pas une valeur pour laquelle on a la meilleure performance pour tous les datasets et donc sa valeur est égale à sa valeur par défaut qui est de 0.1.

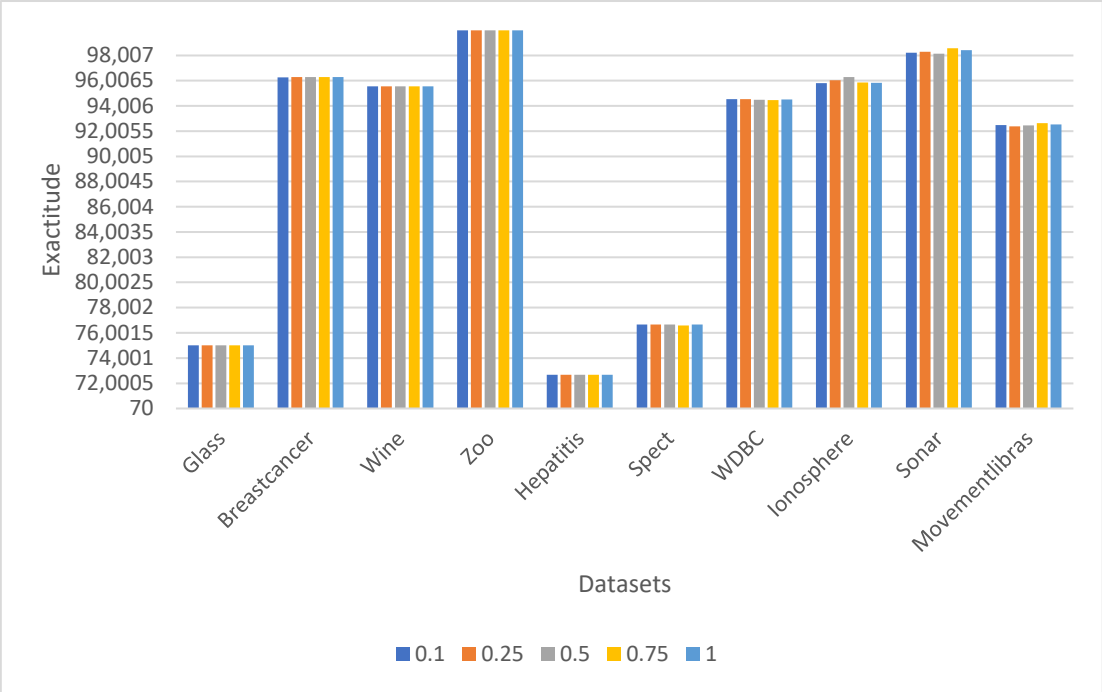


Figure V-10 - Le paramètre taux d'apprentissage pour sQBSO-FS

2) Le facteur de dépréciation (γ)

Analyse de variance :

Le facteur de dépréciation (γ)

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
0.8	10	897.82	89.782	112.123929		
0.85	10	897.228	89.7228	111.390985		
0.9	10	897.25	89.725	111.208383		
0.95	10	896.835	89.6835	112.604362		
0.99	10	895.96	89.596	112.221281		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	0.18939792	4	0.04734948	**0.0004231**	0.99999963	**2.57873918**
A l'intérieur des groupes	5035.94047	45	111.909788			
Total	5036.12986	49				

Tableau V-16 - ANOVA pour l'effet du facteur de dépréciation (sQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du facteur de dépréciation n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

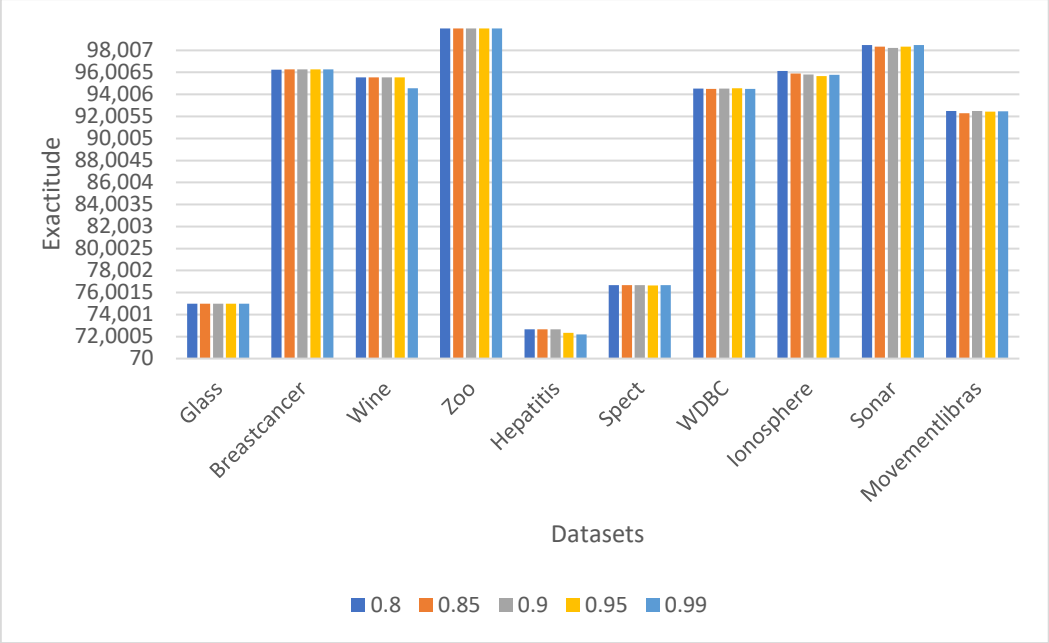


Figure V-11 - Le paramètre facteur de dépréciation pour sQBSO-FS

Pour le paramètre gamma on peut voir qu'il n'y a pas une valeur pour laquelle on a la meilleure performance pour une majorité de datasets et donc sa valeur est égale à sa valeur par défaut qui est de 0.9.

3) Le facteur d'exploration (ε)

Analyse de variance :

Le facteur d'exploration (ε)

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
0.01	10	897.23	89.723	111.322973		
0.05	10	897.02	89.702	111.526218		
0.1	10	897.25	89.725	111.208383		
0.2	10	896.27	89.627	111.763246		
0.5	10	894.606	89.4606	110.568395		
0.8	10	893.09	89.309	108.609388		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	1.455251	5	0.2910502	**0.00262602**	0.99999888	**2.38606986**
A l'intérieur des groupes	5984.98742	54	110.8331			
Total	5986.44267	59				

Tableau V-17 - ANOVA pour l'effet du facteur d'exploration (sQBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du facteur d'exploration n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

D'après le graphique les valeurs 0.01 et 0.1 donnent les meilleurs résultats, nous choisissons la valeur 0.1 vu qu'elle donnent plus de chance à la diversification si on veut utiliser d'autres dataset.

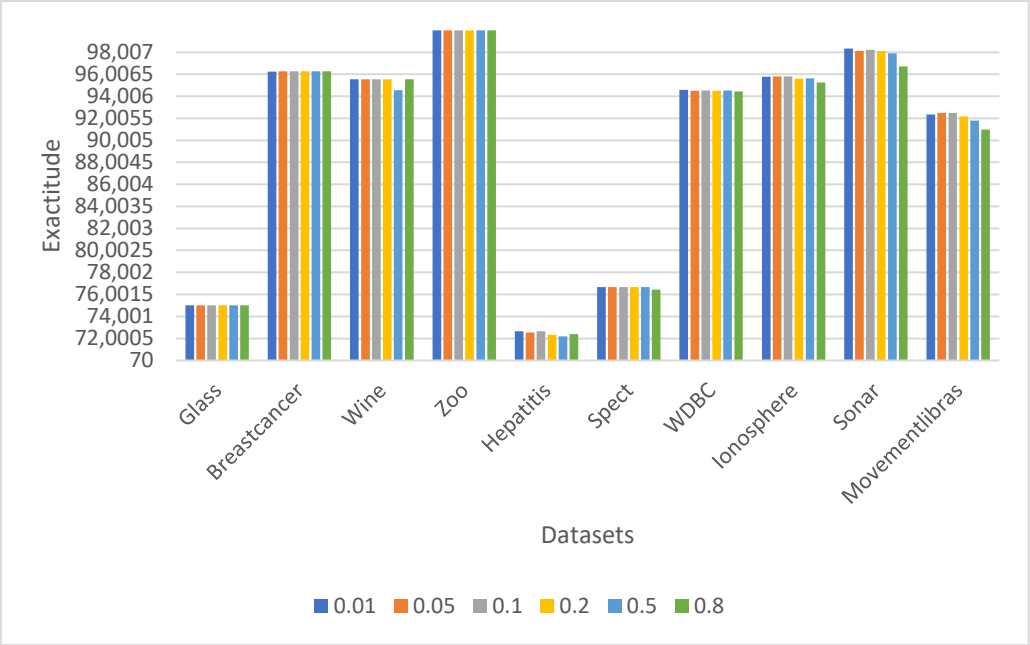


Figure V-12 - Le paramètre taux d'exploration pour sQBSO-FS

4.1.3. QsBSO-FS

A. Paramètres de BSO

1) Nombre d'abeilles

Analyse de variance :

Nombre d'abeilles

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
5	10	866.45	86.645	129.651206		
10	10	856.05	85.605	136.084228		
15	10	855.31	85.531	137.20921		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	7.76024	2	3.88012	**0.02888824**	0.97155503	**3.35413083**
A l'intérieur des groupes	3626.50179	27	134.314881			
Total	3634.26203	29				

Tableau V-18 - ANOVA pour l'effet du nombre d'abeilles (QsBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du nombre d'abeilles n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

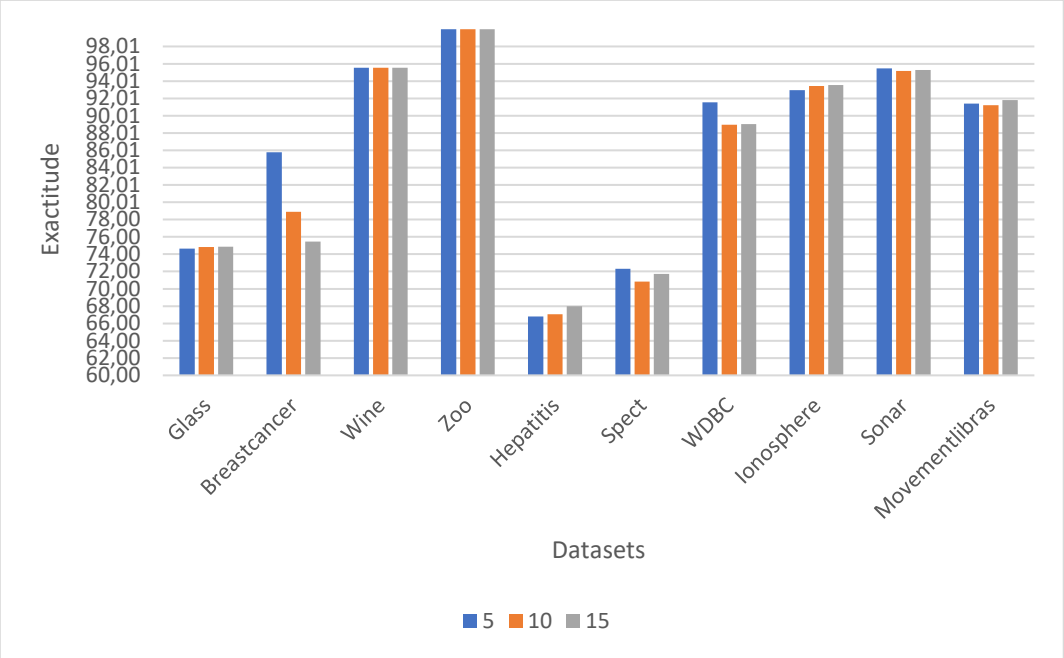


Figure V-13 - Le paramètre nombre d'abeilles pour QsBSO-FS

On peut voir que dans 6 des 10 datasets un nombre d'abeilles qui est égale à 5 donne les meilleurs performance ceci peut s'expliquer par le fait que moins on a de choix de solution de référence à sélectionner plus le choix est facile et plus intéressant (on a plus de chance de tomber sur des solutions déjà visité et donc on aura un apprentissage plus intéressant).

2) Max itérations

Analyse de variance :

Max itérations

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
10	10	859.83	85.983	132.585512		
15	10	860.17	86.017	133.767712		
20	10	858.4	85.84	133.090733		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	0.17644667	2	0.08822333	**0.0006626**	0.99933764	**3.35413083**
A l'intérieur des groupes	3594.99562	27	133.147986			
Total	3595.17207	29				

Tableau V-19 - ANOVA pour l'effet de MaxIterations (QsBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation de MaxIterations n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

Contrairement aux deux méthodes précédentes où MaxIterations n'est pas considéré comme paramètre. Dans cette méthode ce paramètre est équivalent au nombre d'épisodes dans le jargon de l'A/R.
D'après l'histogramme on ne peut pas tirer une meilleure valeur pour ce paramètre. Nous supposons de ce fait que plus on a d'itérations mieux c'est, et nous choisissons donc la valeur 20 pour ce paramètre.

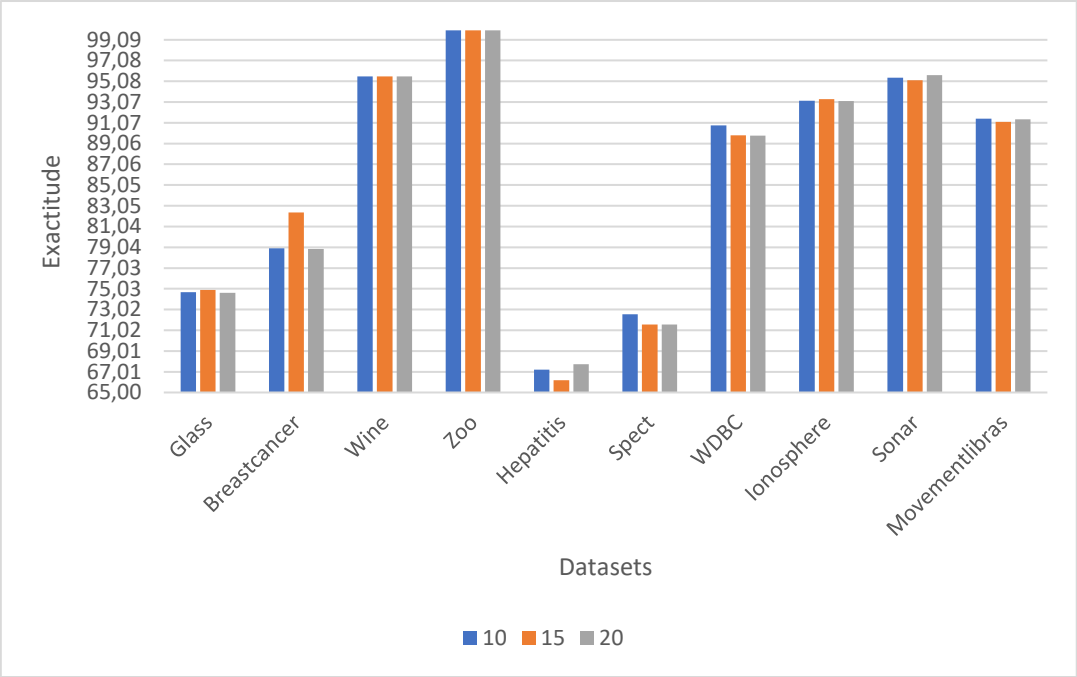


Figure V-14 - Le paramètre MaxIterations pour QsBSO-FS

B. A/R

1) Le taux d'apprentissage (α)

Analyse de variance :

Le taux d'apprentissage (α)

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
0.1	10	875.26	87.526	134.769604		
0.2	10	848.54	84.854	160.276249		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	35.69792	1	35.69792	**0.24198218**	0.6287235	**4.41387342**
A l'intérieur des groupes	2655.41268	18	147.522927			
Total	2691.1106	19				

Tableau V-20 - ANOVA pour l'effet du taux d'apprentissage (QsBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du taux d'apprentissage n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

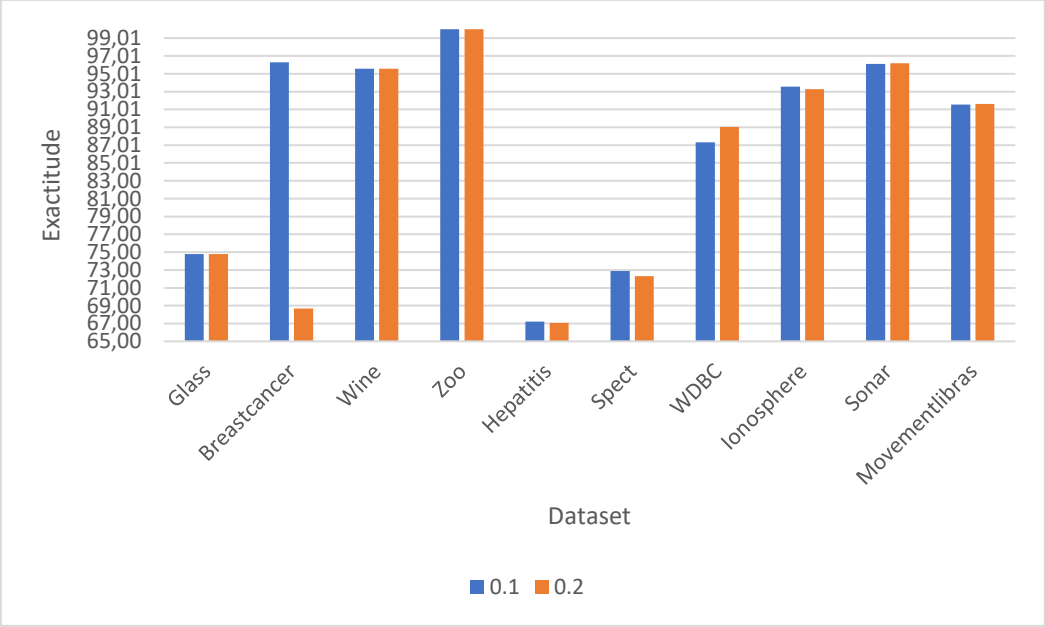


Figure V-15 - Le paramètre taux d'apprentissage pour QsBSO-FS

Pour ce paramètre nous remarquons que pour une valeur de 0.1 on a de meilleures performances pour 4 des 10 datasets, contrairement à la valeur 0.2 qui donne de meilleur résultats pour un seul dataset qui est WDBC, les deux valeurs ont les même performance pour 3 datasets. D'après cette analyse la valeur de alpha est fixée à 0.

2) Le facteur de dépréciation (γ)

Analyse de variance :

Le facteur de dépréciation (γ)

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
0.8	10	879.03	87.903	127.103246		
0.9	10	862.83	86.283	125.943134		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	13.122	1	13.122	**0.10371221**	0.7511296	**4.41387342**
A l'intérieur des groupes	2277.41742	18	126.52319			
Total	2290.53942	19				

Tableau V-21 - ANOVA pour l'effet du facteur de dépréciation (QsBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du facteur de dépréciation n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

D'après le graphique on ne peut pas dire que la valeur 0.8 semble meilleure et donc elle est choisie.

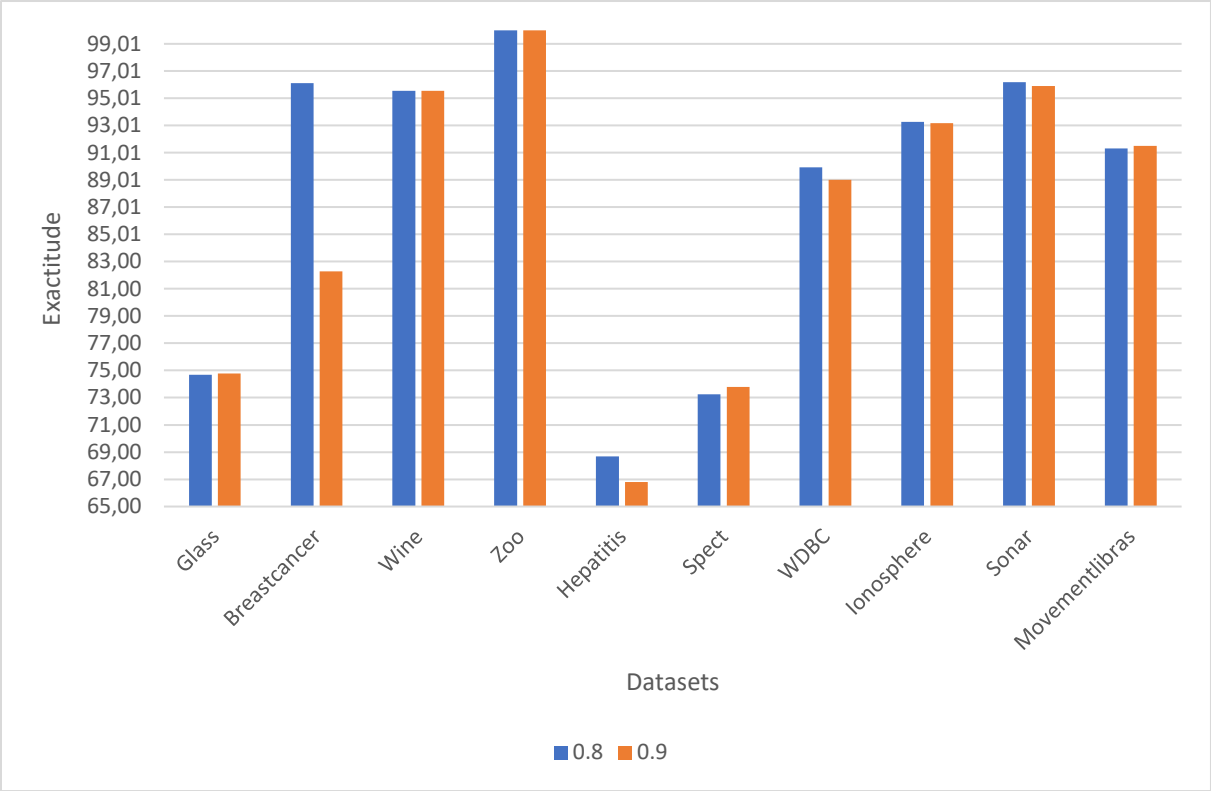


Figure V-16 - Le paramètre facteur de dépréciation pour QsBSO-FS

3) Le facteur d'exploration (ε)

Analyse de variance :

Le facteur d'exploration (ε)

RAPPORT DÉTAILLÉ						
Groupes	*Nombre d'échantillons*	*Somme*	*Moyenne*	*Variance*		
0.1	10	849.23	84.923	156.506868		
0.05	10	858.87	85.887	142.389979		
ANALYSE DE VARIANCE						
Source des variations	*Somme des carrés*	*Degré de liberté*	*Moyenne des carrés*	*F*	*Probabilité*	*Valeur critique pour F*
Entre Groupes	4.64648	1	4.64648	**0.03109086**	0.86200651	**4.41387342**
A l'intérieur des groupes	2690.07162	18	149.448423			
Total	2694.7181	19				

Tableau V-22 - ANOVA pour l'effet du facteur d'exploration (QsBSO-FS)

Dans le tableau ci-dessus on essaie de tester l'hypothèse H_0 pour laquelle la variation du facteur d'exploration n'influe pas sur la qualité de l'exactitude. D'après le tableau on remarque que la valeur de F est considérablement inférieure à la valeur critique de F, l'hypothèse H_0 est donc conservé, mais cela ne veut pas dire que l'hypothèse est juste, mais que nous ne possédons pas d'assez de données pour la réfuter. (Par exemple, l'influence d'un autre facteur (paramètre))
Nous présentons dans les histogrammes ci-dessous les exactitudes pour chaque dataset selon la valeur choisie pour voir en tirer celle qui est la plus prolifique de ce paramètre.

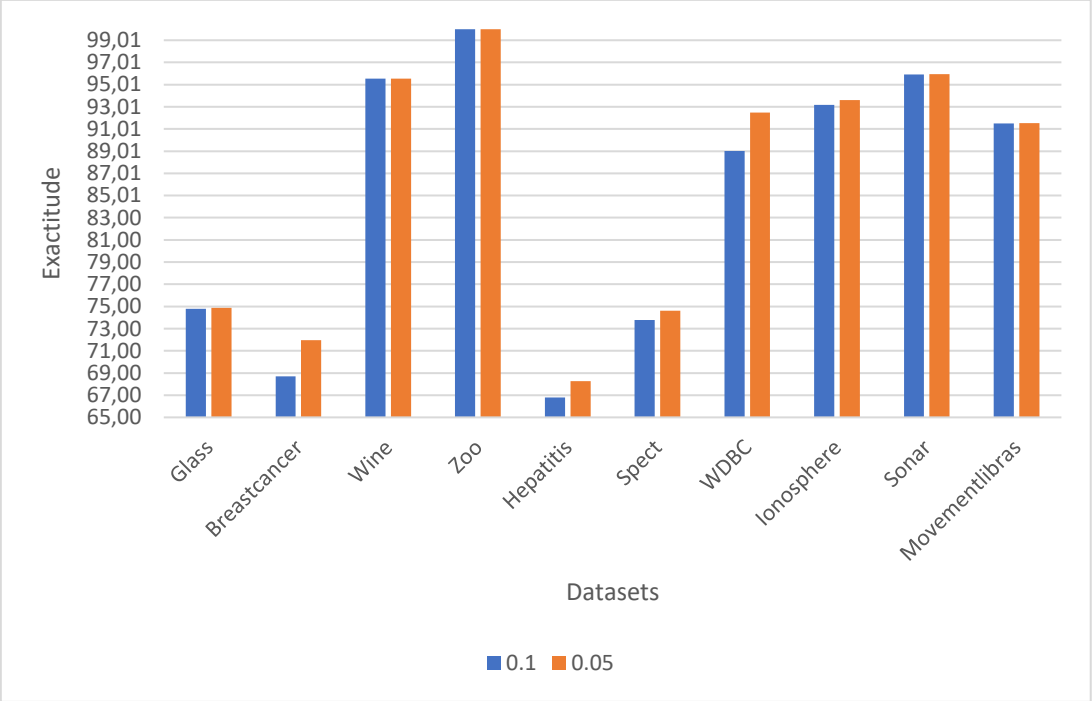


Figure V-17 - Le paramètre taux d'exploration pour QsBSO-FS

D'après le graphique nous pouvons voir que la valeur 0.05 offre les meilleurs résultats et elle est donc choisie.

4.2. Etude comparative

Dans cette sous-section nous allons effectuer en premier lieu une comparaison entre les différentes approches que nous avons proposé, nous choisirons selon cette comparaison la meilleure approche et nous la comparerons à BSO classique ainsi qu'à d'autres méthodes existantes qui traite notre sujet d'études (la sélection d'attributs).

4.2.1. Etude comparative des approches proposées

D'après l'étude empirique que nous avons effectuée, les meilleurs valeurs pour chaque méthode sont comme suit :

1. **iQBSO-FS**

Nombre d'abeilles	Flip	Max chance	Alpha	Gamma	Epsilon
10	4	3	0.1	0.9	0.1

Tableau V-23 - Meilleurs paramètres pour iQBSO-FS

2. **sQBSO-FS**

Nombre d'abeilles	Flip	Max chance	Alpha	Gamma	Epsilon
10	4	3	0.1	0.9	0.1

Tableau V-24 - Meilleurs paramètres pour sQBSO-FS

3. **QsBSO-FS**

Nombre d'abeilles	Max itérations	Alpha	Gamma	Epsilon
5	20	0.05	0.8	0.1

Tableau V-25 - Meilleurs paramètres pour QsBSO-FS

Remarques importantes :

En ce qui concerne les méthodes **iQBSO-FS** et **sQBSO-FS** les paramètres de BSO (nombres, d'abeilles, flip et max chance) sont les mêmes que dans le travail sur BSO-FS effectué par (Sadeg et al., 2015) pour pouvoir comparer la meilleure méthode entre les deux avec cette dernière.

Le tableau ci-dessous représente une comparaison entre les trois approches selon l'exactitude et le nombre d'attributs des solutions trouvées :

Dataset	Algorithme	Moyenne d'attributs trouvée	Moyenne d'exactitude (%)	Meilleur nombre d'attributs trouvé	Meilleure exactitude (%)
Breastcancer	iQBSO-FS	3.43	96.29	3	96.29
	sQBSO-FS	3.43	96.29	3	96.29
	QsBSO	4.9	96.12	4	96.29
Glass	iQBSO-FS	6.3	75.00	6	75.00
	sQBSO-FS	6	75.00	6	75.00
	QsBSO	6.3	74.69	6	75.00
Hepatitis	iQBSO-FS	7	72.67	7	72.67
	sQBSO-FS	7	72.67	7	72.67
	QsBSO	11.5	68.27	8	70.00
Ionosphere	iQBSO-FS	11.4	95.56	8	96.11
	sQBSO-FS	11.27	95.81	8	96.94
	QsBSO	14.3	93.28	12	94.44
Movementlibras	iQBSO-FS	30.8	92.53	25	93.06
	sQBSO-FS	31.87	92.49	23	93.33
	QsBSO	38.4	91.31	30	92.5
Sonar	iQBSO-FS	27.5	98.10	24	98.57
	sQBSO-FS	28	98.22	23	99.05
	QsBSO	28.1	96.19	23	97.62
Spect	iQBSO-FS	16	76.67	16	76.67
	sQBSO-FS	16	76.67	16	76.67
	QsBSO	12.7	73.26	16	76.67
WDBC	iQBSO-FS	12.5	94.51	11	94.51
	sQBSO-FS	12.3	94.54	7	95.61
	QsBSO	15.8	89.93	13	94.39
Wine	iQBSO-FS	6.6	95.56	6	95.56
	sQBSO-FS	6.83	95.56	6	95.56
	QsBSO	7	95.56	7	95.56
Zoo	iQBSO-FS	10.3	100	9	100
	sQBSO-FS	8.17	100	7	100
	QsBSO	9.9	100	9	100

Tableau V-26 - Tableau comparatif des résultats des hybridations

D'après les résultats du tableau ci-dessus nous pouvons voir que la méthode sQBSO-FS est la plus performante en effet cette dernière a la meilleure moyenne d'exactitude pour trois datasets (Ionosphere, Sonar et WDBC) et elle a la même moyenne d'exactitude que iQBSO-FS pour les datasets (Breastcancer, Glass, Hepatitis, Spect), iQBSO-FS bat la sQBSO-FS sur un seul dataset

qui est Movementlibras, cette dernière cependant donne une meilleure exactitude maximale sur ce même dataset (93.33% pour la sQBSO-FS contre 93.06% pour la iQBSO-FS).
Les trois méthodes ont la même performance en termes de moyenne d'exactitude sur les deux datasets restants à savoir Wine et Zoo, néanmoins la sQBSO-FS donne de meilleurs résultats sur la moyenne de nombre d'attributs trouvé pour Zoo et la iQBSO-FS donne la meilleure performance pour Wine pour cette métrique.
D'après la comparaison précédente nous pouvons constater que la iQBSO-FS et la sQBSO-FS sont très proche en termes d'exactitude, et elles sont de loin plus performantes que la QsBSO pour cette même métrique, il serait donc très intéressant de comparer leurs temps d'exécutions pour trouver la meilleure méthode.

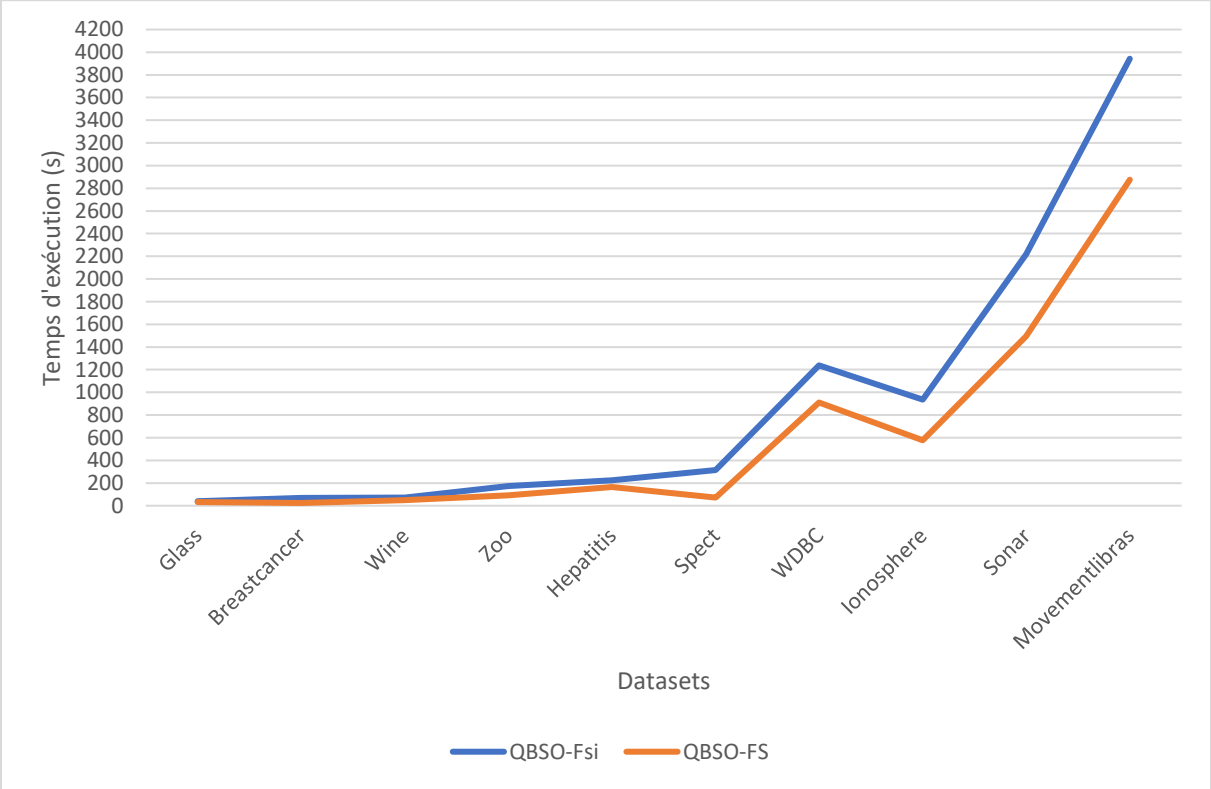


Figure V-18 - Comparaison entre le temps d'exécution de sQBSO-FS et iQBSO-FS

D'après les deux courbes on peut voir que pour les petits datasets (les datasets qui ont moins de 19 attributs) les temps d'exécutions sont assez proche la différence est plus importante quand il s'agit de datasets de moyenne et de grande taille en faveur de la sQBSO-FS et ceci est dû à l'intelligence collective des abeilles dans cette méthode, en effet avoir une Q-table commune pour toutes les abeilles permet d'avoir une même connaissance et une même mémoire pour

toutes les abeilles ceci permettra d'évaluer moins de solutions et donc moins de temps d'exécution.

D'après les résultats précèdent il est clair que la sQBSO-FS est la meilleure méthode, dans ce qui va suivre on va la comparer à la BSO-FS classique ainsi qu'à d'autres travaux existants.

4.2.2. Comparaison entre BSO-FS et sQBSO-FS

Le tableau suivant montre une comparaison entre QBSO et BSO selon l'exactitude et le nombre d'attributs des solutions trouvées :

Dataset	Algorithme	Moyenne d'attributs trouvé	Moyenne d'exactitude (%)	Meilleur nombre d'attributs trouvé	Meilleure exactitude (%)
Breastcancer	Q-BSO	3.43	96.29	3	96.29
	BSO	3.73	96.29	3	96.29
Congress	Q-BSO	6	97.95	6	97.95
	BSO	6	97.95	6	97.95
Diabets	Q-BSO	4	70.65	4	70.65
	BSO	4	70.65	4	70.65
German	Q-BSO	9	74.20	9	74.20
	BSO	9	74.20	9	74.20
Glass	Q-BSO	6	75.00	6	75.00
	BSO	6.47	75.00	6	75.00
Heart-C	Q-BSO	6	58.71	6	58.71
	BSO	4.53	58.59	2	58.71
Heart-StatLog	Q-BSO	5	82.59	5	82.59
	BSO	5	82.59	5	82.59
Hepatitis	Q-BSO	7	72.67	7	72.67
	BSO	6.87	71.96	6	72.67
Ionosphere	Q-BSO	11.27	95.81	8	96.94
	BSO	11.5	95.59	7	96.11
Iris	Q-BSO	2.27	97.33	2	97.33
	BSO	3.87	97.33	2	97.33
Lung-Cancer	Q-BSO	20.97	98.25	16	100
	BSO	25.13	99.67	18	100
Lymphography	Q-BSO	9.87	43.31	10	43.33
	BSO	10	43.33	10	43.33
Movementlibras	Q-BSO	31.87	92.49	23	93.33
	BSO	31.53	93.13	26	93.61
Sonar	Q-BSO	28	98.22	23	99.05
	BSO	26.37	99.04	18	99.52
Spect	Q-BSO	16	76.67	16	76.67
	BSO	16	76.67	16	76.67
Vehicule	Q-BSO	8	74.47	8	74.47
	BSO	8	74.47	8	74.47

Vowel	Q-BSO	8.4	99.34	8	99.45
	BSO	9	99.45	9	99.45
WDBC	Q-BSO	12.3	94.54	7	95.61
	BSO	13	94.55	8	94.56
Wine	Q-BSO	6.83	95.56	6	95.56
	BSO	6.73	95.56	6	95.56
Zoo	Q-BSO	8.17	100	7	100
	BSO	9.47	100	7	100

Tableau V-27 - Comparaison entre BSO-FS et sQBSO-FS

Les résultats numériques donnés dans le tableau ci-dessus montrent que BSO-FS et sQBSO-FS offrent généralement la même exactitude, sauf en ce qui concerne Ionosphère, pour lequel sQBSO-FS surclasse BSO-FS. On peut aussi noter que sQBSO-FS atteint les meilleures exactitudes avec moins d'attributs dans de nombreux datasets. On peut citer comme exemple le dataset Lung-Cancer pour lequel une précision égale à (100%) est obtenue avec seulement 16 attributs sur 56 contre 18 pour BSO-FS. Les résultats montrent aussi que BSO-FS et sQBSO-FS sont très stables puisque leur valeur moyenne d'exactitude est égale à la meilleure exactitude obtenue pour presque tous les datasets.

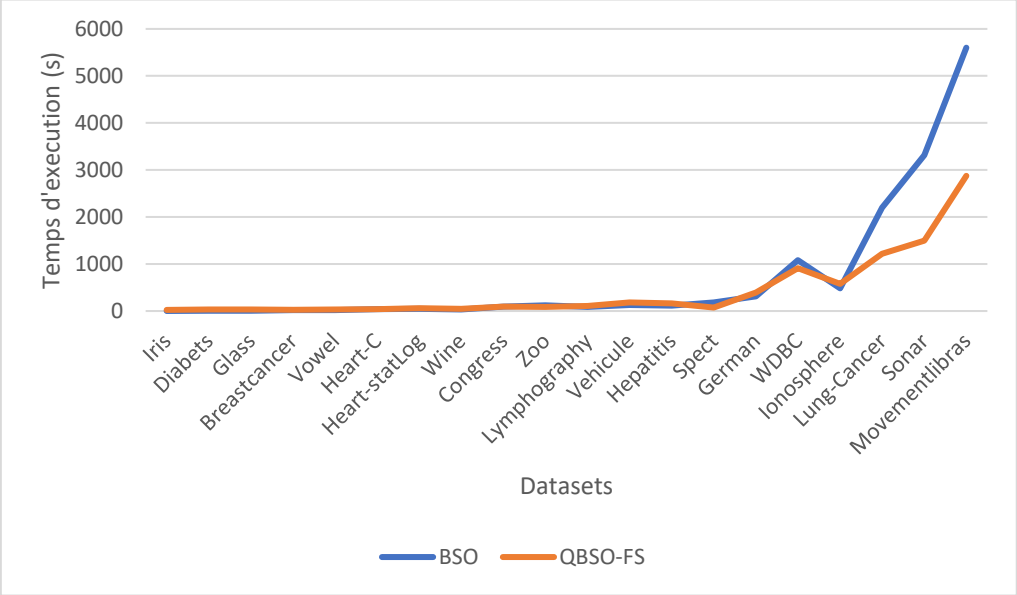


Figure V-19 - Comparaison du temps d'exécution pour BSO-FS et sQBSO-FS

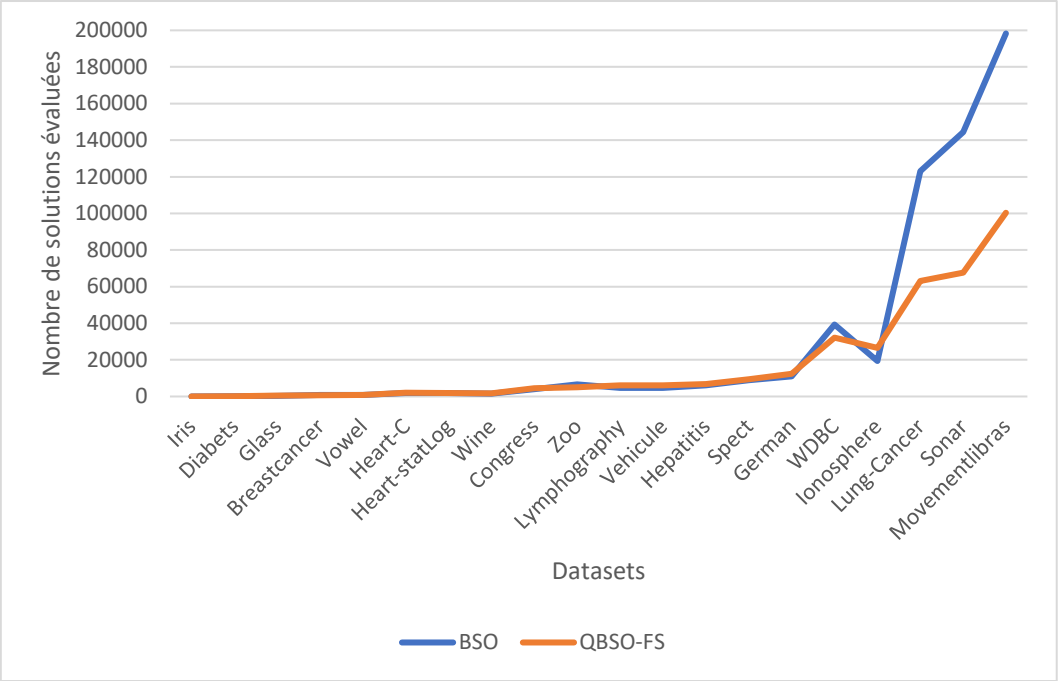


Figure V-20 - Comparaison du nombre de solutions évalué par BSO-FS et sQBSO-FS

En termes de temps d'exécution, les deux graphes précédents montrent que sQBSO-FS et BSO sont équivalente en termes de temps d'exécution pour les instances de petite et moyenne taille. sQBSO-FS est plus rapide pour les grands dataset. Ceci s'explique par le fait que plus le nombre de solutions évaluées précédemment est élevé, plus le processus de recherche est complexe, guidé et adaptatif. Par conséquent, les bonnes solutions sont atteintes plus rapidement.

4.2.3. Comparaison entre sQBSO-FS et les travaux existants

Dans cette sous-section nous allons comparer la **sQBSO-FS** avec trois algorithmes qui traite le problème de la sélection d'attributs et qui utilise la classifieur KNN également, les algorithmes sont Particle Swarm Optimization (PSO) proposé par (Xue, Zhang, & Browne, 2014), Hybrid Binary Bat Enhanced Particle Swarm Optimization (HBBEPSO) proposé par (Tawhid & Dsouza, 2018) et l'algorithme génétique pour la sélection d'attributs (AG) proposé par (Yang & Honavar, 1998).

Les résultats sont présentés dans le tableau suivant :

Dataset	Exactitude sans la sélection d'attributs	PSO	QBSO	HBBEPSO	GA
Wine	81.67	95	95.56	90.1	93
Sonar	80.95	74	98.22	70.4	73
Breastcancer	61.86	97	96.29	95.8	96
Heart-StatLog	60.74	78	82.59	81.3	82
Ionosphere	86.39	84	95.81	83.1	83
Zoo	99.09	83	100	87.3	88
WDBC	90.18	94	94.54	94.5	94
Congress	94.55	94	97.95	94.4	94
Lymphography	26.67	35.4	43.31	42.2	-
Spect	63.70	77	76.67	76.4	78

Tableau V-28 - Comparaison avec les travaux existants

D'après le tableau nous pouvons voir que sQBSO-FS surpasse les autres algorithmes pour 8 des 10 datasets utilisés dans ces tests. En effet, PSO a réalisé une meilleure performance pour Breastcancer (97%) et Spect (77%), l'AG a atteint le meilleur résultat pour ce dernier dataset (78 %) contre 76,67% pour sQBSO-FS. Pour les dataset restants (Wine, Sonar, Heart-StatLog, Ionosphere, Zoo, WDBC, Congress, Lymphographie) sQBSO-FS a obtenu les meilleurs résultats.

5. Conclusions par rapport aux interprétations

D'après l'étude comparative effectué nous pouvons tirer les conclusions suivantes :

- L'utilisation du Q-Learning au niveau de la recherche locale avec une Q-table partagé entre les abeilles est l'hybridation la plus adéquate de BSO avec l'A/R, en effet cette méthode a gardé les mêmes performances de la BSO classique en termes de qualité de solutions (exactitude) pour la majorité des datasets testé et avec moins d'attributs, elle permet également grâce à sa Q-Table d'avoir moins de solutions à évaluer et donc moins de temps d'exécutions pour les datasets avec un grand nombre d'attributs.

- Un apprentissage collectifs entre les abeilles permet une meilleure exactitude et un temps d'exécutions réduits.
- L'utilisation du A/R au niveau de l'essaim pour une métaheuristique comme BSO semble moins intéressantes que son utilisation au niveau de la recherche locale, la force de ce type de méthode est au partage de solutions entre individu, il est donc plus judicieux d'intégré le A/R au niveau de chaque individu et rendre l'apprentissage collectif (partagé).

6. Conclusion

Dans ce chapitre, nous avons présenté les résultats des hybridations proposées dans le chapitre précédent pour la résolution du problème de sélection d'attributs sur des instances de la littérature.

Pour une interprétation juste des résultats obtenus, nous avons procédé à une analyse statistique en utilisant l'ANOVA, ainsi qu'une étude empirique des paramètres utilisés dans les différentes hybridations ainsi qu'une étude comparative, qui a permis de conclure qu'on a effectivement atteints nos objectifs puisque nous obtenons un temps d'exécution réduit pour les grandes instances pour la même qualité d'exactitude de classification.

Nous pouvons dire aussi que les métaheuristiques proposées présentent une stabilité sur la qualité des solutions retournées, notamment, sQBSO-FS qui a permis de publier un article sous le nom de : « *QBSO-FS: A Reinforcement Learning Based Bee Swarm Optimization Metaheuristic for Feature Selection* » (Sadeg et al., 2019).

Puisque le but initiale de ce travail était d'étudier l'effet d'intégration de l'A/R dans BSO, et donc comme perspectives, nous pensons que la prochaine étape est sur deux axes : premièrement, tester ces hybridations sur de différents classifieurs (SVM, Naive Bayes, Random Forest, Decision Trees, …), pour pouvoir observer leur comportement sous ces conditions, et déterminer leur stabilité pour cette dimension. Et deuxièment, explorer les deux autres approches d'intégrer l'A/R aux métaheuristiques, notamment, pour le choix du voisinage et pour le paramétrage.

Conclusion générale et perspectives

La sélection d'attributs, se propose d'épurer un ensembles de données, cette épuration consiste à éliminer les données redondantes et non pertinentes. Cette tâche n'est pas toujours facile, étant donné le grand nombre d'attributs que les données peuvent avoir et la diversification des sources et des types de données.

Dans le cadre de ce travail, nous nous sommes intéressés à l'amélioration des performances d'une métaheuristique appelée BSO (Bee Swarm Optimization) en utilisant l'apprentissage par renforcement, et cela pour la résolution du problème de sélection d'attributs pour la classification supervisée. BSO est une métaheuristique considérée comme étant très bonne en termes de qualité des solutions fournies, mais pour le temps d'exécution, elle peut être trop gourmande, surtout quand les instances du problème à résoudre commencent à prendre de la taille et à devenir plus complexes.

De son côté, l'apprentissage par renforcement, qui est une branche de l'apprentissage automatique, et qui a fait ses preuves face à des problèmes auparavant estimés pouvoir être résolus à l'horizon 2050. Nous nous sommes donc intéressés à son intégration dans le domaine d'optimisation combinatoire et plus précisément aux métaheuristiques, donnant ainsi une nouvelle forme à ce qu'on appelle les métaheuristiques hybrides. L'intégration de l'apprentissage par renforcement à BSO fait partie de ces formes d'hybridations, de plus cette intégration, selon la littérature, peut-être réalisé de quatre manières différentes : en remplaçant la recherche locale de la métaheuristiques, en appliquant l'apprentissage par renforcement directement sur la métaheuristique, en utilisant l'A/R pour l'ajustement des paramètres ou en utilisant l'A/R pour le choix d'un voisinage.

Dans ce présent travail, nous nous sommes intéressés aux deux premières approches, en se basant sur l'algorithme de Q-Learning, qui vise à guider les abeilles en leur fournissant de la connaissance à travers la *Q-Table*. A travers ce travail, notre but a été de proposer des versions hybrides de la métaheuristique BSO dans le but d'améliorer ses performances en essayant de réduire son temps d'exécution tout en maintenant une aussi bonne qualité de solutions.

Nous avons présenté, expliqué, discuté et schématisé l'ensemble des trois métaheuristiques hybrides que nous proposons pour améliorer les performances de la métaheuristique BSO qui étaient : iQBSO-FS, sQBSO-FS et QsBSO-FS. La première consiste à doter l'abeille d'une connaissance supplémentaire, sous forme de *Q-Table* individuelle. La seconde, est semblable à la première mais la *Q-Table* est commune pour toutes les abeilles. Et la dernière, consiste à rendre le choix de la solution de référence guidé.

Pour conclure, nous pouvons dire que l'objectif principal visé par ce travail a été atteint, vu que les algorithmes proposés ont tous tendance à maintenir une qualité aussi proche que celle de BSO en un temps beaucoup plus réduits, même si certaines, comme QsBSO-FS par exemple, peuvent être améliorés, surtout en termes de qualité.

Comme perspectives pour le fonctionnement général de BSO, et avant d'envisager une hybridation, la notion de parallélisme peut être intégrée, vu que, dans BSO les différentes abeilles sont indépendantes les unes des autres, donc nous pouvons intuitivement considérer chaque abeille comme étant un processus à part, ce qui permet d'accélérer d'une façon remarquable la métaheuristique BSO. De plus, la métaheuristique BSO ainsi que les hybridations proposées dans ce travail, peuvent être adaptées pour une éventuelle application sur d'autres problèmes d'optimisation combinatoire. Enfin, d'autres algorithmes d'A/R peuvent être introduits pour remplacer l'algorithme de Q-Learning utilisé pour ce travail, ainsi que d'autres classifieurs autre que KNN.

Bibliographie

[1] Akar, N., & Sahin, C. (2003). Reinforcement learning as a means of dynamic aggregate QoS provisioning. *International Workshop on Architectures for Quality of Service in the Internet*, 100–114. Springer.

[2] Altman, N. S. (1992). An Introduction to Kernel and Nearest-Neighbor Nonparametric Regression. *The American Statistician*, *46*(3), 175. https://doi.org/10.2307/2685209

[3] Arulkumaran, K., Deisenroth, M. P., Brundage, M., & Bharath, A. A. (2017). A brief survey of deep reinforcement learning. *arXiv preprint arXiv:1708.05866*.

[4] Battiti, R., & Campigotto, P. (2008). Reinforcement Learning and Reactive Search: an adaptive MAX-SAT solver. *ECAI*, 909–910.

[5] Ben-Bassat, M. (1982). Pattern recognition and reduction of dimensionality. *Handbook of Statistics*, *2*(1982), 773–910.

[6] Bengio, Y., & Grandvalet, Y. (2004). No unbiased estimator of the variance of k-fold cross-validation. *Journal of machine learning research*, *5*(Sep), 1089–1105.

[7] Besse, P., & Ferré, L. (1993). Sur l'usage de la validation croisée en analyse en composantes principales. *Revue de statistique appliquée*, *41*(1), 71–76.

[8] Bianchi, R. A., Ribeiro, C. H., & Costa, A. H. (2009). On the relation between ant colony optimization and heuristically accelerated reinforcement learning. *1st international workshop on hybrid control of autonomous system*, 49–55. Citeseer.

[9] Bradtke, S. J., & Duff, M. O. (1995). Reinforcement learning methods for continuous-time Markov decision problems. *Advances in neural information processing systems*, 393–400.

[10] Burke, E. K., & Kendall, G. (2005). *Search methodologies*. Springer.

[11] Burke, E. K., Kendall, G., & Soubeiga, E. (2003). A tabu-search hyperheuristic for timetabling and rostering. *Journal of heuristics*, *9*(6), 451–470.

[12] Burke, E., Kendall, G., Newall, J., Hart, E., Ross, P., & Schulenburg, S. (2003). Hyper-heuristics: An emerging direction in modern search technology. In *Handbook of metaheuristics* (p. 457–474). Springer.

[13] Crainic, T. G., Toulouse, M., & Gendreau, M. (1995). Synchronous tabu search parallelization strategies for multicommodity location-allocation with balancing requirements. *Operations-Research-Spektrum*, *17*(2-3), 113–123.

[14] Crites, R. H., & Barto, A. G. (1996). Improving elevator performance using reinforcement learning. *Advances in neural information processing systems*, 1017–1023.

[15] Das, T. K., Gosavi, A., Mahadevan, S., & Marchalleck, N. (1999). Solving semi-Markov decision problems using average reward reinforcement learning. *Management Science*, *45*(4), 560–574.

[16] Dash, M., & Liu, H. (1997). Feature selection for classification. *Intelligent data analysis*, *1*(1-4), 131–156.

[17] Doak, J. (1992). *Cse-92-18-an evaluation of feature selection methodsand their application to computer security*.

[18] Dorigo, M., & Blum, C. (2005). Ant colony optimization theory: A survey. *Theoretical computer science*, *344*(2-3), 243–278.

[19] dos Santos, J. P. Q., de Melo, J. D., Neto, A. D. D., & Aloise, D. (2014). Reactive search strategies using reinforcement learning, local search algorithms and variable neighborhood search. *Expert Systems with Applications*, *41*(10), 4939–4949.

[20] Drias, H., Sadeg, S., & Yahi, S. (2005). Cooperative bees swarm for solving the maximum weighted satisfiability problem. *International Work-Conference on Artificial Neural Networks*, 318–325. Springer.

[21] Even-Dar, E., & Mansour, Y. (2004). Learning Rates for Q-learning. *J. Mach. Learn. Res.*, *5*, 1–25.

[22] Gambardella, L. M., & Dorigo, M. (1995). Ant-Q: A reinforcement learning approach to the traveling salesman problem. In *Machine Learning Proceedings 1995* (p. 252–260). Elsevier.

[23] Gosavi, A. (2004a). A reinforcement learning algorithm based on policy iteration for average reward: Empirical results with yield management and convergence analysis. *Machine Learning*, *55*(1), 5–29.

[24] Gosavi, A. (2004b). Reinforcement learning for long-run average cost. *European Journal of Operational Research*, *155*(3), 654–674.

[25] Gosavi, A. (2009). Reinforcement learning: A tutorial survey and recent advances. *INFORMS Journal on Computing*, *21*(2), 178–192.

[26] GOSAVII, A., Bandla, N., & Das, T. K. (2002). A reinforcement learning approach to a single leg airline revenue management problem with multiple fare classes and overbooking. *IIE transactions*, *34*(9), 729–742.

[27] Guyon, I., & Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of machine learning research*, *3*(Mar), 1157–1182.

[28] Huerta, E. B. (2008). *Logique floue et algorithmes génétiques pour le pré-traitement de données de biopuces et la sélection de gènes* (PhD Thesis). Université d'Angers.

[29] Illingworth, V. (1997). *Dictionary of computing*. Oxford University Press, Inc.

[30] Jain, A., & Zongker, D. (1997). Feature selection: Evaluation, application, and small sample performance. *IEEE transactions on pattern analysis and machine intelligence*, *19*(2), 153–158.

[31] Júnior, F. C. D. L., Neto, A. D. D., & De Melo, J. D. (2010). Hybrid metaheuristics using reinforcement learning applied to salesman traveling problem. In *Traveling Salesman Problem, Theory and Applications*. IntechOpen.

[32] Kaelbling, L. P., Littman, M. L., & Moore, A. W. (1996). Reinforcement learning: A survey. *Journal of artificial intelligence research*, *4*, 237–285.

[33] Kohavi, R., & John, G. H. (1997). Wrappers for feature subset selection. *Artificial intelligence*, *97*(1-2), 273–324.

[34] Lal, T. N., Chapelle, O., Weston, J., & Elisseeff, A. (2006). Feature Extraction: Foundations and Applications. *Studies in Fuzziness and Soft Computing, chapter Embedded methods*, 137–165.

[35] Langley, P., & Sage, S. (1994). Induction of selective Bayesian classifiers. In *Uncertainty Proceedings 1994* (p. 399–406). Elsevier.

[36] Liu, H., & Motoda, H. (2012). *Feature selection for knowledge discovery and data mining* (Vol. 454). Springer Science & Business Media.

[37] Makar, R., Mahadevan, S., & Ghavamzadeh, M. (2001). Hierarchical multi-agent reinforcement learning. *Proceedings of the fifth international conference on Autonomous agents*, 246–253. ACM.

[38] Mitchell, R. S., Michalski, J. G., & Carbonell, T. M. (2013). *An Artificial Intelligence Approach*. Springer.

[39] Nareyek, A. (2003). Choosing search heuristics by non-stationary reinforcement learning. In *Metaheuristics: Computer decision-making* (p. 523–544). Springer.

[40] Peng, J., & Williams, R. J. (1993). Efficient learning and planning within the Dyna framework. *Adaptive Behavior*, *1*(4), 437–454.

[41] Pepyne, D. L., Looze, D. P., Cassandras, C. G., & Djaferis, T. E. (1996). Application of Q-learning to elevator dispatcidng. *IFAC Proceedings Volumes*, *29*(1), 4742–4747.

[42] Pontrandolfo, P., Gosavi, A., Okogbaa, O. G., & Das, T. K. (2002). Global supply chain management: a reinforcement learning approach. *International Journal of Production Research*, *40*(6), 1299–1317.

[43] Rayward-Smith, V. J., Osman, C., Reeves, C. R., & Smith, G. D. (1996). *Modern heuristic search methods*. John Wiley.

[44] Refaeilzadeh, P., Tang, L., & Liu, H. (2009). Cross-validation. *Encyclopedia of database systems*, 532–538.

[45] Sadeg, S., Hamdad, L., Benatchba, K., & Habbas, Z. (2015). BSO-FS: bee swarm optimization for feature selection in classification. *International Work-Conference on Artificial Neural Networks*, 387–399. Springer.

[46] Sadeg, S., Hamdad, L., Remache, A. R., Karech, M. N., Benatchba, K., & Habbas, Z. (2019). QBSO-FS: A Reinforcement Learning Based Bee Swarm Optimization Metaheuristic for Feature Selection. *International Work-Conference on Artificial Neural Networks*, 785–796. Springer.

[47] Schlimmer, J. C. (1993). Efficiently inducing determinations: A complete and systematic search algorithm that uses optimal pruning. *Proceedings of the Tenth International Conference on Machine Learning*, 284–290.

[48] Shannon, C. E. (1948). A note on the concept of entropy. *Bell System Tech. J*, *27*(3), 379–423.

[49] Siedlecki, W., Siedlecka, K., & Sklansky, J. (1988). An overview of mapping techniques for exploratory pattern analysis. *Pattern Recognition*, *21*(5), 411–429.

[50] Singh, S. P., & Bertsekas, D. P. (1997). Reinforcement learning for dynamic channel allocation in cellular telephone systems. *Advances in neural information processing systems*, 974–980.

[51] Stutzle, T., & Dorigo, M. (2002). A short convergence proof for a class of ant colony optimization algorithms. *IEEE Transactions on evolutionary computation*, *6*(4), 358–365.

[52] Sutton, R. S. (1985). *Temporal Credit Assignment in Reinforcement Learning.*

[53] Sutton, R. S., & Barto, A. G. (1998). *Introduction to reinforcement learning* (Vol. 135). MIT press Cambridge.

[54] Tadepalli, P., & Ok, D. (1998). Model-based average reward reinforcement learning. *Artificial intelligence*, *100*(1-2), 177–224.

[55] Talbi, E.-G. (2013). A Unified Taxonomy of Hybrid Metaheuristics with Mathematical Programming, Constraint Programming and Machine Learning. In E.-G. Talbi (Éd.), *Hybrid Metaheuristics* (p. 3-76). https://doi.org/10.1007/978-3-642-30671-6_1

[56] Talbi, E.-G. (2016). Combining metaheuristics with mathematical programming, constraint programming and machine learning. *Annals of Operations Research*, *240*(1), 171–215.

[57] Tawhid, M. A., & Dsouza, K. B. (2018). Hybrid binary bat enhanced particle swarm optimization algorithm for solving feature selection problems. *Applied Computing and Informatics*.

[58] Van Roy, B., Bertsekas, D. P., Lee, Y., & Tsitsiklis, J. N. (1997). A neuro-dynamic programming approach to retailer inventory management. *Proceedings of the 36th IEEE Conference on Decision and Control*, *4*, 4052–4057. IEEE.

[59] Watkins, C. J., & Dayan, P. (1992). Q-learning. *Machine learning*, *8*(3-4), 279–292.

[60] Wauters, T., Verbeeck, K., De Causmaecker, P., & Vanden Berghe, G. (2013). Boosting Metaheuristic Search Using Reinforcement Learning. In E.-G. Talbi (Éd.), *Hybrid Metaheuristics* (p. 433-452). https://doi.org/10.1007/978-3-642-30671-6_17

[61] Xue, B., Zhang, M., & Browne, W. N. (2014). Particle swarm optimisation for feature selection in classification: Novel initialisation and updating mechanisms. *Applied soft computing*, *18*, 261–276.

[62] Yang, J., & Honavar, V. (1998). Feature subset selection using a genetic algorithm. In *Feature extraction, construction and selection* (p. 117–136). Springer.

[63] Yusta, S. C. (2009). Different metaheuristic strategies to solve the feature selection problem. *Pattern Recognition Letters*, *30*(5), 525–534.

[64] Zhang, W., & Dietterich, T. G. (1995). A reinforcement learning approach to job-shop scheduling. *IJCAI*, *95*, 1114–1120. Citeseer.

[65] Zheng, Z., Kohavi, R., & Mason, L. (2001). Real world performance of association rule algorithms. *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '01*, 401-406. https://doi.org/10.1145/502512.502572

Webographie

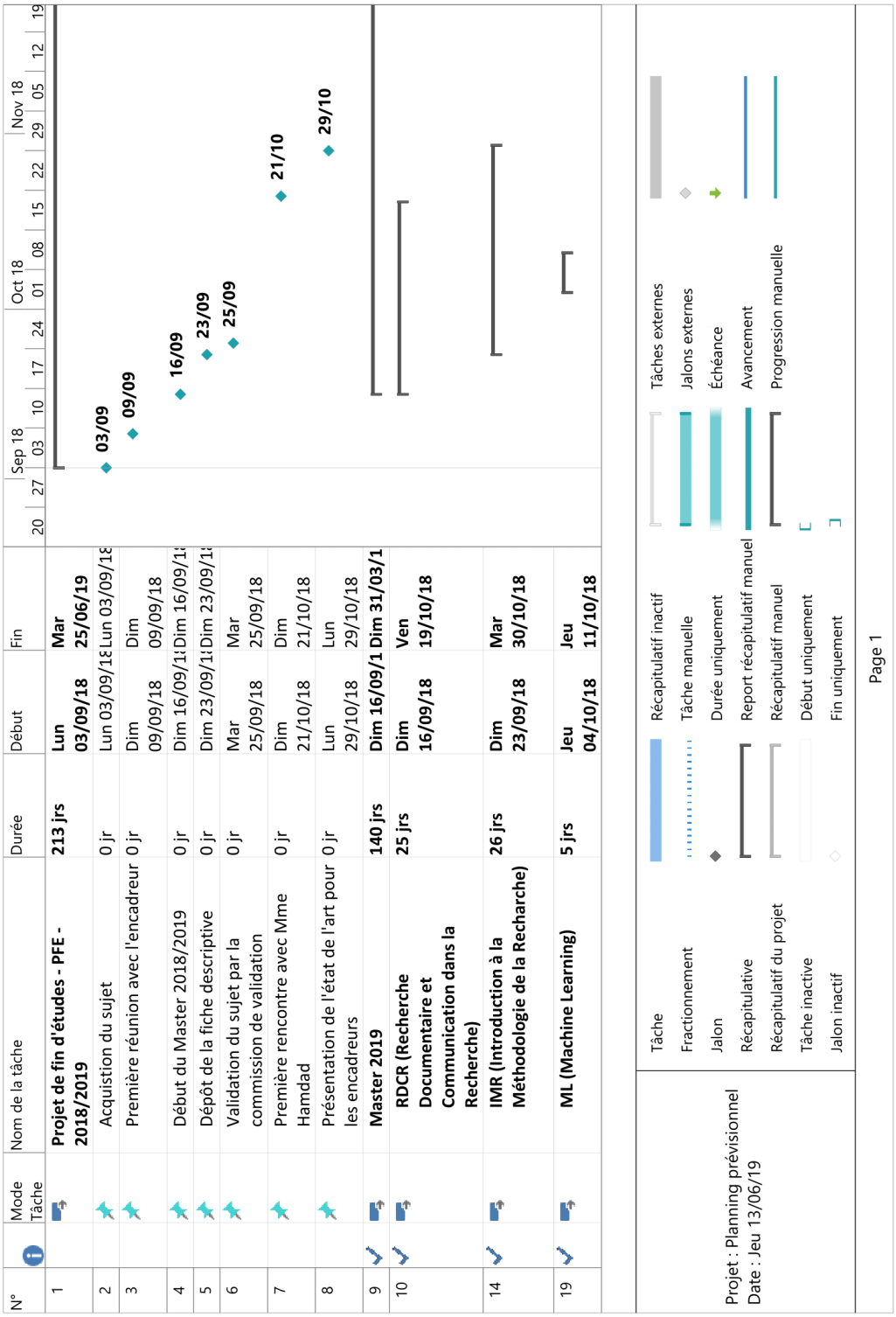
[66] CIFAR Reinforcement Learning Summer School (RLSS) 2017, Montréal

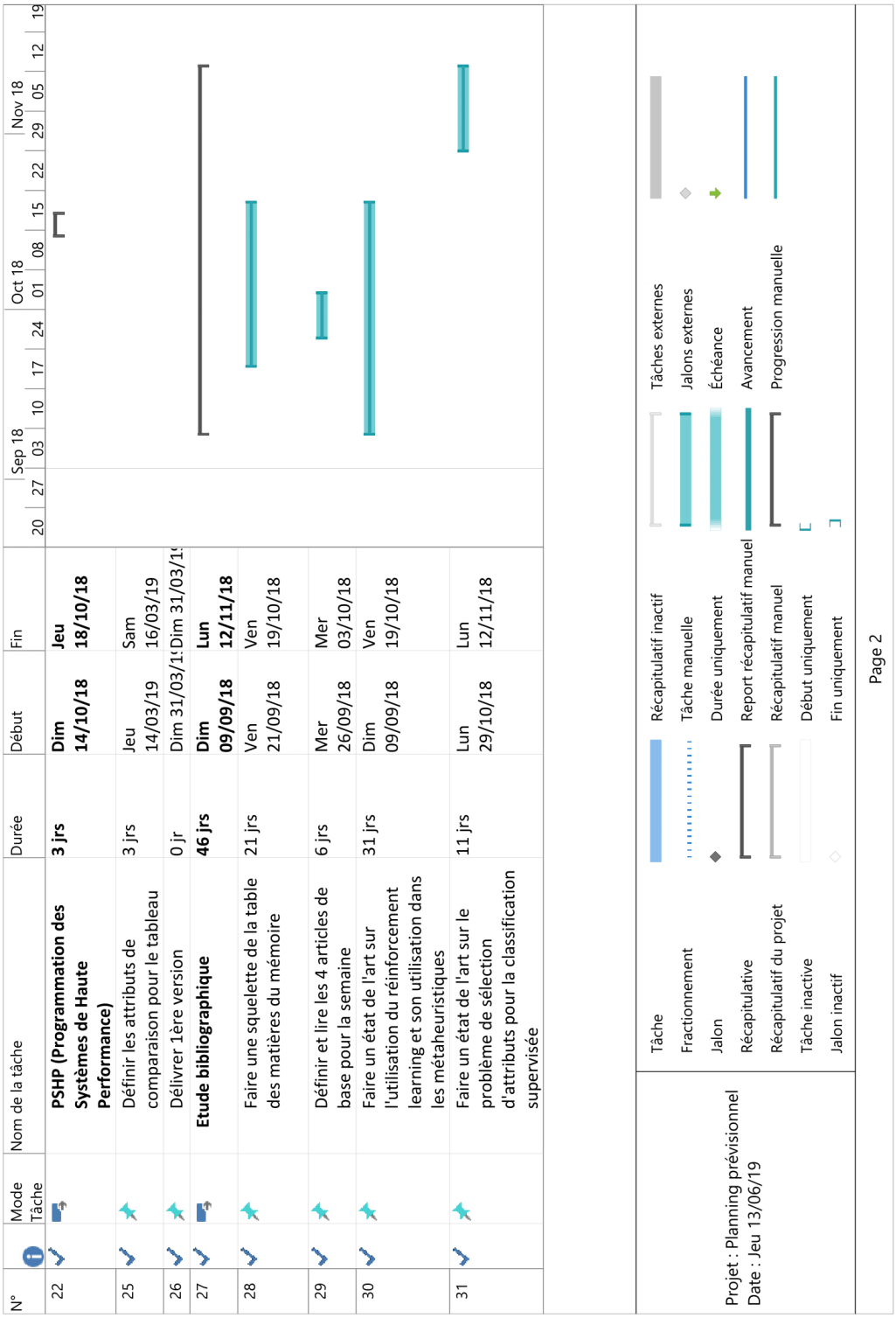
[67] UCI Machine Learning Repository

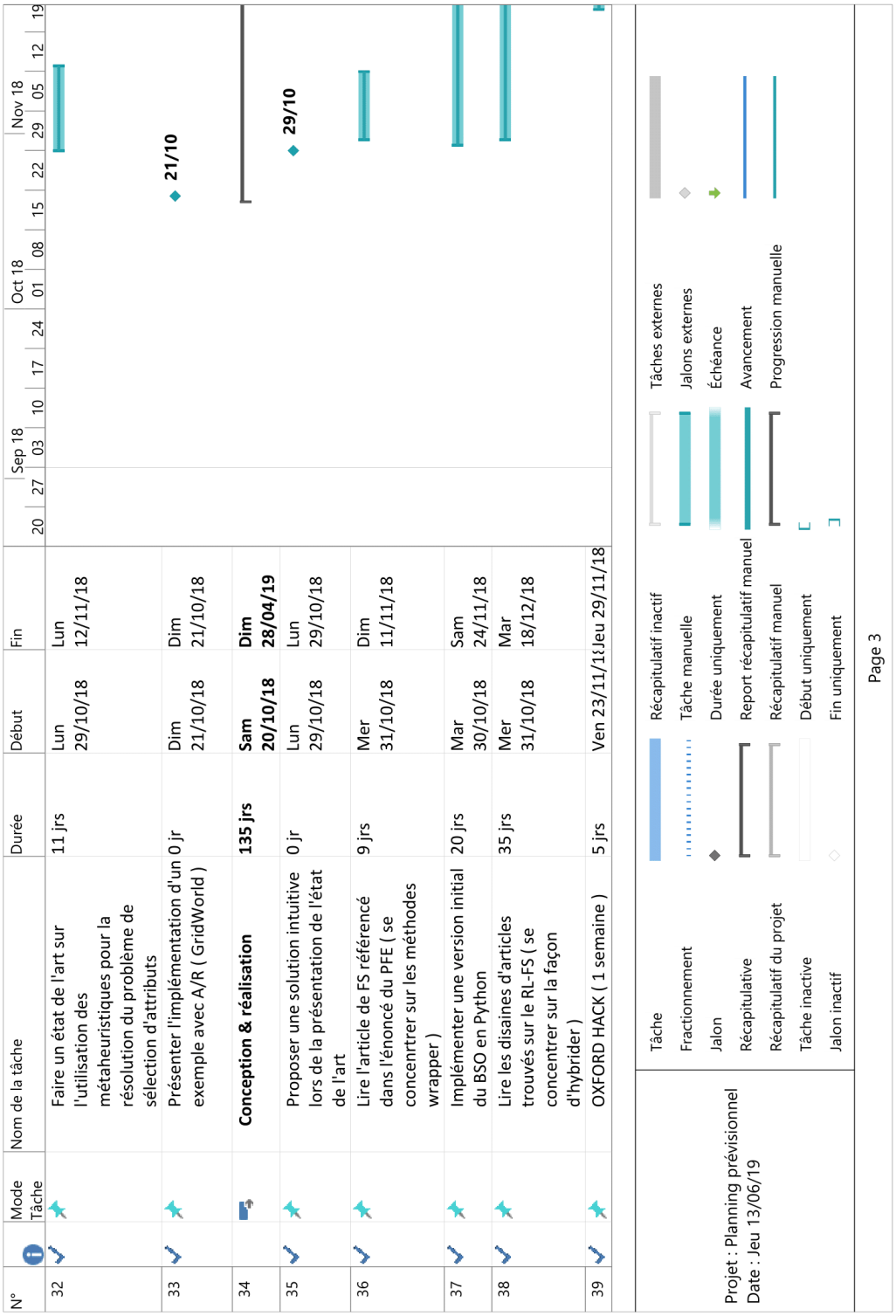
[68] Comprehensive Beginner's Guide to Jupyter Notebooks for Data Science & Machine Learning

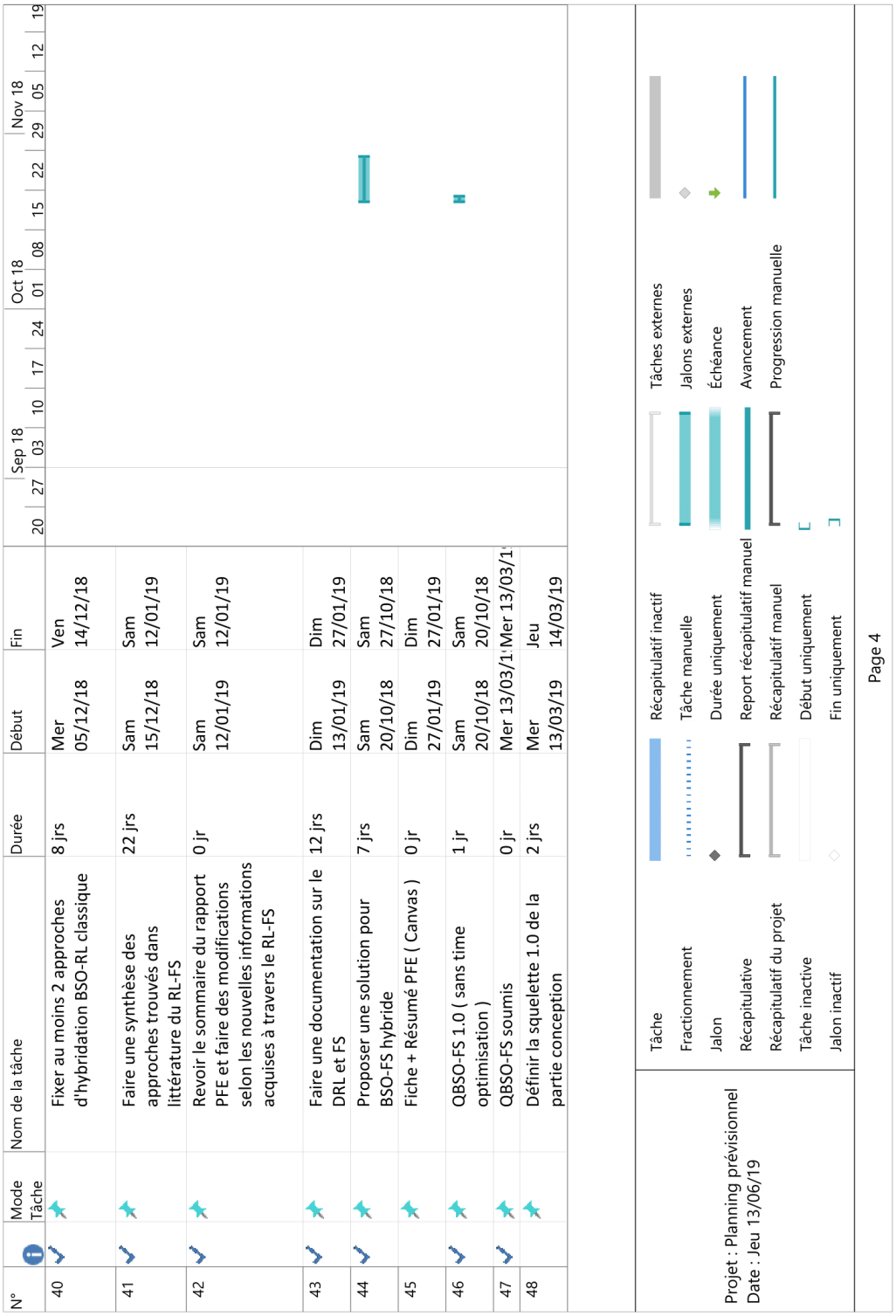
[69] WikiStat : plateforme de ressources pédagogiques

Annexe A : Gestion de projet

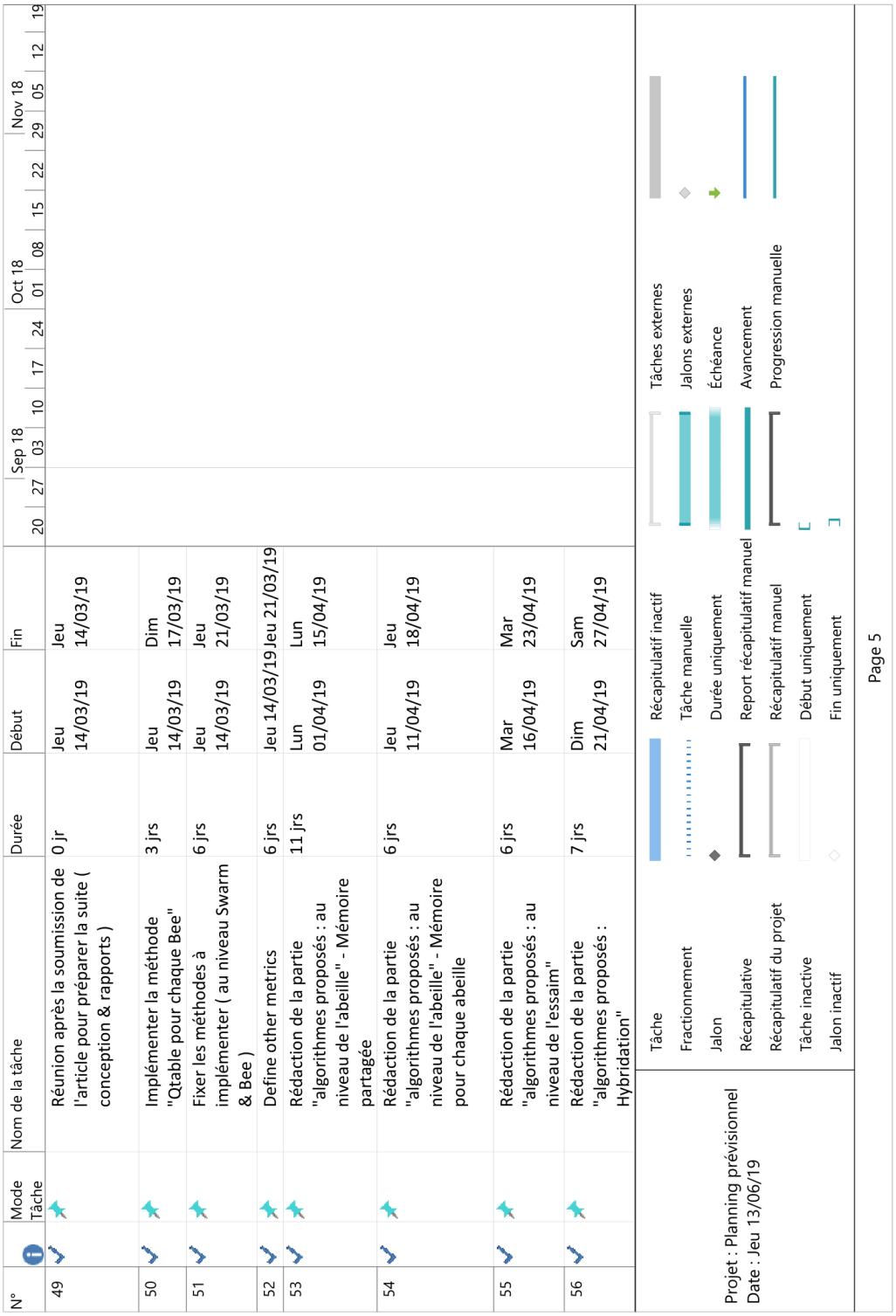



N°
Mode Tâche
Nom de la tâche
Durée
Début
Fin
22 PSHP (Programmation des Systèmes de Haute Performance) 3 jrs Dim 14/10/18 Jeu 18/10/18
25 Définir les attributs de comparaison pour le tableau 3 jrs Jeu 14/03/19 Sam 16/03/19
26 Délivrer 1ère version 0 jr Dim 31/03/1 Dim 31/03/1
27 Etude bibliographique 46 jrs Dim 09/09/18 Lun 12/11/18
28 Faire une squelette de la table des matières du mémoire 21 jrs Ven 21/09/18 Ven 19/10/18
29 Définir et lire les 4 articles de base pour la semaine 6 jrs Mer 26/09/18 Mer 03/10/18
30 Faire un état de l'art sur l'utilisation du réinforcement learning et son utilisation dans les métaheuristiques 31 jrs Dim 09/09/18 Ven 19/10/18
31 Faire un état de l'art sur le problème de sélection d'attributs pour la classification supervisée 11 jrs Lun 29/10/18 Lun 12/11/18
20 27 Sep 18 03 10 17 24 Oct 18 01 08 15 22 29 Nov 18 05 12 19
Projet : Planning prévisionnel
Date : Jeu 13/06/19
Tâche
Fractionnement
Jalon
Récapitulative
Récapitulatif du projet
Tâche inactive
Jalon inactif
Récapitulatif inactif
Tâche manuelle
Durée uniquement
Report récapitulatif manuel
Récapitulatif manuel
Début uniquement
Fin uniquement
Tâches externes
Jalons externes
Échéance
Avancement
Progression manuelle
Page 2


N°
Mode Tâche
Nom de la tâche
Durée
Début
Fin
Sep 18
Oct 18
Nov 18
20 27 03 10 17 24 01 08 15 22 29 05 12 19
32 Faire un état de l'art sur l'utilisation des métaheuristiques pour la résolution du problème de sélection d'attributs 11 jrs Lun 29/10/18 Lun 12/11/18
33 Présenter l'implémentation d'un exemple avec A/R (GridWorld) 0 jr Dim 21/10/18 Dim 21/10/18
21/10
34 Conception & réalisation 135 jrs Sam 20/10/18 Dim 28/04/19
35 Proposer une solution intuitive lors de la présentation de l'état de l'art 0 jr Lun 29/10/18 Lun 29/10/18
29/10
36 Lire l'article de FS référencé dans l'énoncé du PFE (se concenrtrer sur les méthodes wrapper) 9 jrs Mer 31/10/18 Dim 11/11/18
37 Implémenter une version initial du BSO en Python 20 jrs Mar 30/10/18 Sam 24/11/18
38 Lire les disaines d'articles trouvés sur le RL-FS (se concentrer sur la façon d'hybrider) 35 jrs Mer 31/10/18 Mar 18/12/18
39 OXFORD HACK (1 semaine) 5 jrs Ven 23/11/18 Jeu 29/11/18
Projet : Planning prévisionnel
Date : Jeu 13/06/19
Tâche
Fractionnement
Jalon
Récapitulative
Récapitulatif du projet
Tâche inactive
Jalon inactif
Récapitulatif inactif
Tâche manuelle
Durée uniquement
Report récapitulatif manuel
Récapitulatif manuel
Début uniquement
Fin uniquement
Tâches externes
Jalons externes
Échéance
Avancement
Progression manuelle
Page 3



N°	Mode Tâche	Nom de la tâche	Durée	Début	Fin
40		Fixer au moins 2 approches d'hybridation BSO-RL classique	8 jrs	Mer 05/12/18	Ven 14/12/18
41		Faire une synthèse des approches trouvés dans littérature du RL-FS	22 jrs	Sam 15/12/18	Sam 12/01/19
42		Revoir le sommaire du rapport PFE et faire des modifications selon les nouvelles informations acquises à travers le RL-FS	0 jr	Sam 12/01/19	Sam 12/01/19
43		Faire une documentation sur le DRL et FS	12 jrs	Dim 13/01/19	Dim 27/01/19
44		Proposer une solution pour BSO-FS hybride	7 jrs	Sam 20/10/18	Sam 27/10/18
45		Fiche + Résumé PFE (Canvas)	0 jr	Dim 27/01/19	Dim 27/01/19
46		QBSO-FS 1.0 (sans time optimisation)	1 jr	Sam 20/10/18	Sam 20/10/18
47		QBSO-FS soumis	0 jr	Mer 13/03/1	Mer 13/03/1
48		Définir la squelette 1.0 de la partie conception	2 jrs	Mer 13/03/19	Jeu 14/03/19


N° | Mode Tâche | Nom de la tâche | Durée | Début | Fin
49 | Réunion après la soumission de l'article pour préparer la suite (conception & rapports) | 0 jr | Jeu 14/03/19 | Jeu 14/03/19
50 | Implémenter la méthode "Qtable pour chaque Bee" | 3 jrs | Jeu 14/03/19 | Dim 17/03/19
51 | Fixer les méthodes à implémenter (au niveau Swarm & Bee) | 6 jrs | Jeu 14/03/19 | Jeu 21/03/19
52 | Define other metrics | 6 jrs | Jeu 14/03/19 | Jeu 21/03/19
53 | Rédaction de la partie "algorithmes proposés : au niveau de l'abeille" - Mémoire partagée | 11 jrs | Lun 01/04/19 | Lun 15/04/19
54 | Rédaction de la partie "algorithmes proposés : au niveau de l'abeille" - Mémoire pour chaque abeille | 6 jrs | Jeu 11/04/19 | Jeu 18/04/19
55 | Rédaction de la partie "algorithmes proposés : au niveau de l'essaim" | 6 jrs | Mar 16/04/19 | Mar 23/04/19
56 | Rédaction de la partie "algorithmes proposés : Hybridation" | 7 jrs | Dim 21/04/19 | Sam 27/04/19
Sep 18 03 10 17 24
Oct 18 01 08 15 22 29
Nov 18 05 12 19
Projet : Planning prévisionnel
Date : Jeu 13/06/19
Tâche
Fractionnement
Jalon
Récapitulative
Récapitulatif du projet
Tâche inactive
Jalon inactif
Récapitulatif inactif
Tâche manuelle
Durée uniquement
Report récapitulatif manuel
Récapitulatif manuel
Début uniquement
Fin uniquement
Tâches externes
Jalons externes
Échéance
Avancement
Progression manuelle
Page 5


N° | Mode Tâche | Nom de la tâche | Durée | Début | Fin
57 | Rédaction de la partie "Implémentation" | 3 jrs | Jeu 25/04/19 | Dim 28/04/19
58 | Tests et résultats | 128 jrs | Sam 08/12/18 | Mar 04/06/19
59 | Commencer les premiers tests | 0 jr | Sam 08/12/18 | Sam 08/12/18
60 | Faire un plan de tests formel basé sur les résultats de la littérature | 7 jrs | Dim 13/01/19 | Dim 20/01/19
61 | Exécution des premiers tests pour IWANN | 20 jrs | Ven 01/02/19 | Jeu 28/02/19
62 | Préparer le plan de test des paramètres | 4 jrs | Dim 10/03/19 | Mer 13/03/19
63 | Exécution du plan de tests | 46 jrs | Mar 26/03/1 | Mar 28/05/1
64 | Iterprétation des résultats | 5 jrs | Mer 29/05/1 | Dim 02/06/1
65 | Rédaction de la partie Tests | 6 jrs | Mer 29/05/1 | Mar 04/06/1
66 | Finalisation du mémoire de Master | 12 jrs | Dim 26/05/19 | Mar 11/06/19
67 | Finalisation du rapport | 6 jrs | Dim 26/05/1 | Ven 31/05/1
68 | Validation du rapport par les encadreurs | 3 jrs | Ven 31/05/19 | Dim 02/06/19
69 | Dépôt du mémoire | 0 jr | Dim 02/06/1 | Dim 02/06/1
70 | Soutenance du Master | 0 jr | Mar 11/06/1 | Mar 11/06/1
20 27 Sep 18 03 10 17 24 Oct 18 01 08 15 22 29 Nov 18 05 12 19
Projet : Planning prévisionnel
Date : Jeu 13/06/19
Tâche
Fractionnement
Jalon
Récapitulative
Récapitulatif du projet
Tâche inactive
Jalon inactif
Récapitulatif inactif
Tâche manuelle
Durée uniquement
Report récapitulatif manuel
Récapitulatif manuel
Début uniquement
Fin uniquement
Tâches externes
Jalons externes
Échéance
Avancement
Progression manuelle
Page 6


N°
Mode Tâche
Nom de la tâche
Durée
Début
Fin
Sep 18
Oct 18
Nov 18
71 Finalisation du mémoire PFE 11 jrs Sam 01/06/19 Dim 16/06/19
72 Finalisation du rapport 10 jrs Sam 01/06/19 Jeu 13/06/19
73 Validation du rapport par les encadreurs 3 jrs Jeu 13/06/19 Sam 15/06/19
74 Dépôt du mémoire de PFE 0 jr Dim 16/06/19 Dim 16/06/19
75 Soutenance - AP2 - 9h 1 jr Mar 25/06/19 Mar 25/06/19
Projet : Planning prévisionnel
Date : Jeu 13/06/19
Tâche
Fractionnement
Jalon
Récapitulative
Récapitulatif du projet
Tâche inactive
Jalon inactif
Récapitulatif inactif
Tâche manuelle
Durée uniquement
Report récapitulatif manuel
Récapitulatif manuel
Début uniquement
Fin uniquement
Tâches externes
Jalons externes
Échéance
Avancement
Progression manuelle
Page 7

Annexe B : Outils & notions théoriques

1. L'environnement Colaboratory de Google

Colaboratory ou Colar, est un outil (environnement de notebook Jupiter) de recherche pour la formation et la recherche associées au machine learning sous forme de plateforme web créé par Google et qui est totalement gratuit avec tous les avantages qu'il présente (deux processeurs Intel Xeon(R) 2.30 GHz et 12 Go de RAM, plus un accès à un GPU de type *NVIDIA Tesla T4* dédié au calculs des réseaux de neurones qui possède 8.1 TFLOPS et 16 Go de RAM GDDR6 et un TPU, qui est une technologie purement de Google et qui est conçue pour exécuter des modèles de machine learning de pointe avec des services d'IA sur Google Cloud). Colaboratory fonctionne avec la plupart des principaux navigateurs comme Chrome, Firefox et Safari. La différence **entre Jupyter et Colaboratory est que** Jupyter est le projet Open Source sur lequel Colaboratory est basé. Colaboratory vous permet d'utiliser et de partager les notebooks Jupyter avec d'autres personnes sans avoir besoin de télécharger, d'installer ou d'exécuter quoi que ce soit sur votre propre ordinateur, à l'exception d'un navigateur. Tous les notebooks Colaboratory sont stockés dans Google Drive et peuvent être partagés comme les documents Google Docs ou Sheets.

Parmi les avantages qu'offre Colab :

- La disponibilité gratuite de n'importe quel type de processeurs (CPU, GPU et TPU)
- Facile à collaborer avec les services Google
- Pas besoin de préparer un environnement pour écrire et exécuter du code python

2. Analyse de variance (ANOVA)

Les techniques dites d'analyse de variance sont des outils entrant dans le cadre général du modèle linéaire et où une variable quantitative est expliquée par une ou plusieurs variables qualitatives. L'objectif essentiel est alors de comparer les moyennes empiriques de la variable quantitative observées pour différentes catégories d'unités statistiques. Ces catégories sont définies par l'observation des variables qualitatives ou facteurs prenant différentes modalités ou encore de variables quantitatives découpées en classes ou niveaux. Une combinaison de niveaux définit une cellule, groupe ou traitement.

Il s'agit donc de savoir si un facteur ou une combinaison de facteurs (interaction) a un effet sur la variable quantitative en vue, par exemple, de déterminer des conditions optimales de production ou de fabrication, une dose optimale de médicaments … Ces techniques apparaissent aussi comme des cas particuliers de la régression linéaire multiple en associant à chaque modalité une variable indicatrice (dummy variable) et en cherchant à expliquer une variable quantitative par ces variables indicatrices. L'appellation "analyse de variance" vient de ce que les tests statistiques sont bâtis sur des comparaisons de sommes de carrés de variations.

L'analyse de variance est souvent utilisée pour analyser des données issue d'une planification d'expérience au cours de laquelle l'expérimentateur a la possibilité de contrôler a priori les niveaux des facteurs avec pour objectif d'obtenir le maximum de précision au moindre coût. Ceci conduit en particulier à construire des facteurs orthogonaux deux à deux (variables explicatives non linéairement corrélées) afin de minimiser la variance des estimateurs. [69]

2.1. Modèle à un facteur

Cette situation est un cas particulier d'étude de relations entre deux variables statistiques : une quantitative Y admettant une densité et une qualitative T ou facteur qui engendre une partition ou classification de l'échantillon en J groupes, cellules ou classes indicées par j. L'objectif est de comparer les distributions de Y pour chacune des classes en particulier les valeurs des moyennes et variances.

Un préalable descriptif consiste à réaliser un graphique constitué de boites à moustaches parallèles : une pour chaque modalité. Cette représentation donne une première appréciation de la comparaison des distributions (moyenne, variance) internes à chaque groupe.

2.1.1. Modèles

Pour chaque niveau j de T, on observe n_j valeurs y_{1j} , … , y_{nj} de la variable Y et où $n = \sum_{j=1}^{J} n_j$ est la taille de l'échantillon ($n > J$). On suppose qu'à l'intérieur de chaque cellule, les observations sont indépendantes équi-distribuées de moyenne μ_j et de variance homogène $\sigma_j^2 = \sigma^2$.

Ceci s'écrit :

$$y_{ij} = \mu_j + \varepsilon_{ij}$$

Où les ε_{ij} sont i.i.d. suivant une loi centrée de variance σ^2 qui sera supposée $N\ (0, \sigma^2)$ pour la construction des tests. Cette dernière hypothèse n'étant pas la plus sensible. Les espérances μ_j ainsi que le paramètre de nuisance σ^2 sont les paramètres inconnus à estimer.
On note respectivement :

$$\begin{aligned} \bar{y}_{.j} &= \frac{1}{n_j}\sum_{i=1}^{n_j} y_{ij}, \\ s_j^2 &= \frac{1}{n_j - 1}\sum_{i=1}^{n_j}(y_{ij} - \bar{y}_{.j})^2, \\ \bar{y}_{..} &= \frac{1}{n}\sum_{i=1}^{n_j}\sum_{j=1}^{J} y_{ij}, \end{aligned}$$

Les moyennes et variances empiriques de chaque cellule, la moyenne générale de l'échantillon.
Les paramètres μ_j sont estimés sans biais par les moyennes $\overline{y_{.j}}$ et comme le modèle s'écrit alors :

$$y_{ij} = \bar{y}_{.j} + (y_{ij} - \bar{y}_{.j}).$$

L'estimation des erreurs est $e_{ij} = (y_{ij} - \bar{y}_{.j})$ tandis que les valeurs prédites sont $\widehat{y}_{ij} = \bar{y}_{.j}$

Sous l'hypothèse d'homogénéité des variances, la meilleure estimation sans biais de σ^2 est

$$s^2 = \frac{\sum_{j=1}^{J}\sum_{i=1}^{n_j}(y_{ij} - \bar{y}_{.j})^2}{n - J} = \frac{1}{n - J}[(n-1)s_1^2 + \cdots + (n_J - 1)s_J^2]$$

Qui s'écrit donc comme une moyenne pondérée des variances empiriques de chaque groupe.

Notons $\boldsymbol{y}$ le vecteur des observations $[y_{ij}|i = 1, n_j; j = 1, J]'$ mis en colonne, $\mathbf{u} = [\varepsilon_{ij}|i = 1, n_j; j = 1, J]'$ le vecteur des erreurs, $\mathbf{1}_j$ les variables indicatrices des niveaux et **1** la colonne de 1s. Le $i$ème élément d'une variable indicatrice (dummy variable) $\mathbf{1}_j$ prend la valeur 1 si la $i$ème observation y_i est associée au $j$ème et 0 sinon.

Comme dans le cas de la régression linéaire multiple, le modèle consiste à écrire que l'espérance de la variable Y appartient au sous-espace linéaire engendré par les variables explicatives, ici les variables indicatrices :

$$\mathbf{y} = \beta_0\mathbf{1} + \beta_1\mathbf{1}_1 + \cdots + \beta_J\mathbf{1}_J + \mathbf{u}$$

La matrice $\boldsymbol{X}$ alors construite n'est pas de plein rang $p + 1$ mais de rang p. La matrice $\boldsymbol{X'X}$ n'est pas inversible et le modèle admet une infinité de solutions. Nous disons que les paramètres β_j ne sont pas *estimables* ou identifiables. En revanche, certaines fonctions (combinaisons linéaires) de ces paramètres sont estimables et appelées *contrastes*.
Dans le cas du modèle d'analyse de variance à *un* facteur, la solution la plus simple adoptée consiste à considérer un sous-ensemble des indicatrices ou de combinaisons des indicatrices de façon à aboutir à une matrice inversible. Ceci conduit à considérer différents modèles associés à différentes paramétrisation. *Attention*, les paramètres β_j ainsi que la matrice $\boldsymbol{X}$ prennent à chaque fois des significations différentes.
Un premier modèle (cell means model) s'écrit comme celui d'une régression linéaire multiple sans terme constant avec $\boldsymbol{\beta} = [\mu_1, \ldots, \mu_J]'$ le vecteur des paramètres :

$$\begin{aligned} \mathbf{y} &= \beta_1\mathbf{1}_1 + \cdots + \beta_J\mathbf{1}_J + \mathbf{u} \\ \mathbf{y} &= \mathbf{X}\boldsymbol{\beta} + \mathbf{u}. \end{aligned}$$

Les calculs se présentent simplement mais les tests découlant de ce modèle conduiraient à étudier la nullité des paramètres alors que nous sommes intéressés par tester l'égalité des moyennes.
Une autre paramétrisation, considérant cette fois le vecteur $\boldsymbol{\beta} = [\mu_J, \mu_1 - \mu_J, \ldots, \mu_{J-1} - \mu_J]'$
Conduit à écrire le modèle (base cell model) de régression avec terme constant :

$$\mathbf{y} = \beta_0\mathbf{1} + \beta_1\mathbf{1}_1 + \cdots + \beta_{J-1}\mathbf{1}_{J-1} + \mathbf{u}$$

C'est celle de SAS alors que Systat considère des paramètres d'effet différentiel $\mu_j - \mu_.$ par rapport à l'effet moyen $\mu_. = 1/J \sum_{j=1}^{J} \mu_j$. Ce dernier est encore un modèle (group effect model) de régression linéaire avec terme constant mais dont les variables explicatives sont des différences d'indicatrices et avec $\boldsymbol{\beta} = [\mu_., \mu_1 - \mu_., \ldots, \mu_{J-1} - \mu_.]'$:

$$\mathbf{y} = \beta_0\mathbf{1} + \beta_1(\mathbf{1}_1 - \mathbf{1}_J) + \cdots + \beta_{J-1}(\mathbf{1}_{J-1} - \mathbf{1}_J) + \mathbf{u}.$$

2.1.2. Test

On désigne les différentes sommes des carrés des variations par :

$$\begin{aligned}
\text{SST} &= \sum_{j=1}^{J}\sum_{i=1}^{n_j}(y_{ij}-\bar{y}_{..})^2 = \sum_{j=1}^{J}\sum_{i=1}^{n_j} y_{ij}^2 - n\bar{y}_{..}^2, \\
\text{SSW} &= \sum_{j=1}^{J}\sum_{i=1}^{n_j}(y_{ij}-\bar{y}_{.j})^2 = \sum_{j=1}^{J}\sum_{i=1}^{n_j} y_{ij}^2 - \sum_{j=1}^{J} n_j\bar{y}_{.j}^2, \\
\text{SSB} &= \sum_{j=1}^{J} n_j(\bar{y}_{.j}-\bar{y}_{..})^2 = \sum_{j=1}^{J} n_j\bar{y}_{.j}^2 - n\bar{y}_{..}^2,
\end{aligned}$$

Où "**T**" signifie totale, "**W**" (within) intra ou résiduelle, "**B**" (between) inter ou expliquée par la partition. Il est facile de vérifier que **SST=SSB+SSW**.

On considère alors l'hypothèse :

$$H_0 : \mu_1 = \cdots = \mu_J,$$

Qui revient à dire que la moyenne est indépendante du niveau ou encore que le facteur n'ait pas d'effet, contre l'hypothèse

$$H_1 : \exists(j,k) \text{ tel que } \mu_j \neq \mu_k$$

Qui revient à reconnaître un effet ou une influence du facteur sur la variable Y .

Dans les modèles précédents, l'étude de cette hypothèse revient à comparer par un test de Fisher un modèle complet (les moyennes sont différentes) avec un modèle réduit supposant la nullité des paramètres β_j et donc l'égalité des moyennes à celle de la dernière cellule ou à la moyenne générale.

Les résultats nécessaires à la construction du test qui en découle sont résumés dans la table d'analyse de la variance :

Source de variation	*d.d.l.*	*Somme des carrés*	*Variance*	*F*
Modèle (inter)	$J-1$	SSB	MSB=SSB/$(J-1)$	MSB/MSW
Erreur (intra)	$n-J$	SSW	MSW=SSW/$(n-J)$	
Total	$n-1$	SST		

Pratiquement, un programme de régression usuel permet de construire estimation et test de la nullité des β_j sauf pour le premier modèle qui doit tester l'égalité au lieu de la nullité des paramètres.

Dans le cas de deux classes ($J = 2$) on retrouve un test équivalent au test de Student de comparaison des moyennes de deux échantillons indépendants.

3. Le classifieur KNN

Pour notre travail nous avons utilisé un classifieur très adéquat pour notre problème, qui est la sélection d'attributs pour la classification supervisée. Ce classifieur s'appelle KNN.

3.1. Définitions

En intelligence artificielle, la méthode des k plus proches voisins est une méthode d'apprentissage supervisé. En abrégé k-NN ou KNN, de l'anglais k-nearest neighbors.

Dans ce cadre, on dispose d'une base de données d'apprentissage constituée de N couples (entrée-sortie). Pour estimer la sortie associée à une nouvelle entrée x, la méthode des k plus proches voisins consiste à prendre en compte (de façon identique) les k échantillons d'apprentissage dont l'entrée est la plus proche de la nouvelle entrée x, selon une distance à définir. (Altman, 1992)

Une mesure de distance couramment utilisée pour les variables continues est la distance euclidienne. Pour les variables discrètes, telles que la classification, une autre métrique peut être utilisée, telle que la distance de Hamming.

Pour rappel, la distance euclidienne est définie ainsi :

$$D_E = \sqrt{\sum_{i=1}^{k} (x_i - y_i)^2}$$

Et la distance de Hamming est donnée selon la formule suivante :

$$D_H = \sum_{i=1}^{k} |x_i - y_i|$$

$$x = y \Rightarrow D = 0$$

$$x \neq y \Rightarrow D = 1$$

KNN peut être utilisé pour la classification - la sortie est une appartenance à une classe (prédit une classe - une valeur discrète). Un objet est classé à la majorité des voix de ses voisins, l'objet étant attribué à la classe la plus courante parmi ses k voisins les plus proches. Il peut également être utilisé pour la régression – la sortie est la valeur de l'objet (prédit des valeurs continues). Cette valeur est la moyenne (ou médiane) des valeurs de ses k plus proches voisins.

Le pseudo algorithme de KNN :

1. Un entier positif k est spécifié, avec un nouvel échantillon
2. Nous sélectionnons les k individus de notre population qui sont les plus proches du nouvel échantillon
3. Nous trouvons la classification la plus commune de ces entrées
4. C'est la classification que nous donnons au nouvel échantillon

Exemple :

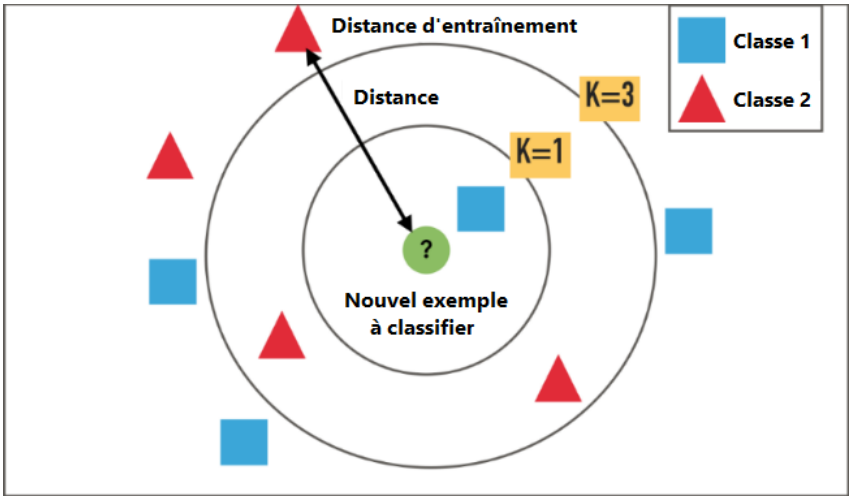


Figure 0-01 - Exemple de l'algorithme KNN

L'échantillon à tester (cercle vert intérieur) doit être classé soit dans la première classe de carrés bleus, soit dans la deuxième classe de triangles rouges. Si k = 3 (cercle extérieur), il est attribué à la deuxième classe car il y a 2 triangles et 1 seul carré à l'intérieur de ce cercle. Si, par exemple, k = 1, il est attribué à la première classe (1 carré contre 0 triangles).

3.2. Avantages de KNN

- Aucune hypothèse requise sur les données - utile, par exemple, pour les données non linéaires
- Algorithme simple - à expliquer et à comprendre ou interpréter
- Haute exactitude (relativement) - elle est assez élevée par rapport à d'autres modèles d'apprentissage supervisé
- Polyvalent - utile pour la classification ou la régression

3.3. KNN dans notre contexte

Comme nous venons de mentionner, KNN est un classifieur très adapté au problème de classification, et pour notre cas, nous avons pris la valeur de $K = 1$, appelé 1-NN, pour les tests de nos métaheuristiques car c'est le classificateur KNN le plus intuitif, qui attribue un point x à la classe de son plus proche voisin dans l'espace des voisins et lorsque la taille de l'ensemble de données d'apprentissage se rapproche de l'infini, ce classifieur garantit un taux d'erreur égal ou moins que le double du taux d'erreur de Bayes (le taux d'erreur minimum réalisable étant donné la distribution des données).

4. Validation croisée

La validation croisée (*cross-validation*, en Anglais) est, en apprentissage automatique, une méthode d'estimation de fiabilité d'un modèle fondé sur une technique d'échantillonnage.
Supposons posséder un modèle statistique avec un ou plusieurs paramètres inconnus, et un ensemble de données d'apprentissage sur lequel on peut entraîner le modèle. Le processus d'apprentissage optimise les paramètres du modèle afin que celui-ci corresponde aux données le mieux possible. Si on prend ensuite un échantillon de validation indépendant issu de la même population d'entraînement, il s'avérera en général que le modèle ne réagit pas aussi bien à la validation que durant l'entraînement : on parle parfois de surapprentissage. La validation croisée est un moyen de prédire l'efficacité d'un modèle sur un ensemble de validation hypothétique lorsqu'un ensemble de validation indépendant et explicite n'est pas disponible. (Besse & Ferré, 1993).

4.1. Surapprentissage

En statistique et en apprentissage automatique, le surapprentissage ou surajustement ou surinterprétation (*overfitting*, en Anglais) est une analyse statistique qui correspond trop étroitement ou exactement à un ensemble particulier de données. Ainsi, cette analyse peut ne pas correspondre à des données supplémentaires ou ne pas prévoir de manière fiable les observations futures. Un modèle surajusté est un modèle statistique qui contient plus de paramètres que ne peuvent le justifier les données.

4.2. Techniques de validation croisée

Le but majeur de la validation croisée étant de prévenir le surapprentissage, il existe au moins trois variantes (Bengio & Grandvalet, 2004):

- « *Testset validation* » ou « *holdout method* » : on divise l'échantillon de taille $\boldsymbol{n}$ en deux sous-échantillons, le premier dit d'apprentissage (communément supérieur à 60 % de l'échantillon) et le second dit de test. Le modèle est bâti sur l'échantillon d'apprentissage et validé sur l'échantillon de test. L'erreur est estimée en calculant un test, une mesure ou un score de performance du modèle sur l'échantillon de test, par exemple l'erreur quadratique moyenne.

- « *K-fold cross-validation* » : on divise l'échantillon original en $\boldsymbol{k}$ échantillons, puis on sélectionne un des $\boldsymbol{k}$ échantillons comme ensemble de validation et les $\boldsymbol{k-1}$ autres échantillons constitueront l'ensemble d'apprentissage. On calcule comme dans la première méthode le score de performance, puis on répète l'opération en sélectionnant un autre échantillon de validation parmi les $\boldsymbol{k-1}$ échantillons qui n'ont pas encore été utilisés pour la validation du modèle. L'opération se répète ainsi $\boldsymbol{k}$ fois pour qu'en fin de compte chaque sous-échantillon ait été utilisé exactement une fois comme ensemble de validation. La moyenne des $\boldsymbol{k}$ erreurs quadratiques moyennes est enfin calculée pour estimer l'erreur de prédiction.

- « *Leave-one-out cross-validation* » (**LOOCV**) : cas particulier de la deuxième méthode où $k = n$, c'est-à-dire que l'on apprend sur $\boldsymbol{n-1}$ observations puis on valide le modèle sur la énième observation et l'on répète cette opération $\boldsymbol{n}$ fois.

5. Classification des métaheuristiques hybrides

L'hybridation des métaheuristiques soulève deux types de problèmes : conception et implémentation. La conception concerne l'algorithme d'hybridation lui-même, son fonctionnement et son architecture. L'implémentation inclut la plate-forme matérielle, le langage de programmation et l'environnement sur lequel l'algorithme doit être exécuté. Dans cette section nous allons présenter la classification des métaheuristiques hybrides selon (Talbi, 2013)

5.1. Classification selon les problèmes liés à la conception

5.1.1. Classification hiérarchique

La figure suivante présente l'hybridation de bas niveau (Low-level) aborde la composition fonctionnelle d'une seule méthode d'optimisation. Dans cette sous-classe un fonctionnement interne donné d'une métaheuristique est remplacé par une autre méthode d'optimisation. Dans les algorithmes hybrides de haut niveau, les différentes méthodes d'optimisation sont autonomes. Il n'y a pas de relation directe avec le fonctionnement interne d'une métaheuristique. Dans l'hybridation relais (Relay), un ensemble de métaheuristiques est appliqué l'une après l'autre, chacune utilisant la sortie de la précédente comme entrée, agissant comme un pipeline. L'hybridation en équipe (Team work) représente la coopération des modèles d'optimisation dans lesquels de nombreux agents coopèrent en parallèle, chaque agent effectue une recherche dans un espace de solutions et peut coopérer avec les autres agents.

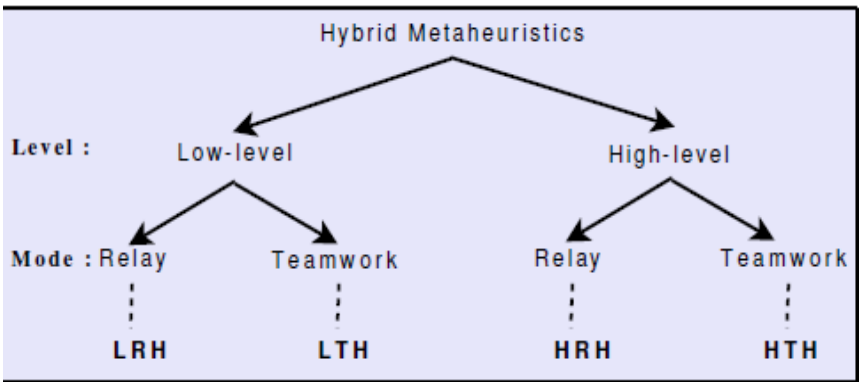


Figure 0-2 - Classification hiérarchique

A. Métaheuristiques hybrides relais de bas niveau (LRH (Low-level Relay Hybrid))

Cette branche représente des algorithmes dans lesquels une métaheuristique donnée est intégré à une métaheuristiques-S (métaheuristique à solution unique) comme la recherche locale ou le recuit simulé.

B. Métaheuristiques hybrides relais de bas niveau (LRH (Low-level Relay Hybrid))

Ces métaheuristiques sont construites sur l'intégration des métaheuristiques-S (métaheuristiques à solution unique) dans les méthodes basées sur population pour assurer la diversification et l'intensification en même temps.

C. Métaheuristiques hybrides relais de haut niveau HRH (High-level Relay Hybrid)

Dans cette approche les métaheuristiques autonomes sont exécutées en séquence, en effet une solution initiale d'une métaheuristique donnée peut être générée par un autre algorithme d'optimisation. Un schéma de combinaison bien connu consiste à générer la solution initiale par une métaheuristique gourmande.

D. Métaheuristiques hybrides HTH (High-level Teamwork Hybrid)

Ce schéma pousse plusieurs algorithmes autonomes à effectuer une recherche en parallèle et coopérer pour trouver un optimum global. Intuitivement, HTH finira par au moins trouver la meilleure solution des algorithmes en parallèle.

5.1.2. Classification plate

A. Homogène/Hétérogène

Dans les hybridations homogènes, tous les algorithmes combinés utilisent la même métaheuristique, ces algorithmes différents dans les initialisations de leurs :

- **Paramètres** : En général, différents paramètres sont utilisés pour les algorithmes. Par exemple, dans le schéma HTH hybride basé sur la recherche tabou, les algorithmes peuvent être initialisés avec des listes tabou de différentes tailles.
- **Composants de la recherche** : Dans le cas d'une métaheuristique, on peut utiliser différentes stratégies pour chaque composant de la recherche (solution initiale, critère d'arrêt …etc.).

Dans les hybridations hétérogènes plusieurs métaheuristiques sont combinées.

B. Globale/Partielle

Dans les méthode hybrides globales, tous les algorithmes explorent le même espace de recherche. Le but est d'explorer l'espace de manière plus approfondie, un algorithme HTH basé sur une recherche taboue a été proposé dans (Crainic, Toulouse, & Gendreau, 1995), où chaque tâche de la recherche taboue effectue un nombre donné d'itérations, puis diffuse la meilleure solution. La meilleure des solutions devient la solution initiale pour la phase suivante.

Dans les méthodes hybrides partielles le problème à résoudre est décomposé en sous problème où chaque problème a son propre espace de recherche pour lequel un algorithme est appliqué, tous les sous-problèmes sont liés, entraînant ainsi des contraintes entre les optimums locaux trouvés par chaque algorithme. Ainsi, les algorithmes communiquent pour respecter ces contraintes et construire une solution globale acceptable au problème.

C. Générale/Spécifique

Les méthodes générales ont toutes pour but de trouver une solution optimale à un seul problème, tandis que la méthode spécifique combine des algorithmes pour trouver des solutions de plusieurs problèmes à la fois.

La figure suivante regroupe les deux classifications précédentes :

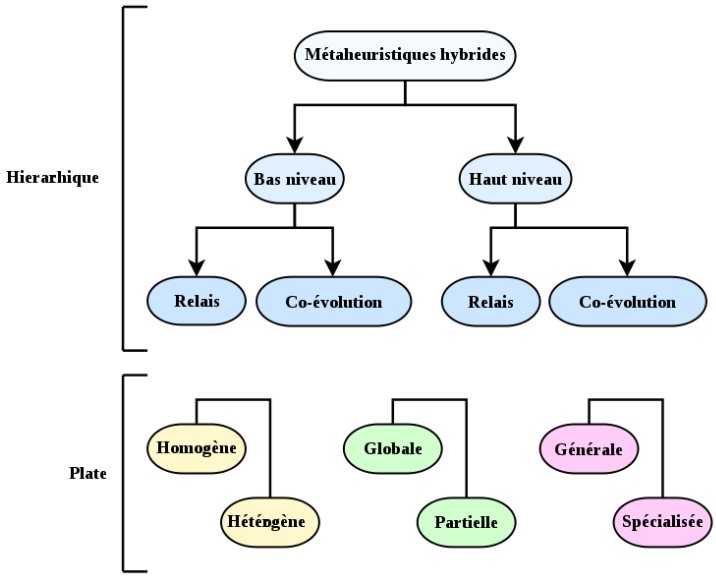


Figure 0-3 - Taxinomie des métaheuristiques hybrides selon les problèmes de conception

5.2. Classification selon les problèmes liés à l'implémentation

La structure de la taxonomie liée à l'implémentation est présentée dans la figure suivante :

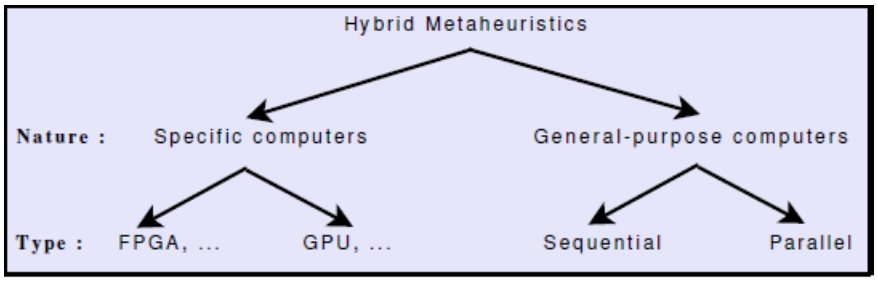


Figure 0-4 - Taxonomie des problèmes liés à l'implémentation

Les ordinateurs dédiés diffèrent des ordinateurs à usage général car ils ne résolvent habituellement qu'une petite gamme de problèmes, mais avec une plus grande vitesse et avec un cout minimal. Leur structure interne est adaptée pour un problème particulier, ce qui leur permet d'obtenir une efficacité et une utilisation du matériel bien supérieures à celles processeurs qui doivent gérer un grand nombre de tâches les FPGA et les GPU sont des exemples de calculateurs dédiés.
La majorité des métaheuristiques hybrides s'exécutent d'une manière séquentielle, En fonction de la taille des problèmes, des implémentations parallèles d'algorithmes hybrides ont été envisagées. Une facilité d'implémentation a été remarqué pour les méthodes HTH.

5.3. Selon les problèmes liés aux approches de programmation mathématique

5.3.1. Méthodes énumératives

Cette classe d'algorithmes contient des algorithmes de recherche arborescente tels que Branch and Bound et la programmation dynamique. Ils sont basés sur une stratégie de division de l'espace de recherche en sous-espaces, puis en optimisant individuellement chaque sous-espace.

5.3.2. Méthodes de relaxation et de décomposition

Basées sur les méthodes de relaxation qui ont pour but d'assouplir une exigence stricte dans le problème d'optimisation comme la relaxation de Lagrange (dont le concept est de supprimer certaines contraintes et de les incorporer dans la fonction objectif. Pour chaque contrainte, une fonction de pénalité est associée), et les approches de décomposition comme la décomposition de Bender qui fixe les valeurs des variables compliquées et résout le problème itérativement.

5.3.3. Méthodes de branch & cut & price

Cette classe d'algorithmes est basée sur la combinatoire polyédrique dans lequel l'espace de recherche est élagué.

5.4. Métaheuristiques hybridées au Machine Learning

Ce schéma d'hybridation peut être considéré comme une extraction de connaissances et une intégration de ces connaissances dans une métaheuristique. Cette connaissance peut prendre différentes formes comme le montre la figure :

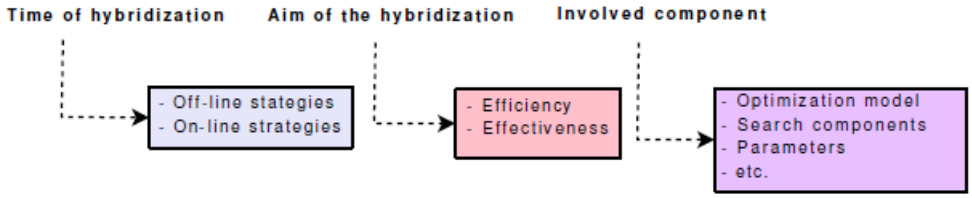


Figure 0-5 - Méthodes d'intégration de la connaissance dans une métaheuristique

Moment de l'extraction de la connaissance (Time of knowledge extraction) : On peut distinguer deux types d'hybridations selon le moment d'extraction de la connaissance, les hybridations qui extraient la connaissance avant le début de la recherche sont appelées stratégies de connaissance off-line et les combinaisons où la connaissance est extraite dynamiquement au cours de la recherche connu sous le nom des stratégies d'extraction de connaissances on-line.

But de l'hybridation (Aim of the hybridization) : L'hybridation peut se faire pour améliorer la diversification ou l'intensification, ou bien les deux à la fois.

Composant impliqué : L'apprentissage peut se faire dans n'importe quel composant de la recherche tel que l'encodage des solutions, les solutions initiales, les opérateurs de variation de la recherche (comme le voisinage par exemple) ou pour fixer les paramètres de l'algorithme.